

RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT
SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE



RAPPORT DU PROJET DE FIN D'ÉTUDES

POUR L'OBTENTION DU DIPLOME NATIONAL D'INGÉNIEUR
EN SYSTÈMES EMBARQUÉS ET IoT

Réalisé par
Doua GHRIS

Développement d'un outil d'investigation des causes de crash
des systèmes embarqués



ACTIA Aeronautic Services Tunisie

Jury

<i>Président :</i>	M. Prénom NOM
<i>Rapporteur :</i>	M. Prénom NOM
<i>Encadrant académique :</i>	M. Lotfi TLIG
<i>Encadrant industriel :</i>	M. Youssef CHAARI

Année universitaire : 2025–2026



DÉDICACE

Je souhaite dédier ce travail

*À mes chers parents,
en témoignage de leur amour, de leurs sacrifices et de leur soutien sans faille.*

*À mon frère et à mes sœurs,
pour l'affection, les encouragements et la présence qu'ils m'ont toujours accordés.*

*À toute ma famille,
en reconnaissance de sa bienveillance et de ses prières.*

*À mes amis,
pour leur appui constant et tous les précieux moments vécus ensemble.*

*À mes enseignants et à mes encadrants,
en témoignage de ma gratitude pour leurs conseils et leur accompagnement.*

*À toutes les personnes qui ont cru en moi,
j'adresse l'expression de ma plus profonde reconnaissance.*

Doua Ghriss



REMERCIEMENTS

En premier lieu, j'exprime ma gratitude à Dieu, le Tout-Puissant, qui m'a donné la santé, la force et la détermination indispensables à l'aboutissement de ce travail.

Je remercie très sincèrement M. Youssef CHAARI, mon encadrant industriel, pour la qualité de son suivi, sa grande disponibilité et la pertinence de ses recommandations, qui ont guidé ma progression durant toute la réalisation de ce projet.

Ma reconnaissance va également à M. Mahmoud FENDRI, Mme Nourelhouda NCIRI et Mme Amina CHERIF, dont la disponibilité, les conseils et la contribution ont favorisé le bon déroulement de cette étude.

Je tiens aussi à témoigner ma gratitude à M. Amir FAKHFAKH, mon manager, pour la confiance qu'il m'a accordée, son soutien, son encadrement et ses encouragements tout au long de cette expérience professionnelle.

J'exprime mes remerciements les plus respectueux à M. Lotfi TLIG, mon encadrant académique, pour sa disponibilité, la justesse de ses conseils, son accompagnement attentif et ses encouragements, qui ont largement contribué à la réussite de ce travail.

Je remercie par ailleurs l'ensemble de l'équipe d'ACTIA Aeronautic Services pour la qualité de son accueil, sa disponibilité et le cadre professionnel particulièrement enrichissant qu'elle m'a offert pendant cette expérience.

Pour terminer, j'adresse toute ma reconnaissance aux enseignants de l'Institut Supérieur d'Informatique et de Multimédia de Gabès (ISIMG), dont les enseignements m'ont apporté une formation solide et précieuse au cours de mon parcours universitaire.



RÉSUMÉ ET ABSTRACT

Résumé

Chez ACTIA Aeronautic Services Tunisie, l'analyse des crashes applicatifs sur les systèmes Linux embarqués ARM64 demeure largement manuelle avec GDB. Cette pratique exige des compétences avancées et produit difficilement des diagnostics homogènes. Ce projet propose donc *Crash Analyzer*, une chaîne locale automatisant l'analyse post-mortem afin de rendre l'investigation plus simple et reproductible.

La solution comprend trois composants. *Crash_Simulator* provoque des défaillances contrôlées, puis *Crash_Analyzer* exploite les fichiers core avec GDB en mode batch pour extraire le signal, le backtrace et les registres, déterminer une cause probable et proposer une correction. Enfin, l'interface Streamlit *Dashboard_Creator* centralise les diagnostics et facilite leur consultation et leur comparaison.

Entièrement hors ligne et sans instrumentation de l'application, l'architecture protège les données industrielles. Les composants, d'abord construits pour x86_64, ont été portés sur ARM64 et validés sur Raspberry Pi 4. La campagne finale réunit 51 incidents associés à SIGSEGV, SIGABRT, SIGFPE, SIGBUS, SIGILL et SIGTRAP. Les résultats confirment la faisabilité d'un diagnostic reproductible et accessible, tout en indiquant les améliorations nécessaires avant l'industrialisation.

Mots-clés : systèmes embarqués, Linux, crash applicatif, fichier core, GDB, diagnostic post-mortem, ARM64.

Abstract

at ACTIA Aeronautic Services Tunisia, crashes on ARM64 embedded Linux systems are still largely investigated manually with GDB. This requires advanced expertise and makes consistent diagnoses difficult to achieve. This project therefore introduces *Crash Analyzer*, a local pipeline that automates post-mortem analysis and improves investigation reproducibility.

The solution comprises three components. *Crash_Simulator* triggers controlled failures, while *Crash_Analyzer* processes the resulting core files with GDB in batch mode to extract the signal, backtrace, and registers, identify a probable cause, and recommend a correction. Lastly, the Streamlit interface *Dashboard_Creator* centralizes the diagnoses for review and comparison.

Operating fully offline without application instrumentation, the architecture protects industrial data. Initially built for x86_64, the components were ported to ARM64 and validated on a Raspberry Pi 4. The final campaign covered 51 incidents involving SIGSEGV, SIGABRT, SIGFPE, SIGBUS, SIGILL, and SIGTRAP. The results confirm that reproducible and accessible diagnosis is feasible while highlighting the improvements required before industrial deployment.

Keywords : embedded systems, Linux, crash, core dump, GDB, post-mortem diagnosis, ARM64.

TABLE DES MATIÈRES

DÉDICACE	i
REMERCIEMENTS	ii
RÉSUMÉ ET ABSTRACT	iii
LISTE DES FIGURES	vi
LISTE DES TABLEAUX	vii
LISTE DES ABRÉVIATIONS	viii
INTRODUCTION GÉNÉRALE	x
1 Cadre général du projet	1
1.1 Introduction	2
1.2 Présentation de l'organisme d'accueil	2
1.3 Contexte et problématique du projet	5
1.4 Méthodologie de travail	12
1.5 Planification du travail	13
2 Étude des besoins et conception de la solution	16
2.1 Introduction	17
2.2 Notions fondamentales sur les systèmes Linux embarqués	17
2.3 Analyse des besoins	22
2.4 Conception de la solution	24
2.5 Architecture logicielle des modules	25
2.6 Diagramme d'activité global	29
2.7 Conception des modules	31
2.8 Interactions entre modules	36
3 Réalisation, intégration et validation	38
3.1 Introduction	39
3.2 Environnement de travail	40
3.3 Développement du module Crash_Simulator	42
3.4 Développement du module Crash_Analyzer	46
3.5 Développement du Dashboard_Creator avec Streamlit	52
3.6 Pipeline de déploiement sur la cible embarquée	58
3.7 Analyse critique des limites constatées	65
CONCLUSION GÉNÉRALE	66
BIBLIOGRAPHIE	68
ANNEXES	72
A.1 Glossaire technique	72
A.2 Interfaces textuelles du projet	72
A.3 Environnement logiciel	73
A.4 Captures complémentaires du déploiement ARM64	74
A.5 Liste des signaux standards sous Linux	75
A.6 Interprétation des principaux registres au moment du crash	76

LISTE DES FIGURES

1.1	Logo du Groupe ACTIA	2
1.2	Implantation internationale du Groupe ACTIA	3
1.3	Secteurs d'activité et domaines d'expertise du Groupe ACTIA	4
1.4	Architecture en couches de la solution <i>Crash Analyzer</i>	10
1.5	Cycle de développement itératif et incrémental appliqué au projet <i>Crash_Analyzer</i>	13
1.6	Diagramme de Gantt du projet	14
1.7	Code couleur des phases du projet	15
2.1	Architecture en couches d'un système Linux embarqué	18
2.2	Principe de la compilation croisée vers une cible ARM64 (aarch64)	19
2.3	Communication entre l'espace utilisateur, le matériel et le noyau Linux lors d'un crash applicatif .	20
2.4	Structure et contenu d'un fichier core au format ELF	21
2.5	Localisation de la faute et backtrace extraits d'un fichier core par GDB	21
2.6	Registres, instruction fautive et thread actif extraits par GDB lors d'un SIGSEGV	22
2.7	Architecture globale de la solution <i>Crash Analyzer</i> et flux entre les espaces utilisateur, noyau et matériel	24
2.8	Fonctionnement général et flux de données du module <i>Crash_Simulator</i>	26
2.9	Acheminement des données à travers le module <i>Crash_Analyzer</i>	26
2.10	Acheminement des données à travers le module <i>Dashboard_Creator</i>	27
2.11	Diagramme d'activité global de la solution <i>Crash Analyzer</i>	30
2.12	Diagramme de classes du module <i>Crash_Analyzer</i>	31
2.13	Diagramme de séquence – analyse d'un dossier de fichiers core	33
2.14	Diagramme de classes du module <i>Crash_Simulator</i>	35
2.15	Diagramme de séquence – scénario nominal de production d'un crash	36
2.16	Diagramme de déploiement UML de la solution entre le PC hôte et le Raspberry Pi 4	37
3.1	Flux global, du crash applicatif à la visualisation du diagnostic	39
3.2	Représentation de l'historique des commits sur GitHub et organisation des branches par sprint . .	41
3.3	Diagramme de cas d'utilisation du Sprint 1 — module <i>Crash_Simulator</i>	43
3.4	Configuration et vérification des paramètres de production des fichiers core	44
3.5	Compilation native en mode Debug du module <i>Crash_Simulator</i> avec CMake	44
3.6	Exécution des derniers scénarios et bilan de la campagne du <i>Crash_Simulator</i>	45
3.7	Exécution de <i>Crash_Analyzer</i> sur un lot de fichiers core générés par le <i>Crash_Simulator</i> — détection des signaux et types de crash	45
3.8	Diagramme de cas d'utilisation du Sprint 2 — analyse, classification et export des fichiers core . .	47
3.9	Structure JSON d'un rapport unitaire — crash SIGABRT avec backtrace détaillé et correctif généré par <i>SolutionProposer</i>	51
3.10	Campagne d'analyse d'un lot de fichiers core — progression, classification et production des rapports	52
3.11	Diagramme de cas d'utilisation du Sprint 3 — <i>Dashboard_Creator</i>	53
3.12	Vue synthétique du <i>Dashboard_Creator</i> : configuration, statistiques et répartition des crashes . .	54
3.13	Tableau filtrable des incidents dans le <i>Dashboard_Creator</i>	55
3.14	Analyse détaillée d'un crash : code source, localisation et stack trace	56
3.15	Fonctions principales du <i>Crash Assistant</i> : explication et proposition de correction	57
3.16	Séquence de déploiement et d'exécution sur la cible Raspberry Pi 4	59
3.17	Cross-compilation ARM64 du module <i>Crash_Analyzer</i>	60
3.18	Exécution du script <i>deploy_all.sh</i> — vérification des binaires ARM64, création des répertoires et transfert vers le Raspberry Pi 4	61
3.19	Configuration des fichiers core sur Raspberry Pi — définition de <i>core_pattern</i> et préparation du répertoire de stockage	61
3.20	Exécution de l'application de test <i>unique_crash_app</i> sur Raspberry Pi — validation de signaux complémentaires au <i>Crash_Simulator</i>	62

3.21	Génération contrôlée d'un fichier core par <i>Crash_Simulator</i> sur Raspberry Pi	62
3.22	Exécution d'applications de test issues de GitHub — production de fichiers core à la suite de dépassements de tampon sur ARM64	63
A.1	Cross-compilation ARM64 du module <i>Crash_Simulator</i>	74
A.2	Transfert des résultats et lancement de leur visualisation	74
A.3	Vues complémentaires du <i>Crash Assistant</i>	74

LISTE DES TABLEAUX

1.1	Comparaison des principales solutions de gestion de crash et positionnement de la solution proposée	7
1.2	Architecture en couches de la solution <i>Crash Analyzer</i>	10
1.3	Comparaison et justification des choix architecturaux et technologiques	11
2.1	Besoins fonctionnels des trois modules de la solution	22
2.2	Besoins non fonctionnels et méthodes de validation	23
2.3	Synthèse des composants de l'architecture globale	25
2.4	Justification des principaux choix de l'architecture logicielle	28
2.5	Synthèse des classes du module <i>Crash_Analyzer</i>	32
2.6	Synthèse des classes du module <i>Crash_Simulator</i>	35
3.1	Caractéristiques des machines utilisées dans le projet	40
3.2	Versions des outils de développement utilisés	40
3.3	Synthèse des choix technologiques et justifications	42
3.4	Backlog du Sprint 1 — module <i>Crash_Simulator</i>	42
3.5	Options de compilation CMake du module <i>Crash_Simulator</i>	44
3.6	Résultats des tests fonctionnels du module <i>Crash_Simulator</i>	46
3.7	Backlog du Sprint 2 — module <i>Crash_Analyzer</i>	47
3.8	Règles de classification appliquées par <i>PatternMatcher</i>	49
3.9	Exemples de règles <i>SolutionProposer</i> — explications et correctifs générés	50
3.10	Comparaison des formats d'export CSV, JSON et SQLite	50
3.11	Résultats de la phase de test du Sprint 2 — détection des signaux par <i>PatternMatcher</i>	51
3.12	Backlog du Sprint 3 — <i>Dashboard_Creator</i>	53
3.13	Résultats de validation fonctionnelle du <i>Dashboard_Creator</i>	58
3.14	Backlog du Sprint 4 — déploiement embarqué	58
3.15	Vérification des exigences non fonctionnelles — résultats mesurés	64
3.16	Analyse critique des principales limites observées pendant la validation	65
A.1	Glossaire des principaux termes techniques	72
A.2	Environnement logiciel et outils utilisés dans le projet	73
A.3	Liste des signaux standards sous Linux et leur signification	75
A.4	Interprétation des principaux registres au moment du crash	76



LISTE DES ABRÉVIATIONS

AAST	ACTIA Aeronautic Services Tunisie
ARM	Advanced RISC Machines
ARM64	Architecture ARM 64 bits
BNF	Besoin Non Fonctionnel
CPU	Central Processing Unit
CSV	Comma-Separated Values
ELF	Executable and Linkable Format
GDB	GNU Debugger
GNU	GNU's Not Unix
IDE	Integrated Development Environment
IQCP	Ingénierie, Qualification et Certification du Produit
JSON	JavaScript Object Notation
LTS	Long-Term Support
NVMe	Non-Volatile Memory Express
PC	Personal Computer
PID	Process Identifier
POSIX	Portable Operating System Interface
RAM	Random Access Memory
RISC	Reduced Instruction Set Computer
SCP	Secure Copy Protocol
SDK	Software Development Kit
SIGABRT	Signal Abort
SIGBUS	Signal Bus Error
SIGFPE	Signal Floating-Point Exception
SIGILL	Signal Illegal Instruction
SIGSEGV	Signal Segmentation Violation
SIGTRAP	Signal Trace/Breakpoint Trap
SQL	Structured Query Language
SSH	Secure Shell

TID Thread Identifier

UML Unified Modeling Language

US User Story



INTRODUCTION GÉNÉRALE

Les systèmes Linux embarqués occupent une place importante dans les équipements industriels, car ils associent les services d'un système d'exploitation généraliste aux contraintes de ressources, de fiabilité et de connectivité propres aux plateformes dédiées [13]. Chez ACTIA Aeronautic Services Tunisie (AAST), les campagnes de validation de logiciels exécutés sur des cibles Linux ARM64 peuvent toutefois révéler des arrêts inattendus. Lorsqu'un processus se termine brutalement et que le système est correctement configuré, le noyau Linux peut enregistrer son état dans un fichier *core*, dont la production dépend notamment de la limite `ulimit -c` et du paramètre `kernel.core_pattern` [6].

Ce fichier conserve des informations essentielles pour rechercher l'origine de la défaillance, notamment le signal reçu, l'état des registres et les données nécessaires à la reconstruction de la pile d'appels. Son exploitation reste toutefois généralement manuelle : l'ingénieur doit retrouver l'exécutable exact, l'associer au core, lancer GDB, lire le *backtrace* et interpréter le contexte d'exécution. Bien que GDB puisse automatiser une partie de ces opérations grâce à son mode batch et à ses commandes non interactives [7], l'analyse complète exige encore une expertise avancée, mobilise du temps et produit des résultats difficiles à uniformiser.

Cette dépendance à l'interprétation individuelle représente un frein dans un contexte industriel où les anomalies doivent être comprises, reproduites et tracées avant la qualification d'un logiciel. Une analyse lente retarde la correction et la reprise des essais, tandis qu'un rapport incomplet complique le suivi de l'incident et la capitalisation des connaissances. Le besoin dépasse donc la simple ouverture d'un fichier core : il revient à transformer automatiquement les informations techniques disponibles en un diagnostic structuré, compréhensible et réutilisable. Cette réflexion conduit à la problématique suivante :

Comment concevoir et valider une chaîne locale, portable et non intrusive, capable d'analyser automatiquement les fichiers core d'applications Linux embarquées ARM64 et de restituer des diagnostics structurés, compréhensibles et traçables, sans modifier l'application ni dépendre d'un service externe ?

Pour répondre à cette problématique, le projet a pour objectif de développer *Crash Analyzer*, une chaîne complète d'analyse post-mortem. La démarche commence par la production de crashes contrôlés et reproductibles, puis automatise l'extraction des informations fournies par GDB. Sur la base de ces données, la solution classe la cause probable avec un niveau de confiance, formule une explication et propose une piste de correction. Le diagnostic est ensuite historisé et décrit dans une interface donnant la possibilité de retrouver et de comparer les incidents.

Ces responsabilités sont réparties entre trois modules complémentaires. *Crash_Simulator* reproduit dix-huit catégories de crashes dans un environnement maîtrisé dans le but de fournir des fichiers core représentatifs. *Crash_Analyzer* prend ensuite en charge leur analyse, structure les informations extraites et exporte les rapports aux formats CSV, JSON et SQLite. Enfin, *Dashboard_Creator* exploite ces rapports pour offrir une consultation filtrée et interactive des diagnostics. Le passage d'un module au suivant repose ainsi sur des artefacts persistants, ce qui limite leur couplage et simplifie leur utilisation indépendante.

Ce choix architectural distingue la solution des plateformes centralisées de remontée d'erreurs. L'ensemble fonctionne hors ligne et conserve les données dans l'environnement de validation. Il n'impose ni SDK, ni gestionnaire de crash intégré, ni surveillance permanente, puisque l'analyse intervient après l'incident à partir du fichier core produit nativement par Linux. La solution complète ainsi les capacités techniques de GDB par une classification, une historisation et une visualisation homogènes. Son apport principal revient à rendre le diagnostic plus reproductible et plus accessible, tout en limitant les contraintes de déploiement et en préservant la confidentialité des données.

Afin de vérifier ces choix, la chaîne a d'abord été construite sur x86_64, puis portée vers ARM64 et déployée sur Raspberry Pi 4. Une campagne de fichiers core couvrant les principaux signaux POSIX a permis d'évaluer le fonctionnement des trois modules et la cohérence des diagnostics produits. Cette validation apporte le fil conducteur du mémoire : le premier chapitre détaille le contexte industriel, les solutions existantes et le positionnement du projet. Le deuxième formalise les besoins et décrit l'architecture retenue, tandis que le troisième expose la réalisation, le déploiement sur ARM64 et les résultats de validation. La conclusion générale revient enfin sur les apports obtenus, les limites observées et les perspectives d'évolution de la solution.

Cadre général du projet

Sommaire

1.1	Introduction	2
1.2	Présentation de l'organisme d'accueil	2
1.2.1	Présentation du Groupe ACTIA	2
1.2.2	Présentation d'ACTIA Aeronautic Services Tunisie	3
1.2.3	Secteurs d'activité et structure des départements	3
1.3	Contexte et problématique du projet	5
1.3.1	Solutions existantes	5
1.3.2	Limites des solutions existantes	7
1.3.3	Problématique	8
1.3.4	Positionnement du projet	9
1.3.5	Solution proposée	9
1.3.6	Objectifs du projet	11
1.3.7	Justification des choix architecturaux	11
1.4	Méthodologie de travail	12
1.5	Planification du travail	13

1.1 Introduction

Le présent chapitre expose le cadre général dans lequel s'inscrit le projet de fin d'études. Il s'ouvre sur une présentation du groupe ACTIA et de l'organisme d'accueil, ACTIA Aeronautic Services Tunisie (AASST), ainsi que des activités directement liées au contexte du projet.

Le chapitre expose ensuite le contexte industriel du diagnostic des crashes applicatifs dans les systèmes Linux embarqués et analyse les principales solutions existantes. Cette étude donne la possibilité d'identifier leurs limites vis-à-vis des contraintes du projet, notamment en matière de fonctionnement local, de confidentialité, d'absence d'instrumentation et d'exploitation des fichiers core sur une architecture ARM64.

Sur la base de cette analyse, la problématique et le positionnement du projet sont définis, puis la solution *Crash Analyzer* et ses principaux objectifs sont présentés. Les choix architecturaux et technologiques retenus sont ensuite justifiés avant de présenter la planification adoptée pour la réalisation du projet.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation du Groupe ACTIA

ACTIA Group, dont le logo est décrit dans la figure 1.1, est une entreprise internationale fondée en 1986 et basée à Toulouse, en France. Le groupe intervient principalement dans les secteurs de l'automobile, de l'aéronautique et des télécommunications. Grâce à son expertise, ACTIA participe à de nombreux projets technologiques liés au développement logiciel, aux systèmes embarqués et aux systèmes de communication, tout en proposant des solutions adaptées aux exigences industrielles et aux évolutions technologiques [1].



FIGURE 1.1 – Logo du Groupe ACTIA

Le Groupe ACTIA bénéficie aujourd'hui d'une forte présence internationale, avec plusieurs filiales, centres de production et centres de recherche répartis dans divers pays à travers le monde. Il totalise 21 sociétés implantées dans 15 pays et une présence commerciale dans plus de 140 pays, comme le détaille la figure 1.2, qui recense les principaux pays d'implantation et les filiales associées à chacun.

En Tunisie, ACTIA est présente à travers plusieurs entités complémentaires :

- CIPI ACTIA est un site industriel intégré au groupe depuis 1997 et spécialisé dans les services de fabrication électronique. Il assure la production en grande série de modules électroniques destinés notamment à la télématique, à l'électronique de puissance, aux ordinateurs embarqués et aux systèmes domotiques ;
- ACTIA Engineering Services est un centre de recherche et développement du groupe, spécialisé dans la conception et le développement de produits, de systèmes embarqués et de solutions logicielles. Créée en 2005 sous le nom d'ARDIA, cette entité regroupe également des activités de qualification et d'ingénierie ;



FIGURE 1.2 – Implantation internationale du Groupe ACTIA

- ACTIA Tunisie est une filiale industrielle du Groupe, créée en 2008 et spécialisée dans l'intégration et le test de produits électroniques destinés notamment à l'automobile ;
- ACTIA Aeronautic Services Tunisie (AASST) est une filiale de recherche et développement du Groupe ACTIA en Tunisie, fondée en 2025 et intégrant progressivement l'aéronautique comme nouvelle activité stratégique.

1.2.2 Présentation d'ACTIA Aeronautic Services Tunisie

ACTIA Aeronautic Services Tunisie (AASST) est l'organisme d'accueil au sein duquel ce projet de fin d'études a été mené. Filiale du Groupe ACTIA implantée au Technopôle de Sfax, l'entreprise est spécialisée dans le développement de solutions logicielles et de systèmes embarqués destinés aux secteurs aéronautique, spatial, automobile et industriel.

Ses activités couvrent notamment les logiciels embarqués, les systèmes connectés et les solutions critiques à forte valeur technologique. L'entreprise évolue ainsi dans un environnement exigeant qui requiert un haut niveau de qualité, de fiabilité et d'innovation. Elle poursuit également le développement de ses activités dans le but de renforcer son expertise et d'accompagner les évolutions technologiques de son secteur.

Dans cet environnement, la traçabilité des anomalies logicielles occupe une place essentielle. Les campagnes de validation menées sur des cibles Linux ARM64 peuvent produire des crashes dont l'analyse exige la manipulation de fichiers core et de débogueurs spécialisés. Ce besoin représente l'origine directe du projet *Crash Analyzer*.

1.2.3 Secteurs d'activité et structure des départements

ACTIA intervient principalement dans les secteurs de l'automobile, du transport et de l'aéronautique, tout en élargissant progressivement ses domaines d'activité dans le but de diversifier son expertise technologique.

L'entreprise participe à des projets à forte valeur ajoutée liés aux systèmes embarqués, au développement logiciel, à la mécatronique ainsi qu'à la validation des systèmes électroniques. La figure 1.3 expose les principaux secteurs d'activité du Groupe ACTIA ainsi que ses domaines d'expertise technologique.



FIGURE 1.3 – Secteurs d'activité et domaines d'expertise du Groupe ACTIA

L'entreprise est organisée autour de plusieurs départements spécialisés :

- le département Ingénierie, qualification et certification du produit (IQCP), chargé d'assurer la conformité des produits aux normes internationales ;
- le département de développement matériel, responsable de la conception mécanique, du support industriel, des tests fonctionnels et du suivi de fabrication ;
- le département des systèmes embarqués, dédié à la conception et au développement de logiciels embarqués ;
- le département de développement logiciel, chargé de la conception, du développement et de la maintenance des applications spécifiques ;
- le département du diagnostic automobile, spécialisé dans le développement d'outils de diagnostic pour les constructeurs automobiles ;
- le département de validation, responsable des activités de vérification et de validation des systèmes matériels et logiciels développés.

Le projet se situe à l'interface de ces compétences : il a pour objectif de simplifier l'investigation des crashes rencontrés pendant la validation de logiciels embarqués, sans modifier l'application analysée ni transférer les données techniques vers une infrastructure externe.

1.3 Contexte et problématique du projet

Dans les secteurs automobile et aéronautique, un crash peut interrompre la validation, retarder une livraison et accroître le coût de correction. Les équipes doivent donc identifier rapidement sa cause et capitaliser les diagnostics.

Ce projet automatise l'analyse post-mortem sur Linux embarqué. Après un arrêt inattendu, le noyau peut produire un *core dump* représentant l'état du processus. La solution transforme cette donnée, jusque-là étudiée manuellement, en diagnostic reproductible et partagé.

Pour déterminer dans quelle mesure ce besoin est déjà couvert, la sous-section suivante examine les principales solutions disponibles et leur adéquation avec un environnement Linux embarqué.

1.3.1 Solutions existantes

Plusieurs solutions de *crash reporting* existent dans l'industrie du logiciel. Sentry, Breakpad et Crashpad représentent trois approches courantes. Elles sont étudiées ici de manière comparative dans le but de préciser leur adéquation avec le contexte embarqué d'ACTIA.

Sentry est une plateforme de supervision applicative et de remontée centralisée des erreurs [2]. Son fonctionnement repose généralement sur un SDK intégré à l'application surveillée. Lorsqu'une erreur est détectée, le SDK collecte la pile d'appels, la version logicielle et d'autres informations contextuelles, puis les communique à un serveur Sentry.

Sentry offre une interface Web, regroupe les incidents et simplifie leur suivi à grande échelle. Une instance auto-hébergée peut réduire la dépendance à un fournisseur externe, mais elle ne supprime ni l'intégration du SDK dans l'application ni le besoin d'un serveur joignable pour centraliser les événements. Ce modèle est pertinent pour la supervision continue d'applications connectées; il répond moins bien à un banc isolé où l'application validée ne doit pas être modifiée et où les fichiers core doivent rester dans l'environnement interne.

Breakpad est une bibliothèque open source initialement développée par Google [3]. Elle produit des fichiers *minidumps*, ensuite analysés à l'aide de symboles de débogage préalablement extraits. Cette solution est gratuite, relativement légère et adaptée aux applications natives C/C++.

Son intégration requiert toutefois un gestionnaire de crash dans l'application et une chaîne de symbolisation propre aux *minidumps*. En compilation croisée vers ARM64, la correspondance entre chaque version du binaire, ses symboles et les rapports générés doit être maintenue avec rigueur. Breakpad résout donc efficacement la collecte compacte d'un incident, mais pas l'ensemble du besoin étudié : il ne classe pas la cause probable, ne propose pas d'explication adaptée à l'ingénieur et ne fournit pas de tableau de bord intégré.

Crashpad représente l'évolution moderne de Breakpad et est maintenu par le projet Chromium [4]. Il s'appuie sur un processus séparé, le *handler*, chargé de surveiller l'application et de collecter les rapports. Ce découpage améliore la robustesse de la collecte lorsque l'application principale devient instable.

Cette robustesse entraîne toutefois une complexité de déploiement supplémentaire. Le processus *handler* doit être intégré, configuré et supervisé, tandis que les symboles doivent toujours être gérés pour chaque version et chaque architecture. Crashpad convient ainsi aux

produits qui doivent capturer les défaillances en exploitation, y compris lorsque le processus principal devient instable. Dans le présent projet, la priorité est différente : analyser à la demande un core Linux déjà produit, sans ajouter de composant permanent à la configuration testée.

Linux fournit aussi `systemd-coredump` et `coredumpctl` pour acquérir, stocker et retrouver les cores et leurs métadonnées [21]. Ces outils représentent une base de collecte robuste, mais ils laissent à l'ingénieur l'interprétation du signal, du backtrace et de la cause. Ils ne couvrent donc ni la classification, ni l'explication, ni la restitution synthétique recherchées dans ce projet.

a) Analyse critique des travaux récents

Les recherches récentes confirment que la pile d'appels seule ne suffit pas toujours à identifier une cause. Les approches de *crash bucketing* exploitent la sémantique du programme [17], les graphes de flot et la cause racine [18], l'information mutuelle [19] ou l'apprentissage profond [20]. Elles sont particulièrement utiles lorsqu'il faut regrouper rapidement un volume massif de rapports. Leur efficacité dépend toutefois de données d'apprentissage, de traces d'exécution, de transformations du programme ou d'un corpus suffisamment riche. Ces prérequis sont disproportionnés pour une première chaîne locale qui doit produire un diagnostic déterministe, explicable et utilisable avec un nombre limité d'incidents de validation.

D'autres travaux se rapprochent davantage du besoin de diagnostic. *CrashTalk* combine analyses statique et dynamique pour produire des descriptions lisibles de crashes de sécurité [14]. *Alligator in Vest* et *FirmRCA* montrent l'intérêt des informations d'exécution et de l'analyse causale sur ARM, mais s'appuient sur des mécanismes matériels, des traces ou une réexécution orientée micrologiciels [15, 25]. Enfin, *LEMIX* déplace des applications embarquées vers Linux x86 dans le but de simplifier leur analyse dynamique [26] ; cette stratégie améliore les essais, mais ne remplace pas l'investigation du core réellement produit sur la cible Linux ARM64. Ces travaux récents inspirent la classification et l'explication proposées, sans représenter des solutions directement déployables dans le contexte du stage.

Cette lecture distingue donc trois niveaux complémentaires : la collecte de l'incident, bien couverte par Sentry, Breakpad, Crashpad ou `systemd-coredump` selon le contexte ; l'analyse avancée, approfondie par les travaux de recherche mais souvent coûteuse en données et en instrumentation ; et la chaîne opérationnelle locale, qui doit ici relier un core natif à un diagnostic historisé et consultable. C'est majoritairement ce troisième niveau qui reste insuffisamment couvert.

Ce tableau 1.1 synthétise cette analyse critique et situe la solution proposée par rapport aux outils existants.

Critère	Sentry	Breakpad	Crashpad	Solution proposée
Architecture	Cloud / SaaS	Locale	Locale avec <i>Handler</i>	Locale, post-mortem
Langages	Plusieurs langages via SDK	C/C++	C/C++	C/C++
Modification applicative	SDK requis	Gestionnaire intégré	Initialisation du <i>handler</i>	Non
Dépendance réseau	Oui	Non	Variable selon le déploiement	Non
Coût logiciel	Gratuit limité, puis payant	Open source	Open source	Solution ACTIA propriétaire
Adéquation à l'environnement ARM embarqué	Limitée par le réseau et les ressources	Partielle, symbolisation complexe	Partielle, déploiement lourd	Faible dépendance vis-à-vis des ressources externes et déploiement entièrement compatible avec l'architecture ARM64
Tableau de bord	Oui	Non	Non	Oui, avec Streamlit
Historisation	Centralisée côté serveur	À organiser	À organiser	SQLite locale
Confidentialité	Données confiées à une plateforme tierce	Locale	Locale selon le déploiement	Entièrement locale

TABLE 1.1 – Comparaison des principales solutions de gestion de crash et positionnement de la solution proposée

Cette comparaison met en évidence des avantages réels, mais également des écarts avec les contraintes du projet. Ces écarts sont analysés plus précisément dans la sous-section ci-après.

1.3.2 Limites des solutions existantes

L'analyse précédente fait ressortir six écarts majeurs entre les solutions généralistes et le besoin embarqué du projet. Ces écarts ne signifient pas que Sentry, Breakpad ou Crashpad sont inefficaces dans leur domaine d'origine ; ils mettent en évidence que leur architecture et leur mode d'intégration ne satisfont pas simultanément les contraintes d'une campagne de validation locale sur une cible Linux ARM64 :

Infrastructure distante : Sentry et certains déploiements centralisés de Crashpad reposent sur un serveur ou une plateforme distante. Cette architecture est peu adaptée aux campagnes menées sur des bancs isolés ou sans connectivité permanente. Breakpad peut fonctionner hors ligne, mais requiert une infrastructure de collecte et de symbolisation à organiser localement.

Confidentialité : Un core peut exposer mémoire, chemins, fonctions et paramètres internes. Les cores, symboles et rapports doivent donc rester dans l'environnement ACTIA.

Intégration intrusive : Sentry, Breakpad et Crashpad ajoutent respectivement un SDK, un gestionnaire ou un *handler*. Même lorsque cette intégration est techniquement légère, elle

modifie le binaire, son initialisation ou son environnement d'exécution. Pour éviter qu'un mécanisme de supervision influence l'application soumise aux essais, la solution doit exploiter le core natif sans modifier celle-ci.

Gestion des symboles : Breakpad et Crashpad exigent de convertir les symboles natifs dans leur propre format, puis de maintenir la correspondance entre le *minidump*, la version du binaire et le fichier de symboles. L'approche retenue n'élimine pas cette exigence : chaque fichier core doit être analysé avec l'exécutable exact ayant produit le crash et avec les symboles de débogage issus de la même compilation. Elle évite toutefois l'étape de conversion intermédiaire, puisque GDB exploite directement le couple formé par l'exécutable ELF et le fichier core. La gestion des versions reste donc nécessaire, mais la chaîne de symbolisation est simplifiée.

Déploiement embarqué : SDK, agent ou *handler* consomment des ressources et ajoutent des services. Une analyse post-mortem lancée à la demande convient mieux à la cible.

Capitalisation : Breakpad et Crashpad se concentrent sur la capture, Sentry sur la centralisation et GDB sur l'extraction technique. Or, l'équipe doit comparer les incidents, filtrer les résultats et conserver un diagnostic homogène. Le besoin porte donc sur un historique local CSV, JSON et SQLite relié à un tableau de bord, et non sur un fichier de sortie supplémentaire à interpréter manuellement.

Dans cette perspective, aucune solution n'est écartée pour un manque général de performance : chacune répond efficacement à un autre scénario. Sentry convient à l'observabilité centralisée, tandis que Breakpad et Crashpad privilégient la capture robuste en exploitation. Leur limite est leur inadéquation simultanée aux critères prioritaires du projet : absence d'instrumentation, fonctionnement hors ligne, exploitation directe des cores Linux ARM64 et restitution locale intégrée. GDB est donc conservé comme moteur d'analyse éprouvé, puis automatisé et complété par la classification, l'historisation et la visualisation.

Ce choix introduit aussi des compromis. Une analyse strictement post-mortem dispose de moins de contexte qu'un agent actif au moment du crash, et sa précision dépend de l'exécutable exact ainsi que des symboles de débogage. En contrepartie, elle n'intègre aucune charge pendant l'exécution normale et préserve la configuration testée. La solution retenue n'ambitionne donc pas de remplacer toutes les plateformes de *crash reporting* ; elle vise le meilleur alignement avec le processus de validation local d'ACTIA.

1.3.3 Problématique

GDB apporte les données techniques, mais son usage manuel exige une expertise du débogueur. Les plateformes cloud ajoutent la connectivité, la confidentialité et l'instrumentation ; les *minidumps* ajoutent un gestionnaire et une chaîne de symbolisation.

Cette difficulté ne consiste donc pas uniquement à récupérer les données d'un crash, mais à les transformer automatiquement en un diagnostic compréhensible, reproductible et traçable,

sans alourdir la cible ni modifier l'application surveillée. La question de recherche de ce projet est ainsi formulée :

Comment concevoir et valider une chaîne d'analyse post-mortem locale, portable et non intrusive, capable de transformer automatiquement les fichiers core d'applications Linux embarquées en diagnostics structurés et accessibles, tout en préservant la confidentialité des données et sans dépendre d'une infrastructure externe ni modifier l'application surveillée ?

Dans cette perspective, cette problématique s'articule autour de quatre dimensions complémentaires : **l'automatisation de l'extraction et de la classification, la robustesse du traitement sur une cible ARM64 contrainte, l'autonomie vis-à-vis du réseau et des services externes**, ainsi que **l'accessibilité du résultat pour les ingénieurs ne maîtrisant pas GDB en détail**. Le positionnement présenté ci-après précise la manière dont le projet répond à ces dimensions.

1.3.4 Positionnement du projet

Ce projet *Crash Analyzer* se positionne sur l'analyse post-mortem locale des crashes sur systèmes embarqués Linux. Il se situe entre les plateformes cloud généralistes, écartées en raison de leurs contraintes d'infrastructure et de confidentialité, et l'analyse manuelle avec GDB, complexe et dépendante de l'expertise individuelle.

La solution privilégie quatre propriétés :

Ce positionnement repose d'abord sur l'autonomie, puisque le fonctionnement normal ne requiert ni serveur externe ni dépendance réseau. Il simplifie ensuite le déploiement, car aucun SDK ne doit être intégré dans l'application surveillée. L'analyse peut par ailleurs commencer dès que le fichier core, l'exécutable, GDB et les symboles sont disponibles. Enfin, les données restent sur la cible et sur le poste de l'ingénieur, ce qui préserve leur confidentialité.

Ce positionnement répond au besoin d'un outil rapide et confidentiel pour les campagnes de validation internes, sans infrastructure distante ni coût récurrent. Sur cette base, la sous-section suivante caractérise l'architecture fonctionnelle retenue pour concrétiser ce positionnement.

1.3.5 Solution proposée

La solution *Crash Analyzer* a pour objectif d'automatiser l'extraction, l'analyse, la centralisation et la visualisation des informations relatives aux crashes applicatifs. Elle exploite les fichiers core générés au moment de l'incident, sans modifier le code des applications surveillées.

La figure 1.4 illustre l'architecture générale de la solution proposée ainsi que les principales interactions entre ses différents composants.

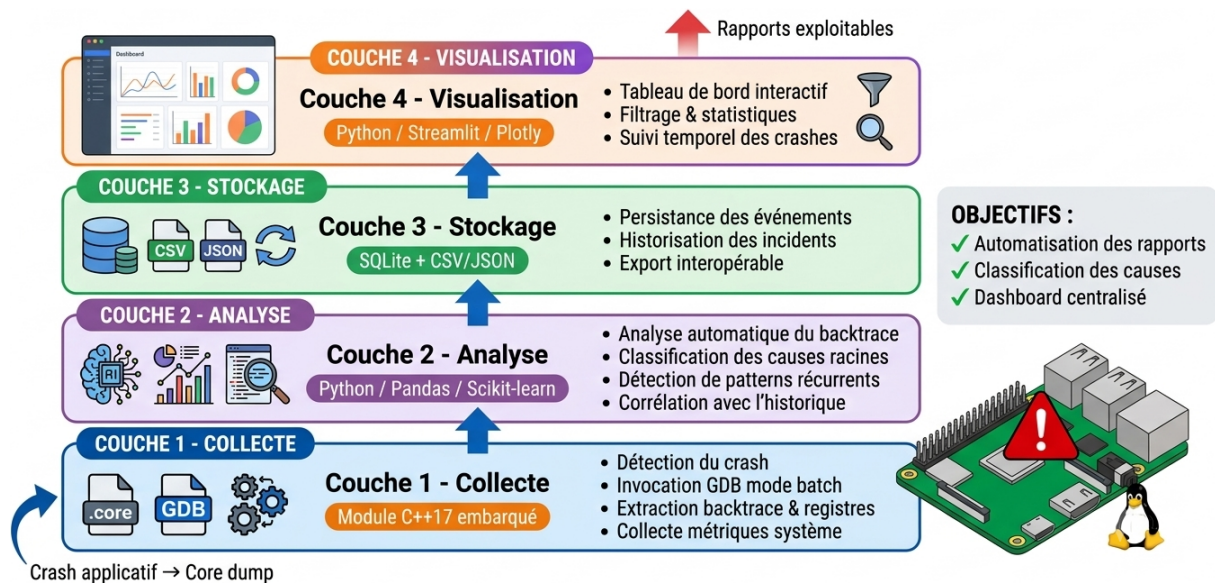


FIGURE 1.4 – Architecture en couches de la solution *Crash Analyzer*

Le principe retenu est une analyse locale réalisée après le crash : lorsqu'un incident survient, l'outil traite le fichier core, extrait les informations de diagnostic disponibles, puis conserve le résultat dans un format structuré. Les incidents peuvent ensuite être consultés depuis un tableau de bord dans le but de simplifier leur suivi et leur comparaison. Le tableau 1.2 résume une telle organisation en quatre couches.

TABLE 1.2 – Architecture en couches de la solution *Crash Analyzer*

Couche	Technologie	Rôle
Couche 1. Collecte	C++17 embarqué	Détection des fichiers core et extraction du contexte avec GDB
Couche 2. Analyse	C++17 / PatternMatcher	Classification, explication et correctif
Couche 3. Stockage	SQLite + CSV/JSON	Historisation et export des incidents
Couche 4. Visualisation	Python / Streamlit / Plotly	Dashboard_Creator, filtres et statistiques

Cette architecture répond à quatre besoins complémentaires. La couche de collecte détecte les fichiers core et extrait leur contexte avec GDB. La couche d'analyse classe la cause du crash et produit une explication ainsi qu'un correctif. La couche de stockage conserve un historique structuré. Enfin, la couche de visualisation rend le diagnostic accessible aux profils ne maîtrisant pas l'utilisation avancée de GDB.

L'originalité de la solution proposée réside dans son caractère générique : elle n'est pas liée à une application particulière et peut analyser le crash de tout exécutable ELF disposant des symboles de débogage nécessaires. Elle intervient après le crash, sans instrumentation du code applicatif et sans dépendance à une plateforme cloud. Cette architecture apporte le cadre à partir duquel les objectifs mesurables du projet sont définis.

1.3.6 Objectifs du projet

L'objectif général revient à développer un outil capable d'automatiser la collecte, l'analyse et la restitution des informations relatives aux crashes sur des systèmes embarqués Linux. Il a pour objectif de standardiser le diagnostic, à limiter la dépendance à l'expertise individuelle et à améliorer la traçabilité des incidents.

Cet objectif est décliné en objectifs spécifiques mesurables :

Le premier objectif spécifique, OS1, revient à produire automatiquement, pour chaque fichier core, un rapport structuré aux formats CSV, JSON et SQLite. Sur la base des informations extraites, OS2 a pour objectif de classer la cause probable des crashes associés aux principaux signaux POSIX et à lui attribuer un niveau de confiance. OS3 complète ce diagnostic par une explication en langage clair et une piste de correction. Les résultats doivent ensuite être rendus accessibles conformément à OS4, au moyen d'un tableau de bord Web permettant leur consultation, leur filtrage et leur exploration détaillée. Enfin, OS5 porte sur la validation de la chaîne complète sur x86_64 et ARM64 grâce à la compilation croisée.

L'atteinte de ces objectifs dépend directement des choix d'architecture et des technologies employées. Leur adéquation avec les contraintes de robustesse, de portabilité et de fonctionnement local est justifiée ci-après.

1.3.7 Justification des choix architecturaux

Ces technologies retenues répondent à la séparation des rôles entre la collecte sur la cible, l'analyse post-mortem, la persistance locale et la visualisation sur le poste hôte. Le tableau 1.3 synthétise chaque décision, l'alternative écartée et sa justification.

TABLE 1.3 – Comparaison et justification des choix architecturaux et technologiques

Critère	Choix retenu	Alternative écartée	Justification
Mode de diagnostic	Analyse post-mortem à partir du fichier core	Surveillance continue de l'application	L'analyse intervient après le crash, ne requiert aucune surveillance permanente et n'introduit aucune surcharge pendant l'exécution normale.
Déploiement	Solution entièrement locale	Plateforme cloud	Les fichiers core, les symboles et les rapports restent sur la cible ou sur le poste de l'ingénieur, ce qui protège les données et permet une utilisation sans réseau.
Intégration applicative	Absence d'instrumentation	SDK ou gestionnaire de crash intégré	L'exploitation des fichiers core générés nativement par Linux donne la possibilité d'analyser une application tierce sans modifier son code, dès lors que l'exécutable et ses symboles sont disponibles.
Langage des modules critiques	C++17 pour Crash_Simulator et Crash_Analyzer	Python pour le cœur de traitement	C++17 offre une exécution native, un accès direct aux processus, aux signaux et aux fichiers, ainsi qu'une gestion déterministe des ressources et une portabilité vers x86_64 et ARM64.
Moteur d'analyse	GDB en mode batch	Débogueur spécifique ou instrumentation applicative	GDB prend directement en charge les exécutables ELF, les fichiers core et les symboles. Son mode batch automatise l'extraction reproductible du signal, du backtrace, des registres et des threads.
Persistance et export	SQLite complété par CSV et JSON	Serveur de base de données ou fichiers plats seuls	SQLite apporte une historisation structurée sans service permanent. CSV simplifie l'exploitation tabulaire et JSON conserve une représentation structurée et interopérable des rapports.
Visualisation	Python, Streamlit et Plotly sur le poste hôte	Framework Web complet tel que Flask ou Django	Streamlit donne la possibilité de construire une interface orientée données sans développer séparément les routes et les pages HTML, tandis que Plotly apporte les graphiques interactifs sans alourdir la cible.

Ces choix assurent une cohérence entre les contraintes techniques et les responsabilités de chaque module. Une fois l'architecture et les technologies définies, leur mise en œuvre peut être organisée selon la planification décrite dans la section suivante.

1.4 Méthodologie de travail

La réalisation du projet s'appuie sur une démarche itérative et incrémentale. Cette démarche construit la solution progressivement, plutôt que de développer tous les composants en une seule phase avant validation. Elle est adaptée au projet *Crash Analyzer*, car les trois modules sont interdépendants et doivent être validés successivement sur deux architectures, x86_64 et ARM64.

Le caractère itératif signifie que le travail est organisé en cycles courts et répétés. Chaque itération suit les mêmes activités principales : planification, analyse des exigences, conception, implémentation et tests. À la fin du cycle, les résultats sont examinés dans le but d'identifier les erreurs, les contraintes techniques et les améliorations nécessaires. Ces observations alimentent l'itération suivante. Une fonctionnalité peut ainsi être conçue, réalisée, testée puis corrigée jusqu'à atteindre le niveau de fonctionnement attendu.

Le caractère incrémental concerne, quant à lui, la construction progressive du produit. Chaque cycle ajoute un ensemble cohérent de fonctionnalités à la version précédente. L'incrément obtenu doit être exécutable et vérifiable, même si la solution complète n'est pas encore terminée. Une telle organisation donne la possibilité de disposer rapidement de résultats intermédiaires concrets et d'éviter de reporter l'intégration de tous les modules à la fin du projet.

Une itération commence par la planification du travail à réaliser et la définition d'un objectif vérifiable. Les exigences associées sont ensuite analysées dans le but de préciser les entrées, les sorties, les contraintes et les critères d'acceptation. La conception détermine les classes, les interfaces et les échanges nécessaires. L'implémentation traduit cette conception en code source, puis les tests vérifient le comportement obtenu. Lorsqu'un défaut ou une limite est détecté, le cycle revient vers la planification ou la conception dans le but d'intégrer les corrections nécessaires. Le déploiement intervient lorsque l'incrément atteint un niveau de stabilité suffisant.

La figure 1.5 illustre cette méthodologie appliquée au projet, en détaillant les cycles et les incréments successifs. Les flèches de retour entre les étapes montrent que les tests ne représentent pas uniquement une phase finale : ils alimentent directement les décisions du cycle suivant. Git complète cette démarche en conservant l'historique des modifications, en séparant les travaux par branches et en facilitant l'intégration progressive des versions validées.

Cette représentation apporte le cadre de mise en œuvre retenu pour le projet. L'application de cette méthodologie est structurée en quatre incréments. Le premier correspond au développement du *Crash_Simulator*. Il donne la possibilité de configurer l'environnement de production des fichiers core, de déclencher des défaillances contrôlées et de vérifier que chaque scénario produit le signal attendu. Les fichiers core obtenus représentent ensuite des données d'essai reproductibles pour le module d'analyse.

Le deuxième incrément introduit *Crash_Analyzer*. Son développement commence par la détection des fichiers core et le pilotage de GDB en mode batch. Les itérations suivantes ajoutent progressivement l'extraction du signal, du backtrace, des registres et des threads, puis la classification de la cause probable, la production d'une explication et l'export des diagnostics. Chaque évolution est testée sur les fichiers produits par l'incrément précédent dans le but de vérifier la cohérence de la chaîne.

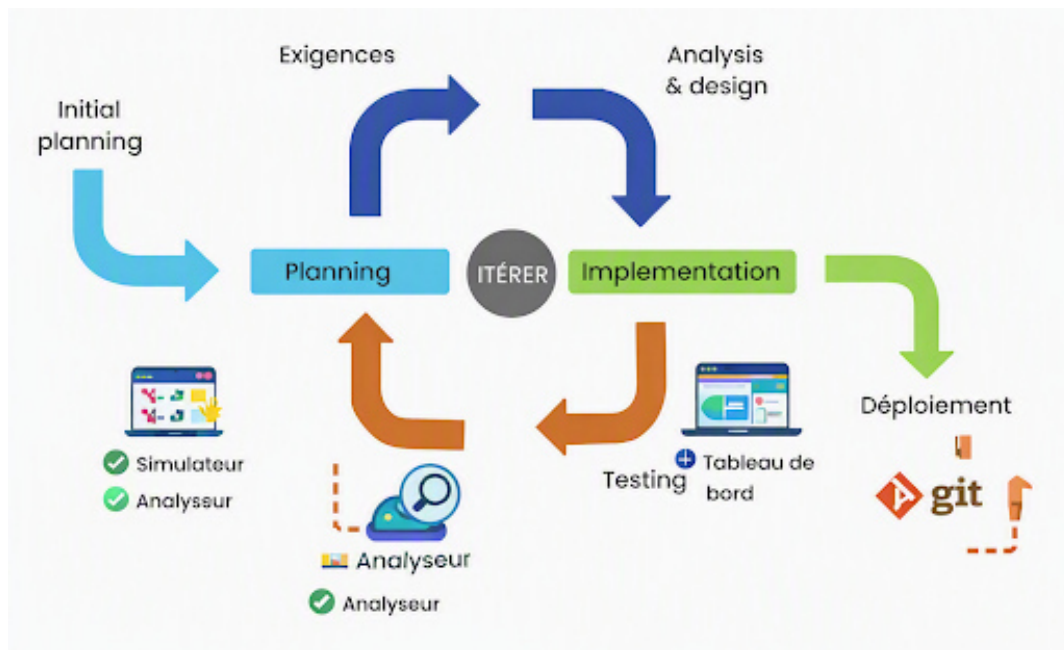


FIGURE 1.5 – Cycle de développement itératif et incrémental appliqué au projet *Crash_Analyzer*.

Le troisième incrément concerne Dashboard_Creator. Les premiers cycles portent sur le chargement des rapports CSV, JSON et SQLite. Les cycles suivants ajoutent les indicateurs, les graphiques, les filtres et la consultation détaillée d'un incident. Les résultats affichés sont comparés aux rapports générés par Crash_Analyzer, ce qui donne la possibilité de détecter rapidement les incohérences entre l'analyse, le stockage et la visualisation.

Le quatrième incrément porte sur l'intégration et le déploiement. Les composants C++ sont compilés pour ARM64, transférés sur le Raspberry Pi 4, puis exécutés dans l'environnement cible. Les erreurs de dépendances, de chemins ou de configuration des fichiers core donnent lieu à de nouvelles itérations jusqu'à l'obtention d'une chaîne complète et reproductible. La validation finale vérifie alors l'enchaînement allant de la production du crash jusqu'à l'affichage du diagnostic dans le tableau de bord.

Une telle organisation assure la traçabilité des évolutions, la validation progressive des modules et la reproductibilité des résultats sur les architectures ARM64 et x86_64. Elle prépare ainsi la planification détaillée des phases et des jalons du projet.

1.5 Planification du travail

Le stage a été organisé en huit phases, depuis la formation initiale jusqu'à la validation et à la rédaction. La figure 1.6 présente leur enchaînement et leurs périodes de chevauchement.

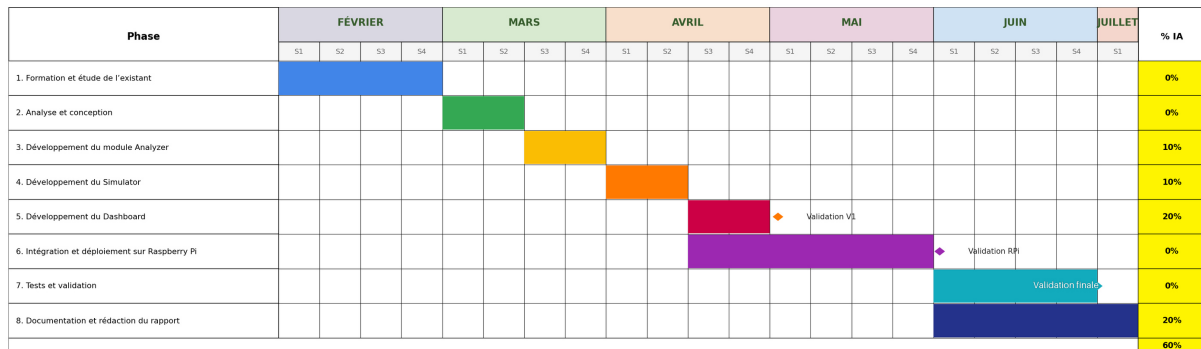


FIGURE 1.6 – Diagramme de Gantt du projet

La séquence majeure du projet suit l'enchaînement suivant : analyse et conception, développement des modules, intégration ARM64 et validation finale. La formation et l'étude de l'existant occupent le mois de février, puis l'analyse et la conception couvrent le début du mois de mars. Le développement du Crash_Analyzer se déroule à la fin du mois de mars, puis celui du Crash_Simulator en avril. Ces phases sont dépendantes : les choix de conception conditionnent le développement, et les résultats alimentent le tableau de bord, l'intégration et les tests. Un retard affecterait donc les phases en aval.

Trois jalons critiques structurent le projet : validation du tableau de bord (mai), validation sur Raspberry Pi 4 (juin) et validation finale après la campagne de tests (fin juin). Ils représentent des points de contrôle entre les sprints de développement, l'intégration ARM64 et la validation globale.

La planification prévoit des chevauchements : le développement du tableau de bord et l'intégration sur Raspberry Pi 4 commencent simultanément à la mi-avril et se poursuivent en mai, dans le but de traiter les difficultés de compilation croisée et de configuration avant le jalon de juin. La documentation débute en parallèle des tests au début du mois de juin et se prolonge jusqu'au début du mois de juillet, ce qui donne la possibilité d'intégrer progressivement les résultats de validation dans le rapport. La figure 1.7 rappelle le code couleur utilisé.

Conclusion

Le présent chapitre a exposé le cadre général dans lequel s'inscrit le projet de fin d'études. Il a d'abord introduit le groupe ACTIA et l'organisme d'accueil, ACTIA Aeronautic Services Tunisie (AAS), ainsi que les activités directement liées au contexte du projet.

Il a ensuite exposé le contexte industriel du diagnostic des crashes applicatifs dans les systèmes Linux embarqués et analysé les principales solutions existantes. Cette étude a permis d'identifier leurs limites au regard des contraintes du projet, notamment en matière de fonctionnement local, de confidentialité, d'absence d'instrumentation et d'exploitation des fichiers core sur une architecture ARM64.

Sur la base de cette analyse, la problématique et le positionnement du projet ont été définis, puis la solution *Crash Analyzer* et ses principaux objectifs ont été présentés. Enfin, les choix architecturaux et technologiques retenus ont été justifiés avant de décrire la planification adoptée

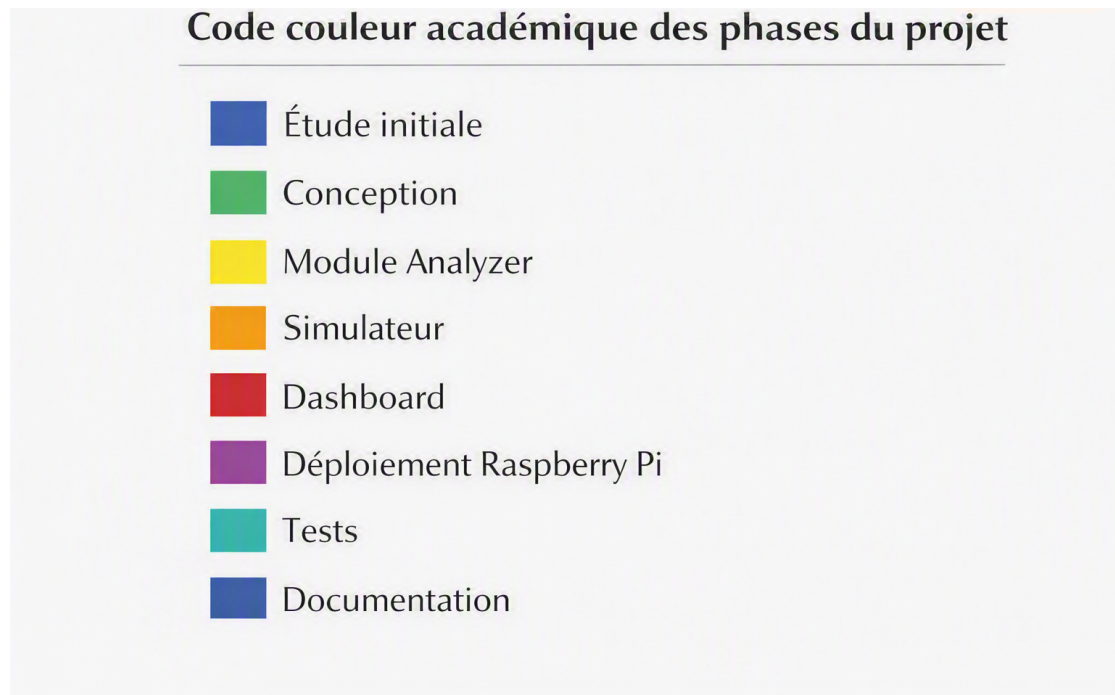


FIGURE 1.7 – Code couleur des phases du projet

pour la réalisation du projet. Le chapitre suivant approfondit ces orientations en détaillant les besoins et la conception de la solution.

Étude des besoins et conception de la solution

Sommaire

2.1	Introduction	17
2.2	Notions fondamentales sur les systèmes Linux embarqués . .	17
2.3	Analyse des besoins	22
2.3.1	Besoins fonctionnels	22
2.3.2	Besoins non-fonctionnels	23
2.4	Conception de la solution	24
2.5	Architecture logicielle des modules	25
2.6	Diagramme d'activité global	29
2.7	Conception des modules	31
2.7.1	Conception du module <code>Crash_Analyzer</code>	31
2.7.2	Conception du module <code>Crash_Simulator</code>	34
2.8	Interactions entre modules	36

2.1 Introduction

Ce chapitre expose l'étude des besoins et la conception de la solution proposée. Il introduit d'abord les notions techniques mobilisées dans le projet, notamment les systèmes Linux embarqués, l'architecture ARM, la compilation croisée et les mécanismes de crash sous Linux. Ces éléments permettent de comprendre les contraintes imposées par la cible ARM64, le rôle du noyau dans la production des fichiers core et les informations exploitées par GDB. Le chapitre formalise ensuite les besoins fonctionnels et non fonctionnels pour relier chaque exigence à un module et à une méthode de validation. Il détaille enfin l'architecture globale, la répartition des responsabilités entre les trois composants et leurs interfaces par fichiers persistants. Les diagrammes d'activité, de classes, de séquence et de déploiement complètent cette description en représentant les traitements, les échanges et l'implantation physique de la solution.

2.2 Notions fondamentales sur les systèmes Linux embarqués

Cette section rappelle brièvement les contraintes de la cible Linux embarquée et l'écart d'architecture entre le poste hôte et le Raspberry Pi 4. Elle se concentre ensuite sur les notions directement mobilisées par le projet : compilation croisée, production des fichiers core, format ELF et automatisation de GDB.

2.2.1 Systèmes Linux embarqués : définition et caractéristiques

Un système Linux embarqué relie un noyau Linux et un espace utilisateur adaptés à un équipement dédié, généralement soumis à des contraintes de ressources, de fiabilité et de connectivité [13]. Dans ce projet, les caractéristiques déterminantes sont l'exécution sans interface graphique locale, le fonctionnement possible sur un réseau isolé et la nécessité de limiter les composants installés sur la cible.

Ces contraintes conduisent à lancer uniquement les traitements essentiels en C++ sur le Raspberry Pi 4, tandis que la visualisation est déportée sur le poste hôte. Elles justifient également une analyse locale fondée sur les fichiers core, sans service distant ni instrumentation permanente de l'application.

2.2.2 Écart d'architecture entre l'hôte et la cible

Le développement est réalisé sur un poste Ubuntu x86_64, alors que l'exécution embarquée cible le Raspberry Pi 4 en ARM64 (aarch64). Ces architectures reposent sur des jeux d'instructions et des conventions binaires divers : un exécutable compilé nativement pour le PC hôte x86_64 ne peut donc pas être exécuté directement sur la cible ARM64.

La conséquence utile pour le projet n'est pas une comparaison générale entre ARM et x86, mais la nécessité de produire des binaires et de résoudre des dépendances adaptés à aarch64. Cette contrainte établit la chaîne de compilation croisée, la séparation des répertoires de construction et les contrôles d'architecture effectués avant le déploiement.

2.2.3 Architecture d'un système Linux embarqué

Après avoir distingué les architectures matérielles de l'hôte et de la cible, la figure 2.1 expose les principales couches d'un système Linux embarqué.

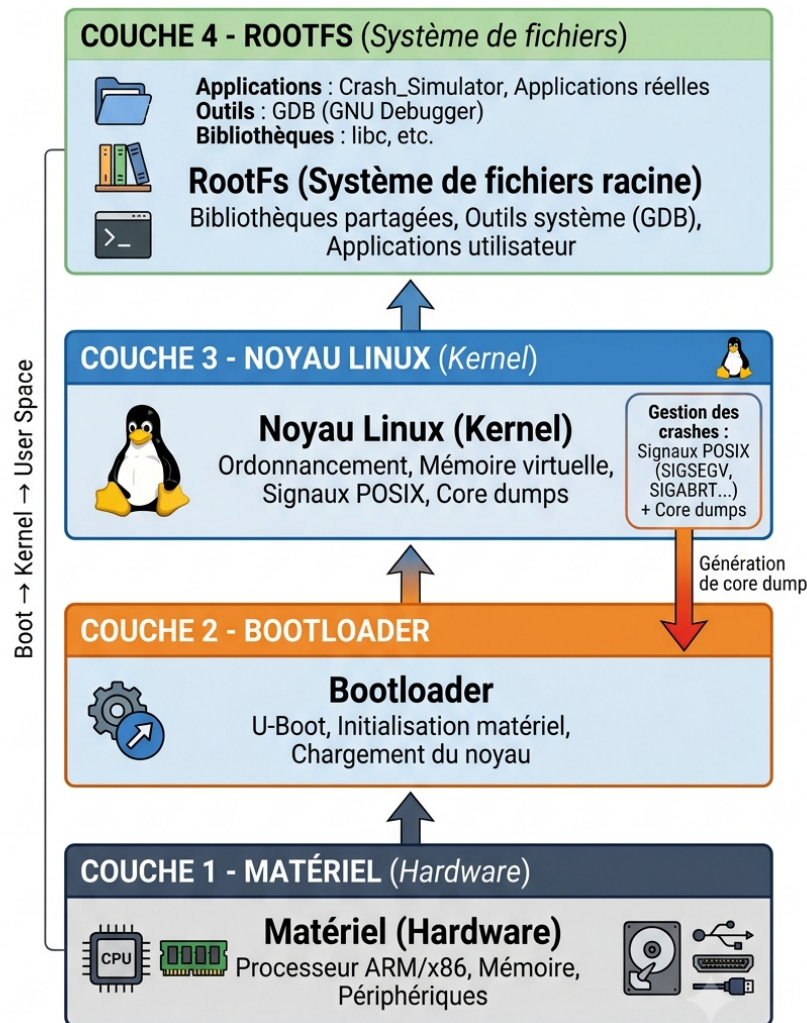


FIGURE 2.1 – Architecture en couches d'un système Linux embarqué

La couche matérielle (*Hardware Layer*) regroupe les ressources physiques de la cible, notamment le processeur, la mémoire et les périphériques. Le chargeur d'amorçage (*Bootloader*) assure l'initialisation du système et le chargement du noyau Linux (*Linux Kernel*), qui assure la gestion des ressources matérielles, de la mémoire et des processus. Le système de fichiers racine (*RootFS*) apporte les bibliothèques, outils et applications nécessaires à l'espace utilisateur (*User Space*). Cette organisation permet de situer l'exécution des applications et des binaires ARM64 (aarch64) sur la cible, ainsi que la production et la prise en charge des fichiers de crash analysés dans le cadre de ce projet.

2.2.4 Compilation croisée : principe et choix du projet

Comme l'illustre la figure 2.2, la compilation croisée revient à construire le code C++ sur le PC hôte x86_64 pour produire un exécutable ARM64 destiné au Raspberry Pi 4. La chaîne `aarch64-linux-gnu-g++` produit un binaire compatible avec l'architecture aarch64, sans imposer la compilation directement sur la cible [8]. Cette organisation réduit la durée des cycles de construction et évite d'installer un environnement de développement complet sur un équipement dont les ressources CPU, mémoire et stockage sont limitées. Le Raspberry Pi reste ainsi consacré à l'exécution et à la validation du logiciel dans son environnement embarqué réel.

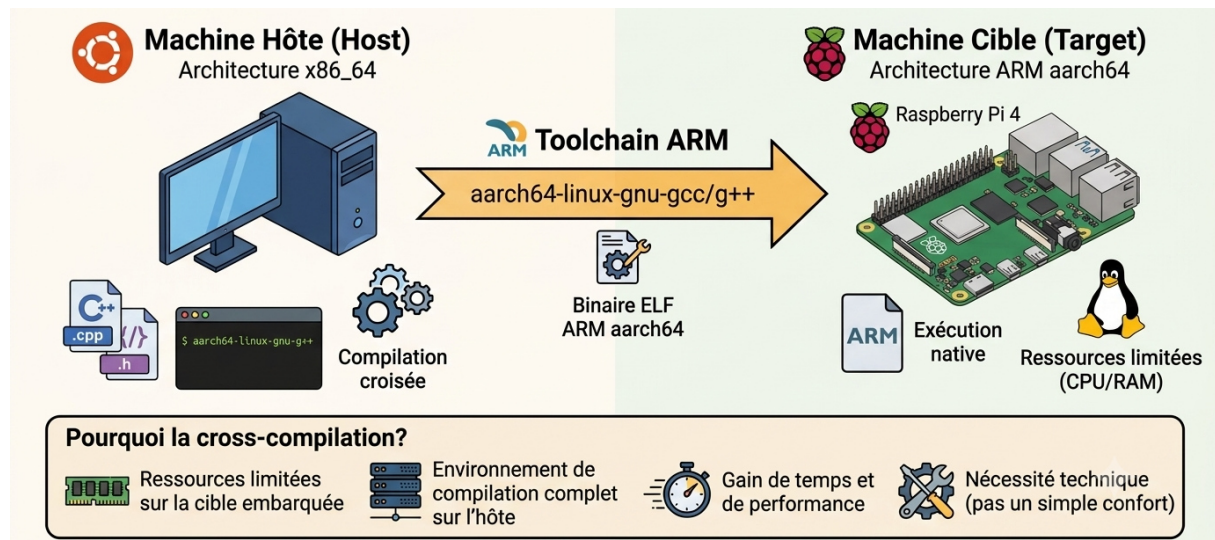


FIGURE 2.2 – Principe de la compilation croisée vers une cible ARM64 (aarch64)

Ce choix répond au besoin de disposer d'une solution portable et vérifiable sur ARM64 sans mobiliser les ressources de la cible pour sa construction.

Ainsi, la compilation croisée représente une étape essentielle pour optimiser l'implémentation et la validation du système embarqué. Cette démarche s'intègre naturellement dans le flux de travail global du projet, garantissant une meilleure portabilité et une efficacité accrue. En résumé, cette démarche permet de concilier les performances du processus de développement avec les contraintes matérielles propres aux systèmes embarqués.

2.2.5 Mécanismes de crash sous Linux embarqué

La figure 2.3 synthétise les échanges entre l'espace utilisateur, le matériel et le noyau lors d'un crash applicatif. L'application exécute d'abord une opération invalide, telle qu'un accès mémoire interdit. Le processeur détecte cette anomalie et déclenche une exception matérielle transmise au noyau Linux. Celui-ci la traduit alors en un signal POSIX envoyé au processus fautif et, lorsque la configuration du système l'autorise, produit le fichier core destiné à l'analyse post-mortem.

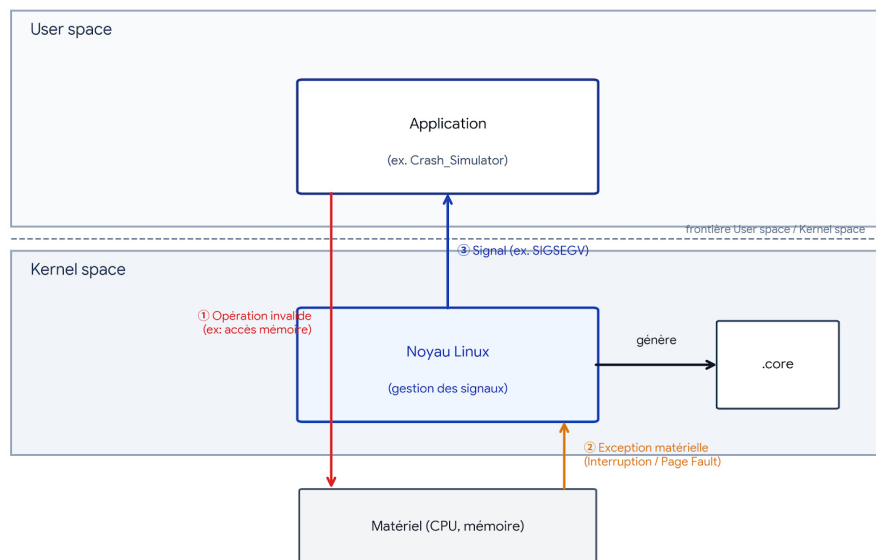


FIGURE 2.3 – Communication entre l’espace utilisateur, le matériel et le noyau Linux lors d’un crash applicatif

a) Signaux POSIX et production des fichiers core

Un signal POSIX est une notification asynchrone envoyée à un processus pour lui signaler un événement. Certains signaux, considérés comme les plus critiques, entraînent par défaut la terminaison du processus. Lorsque la configuration du système l’autorise, le noyau génère alors un fichier *core dump* contenant l’état du programme au moment de son arrêt [5].

Il convient de distinguer les signaux Linux des types de crashes applicatifs : les 31 signaux standards ne correspondent pas tous à une défaillance et ne produisent pas tous un fichier core. Crash_Analyzer extrait et traite le signal enregistré dans chaque fichier core valide, tandis que la classification détaillée et la validation réalisées dans ce projet portent majoritairement sur les signaux les plus critiques : SIGILL, SIGTRAP, SIGABRT, SIGBUS, SIGFPE et SIGSEGV. La liste complète des signaux standards, avec leur code, leur type et leur signification, est fournie à l’annexe A.5.

La production du fichier core dépend principalement de la limite définie par `ulimit -c` et du paramètre noyau `kernel.core_pattern`, qui détermine son emplacement et son nom [6]. Sur les distributions utilisant `systemd`, le service `systemd-coredump` peut intercepter, journaliser et stocker ces vidages mémoire avec leurs métadonnées [21]. La configuration retenue dans ce projet privilégie toutefois une écriture directe dans un répertoire maîtrisé. La configuration de ces paramètres relève du module `Crash_Simulator`, tandis que l’exploitation du fichier obtenu représente le point de départ de `Crash_Analyzer`.

b) Format ELF d’un fichier core

Le format ELF (*Executable and Linkable Format*), utilisé sous Linux pour les exécutables et les bibliothèques, sert également aux fichiers core. Un fichier core enregistre l’état des registres, les piles d’appels et les zones mémoire mappées du processus au moment du crash. Comme l’illustre la figure 2.4, ces données doivent être associées à l’exécutable d’origine et à ses symboles de débogage pour relier les adresses mémoire aux fonctions et aux lignes du code source.

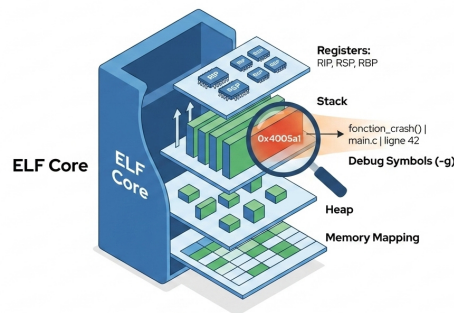


FIGURE 2.4 – Structure et contenu d'un fichier core au format ELF

L'option de compilation `-g` conserve les symboles nécessaires à cette résolution. Sans elle, le fichier core reste exploitable, mais le backtrace renferme principalement des adresses brutes, ce qui réduit fortement la précision du diagnostic.

c) GDB en mode batch et extraction du contexte

GDB analyse ensemble l'exécutable et le fichier core correspondant. Avec `-batch` et `-ex`, il lance des commandes sans interaction et renvoie un texte exploitable [7]. Il a été préféré à LLDB [22] pour sa disponibilité sous Linux ARM64. La figure 2.5 illustre la localisation de la faute et le backtrace.

CoreFileParser lance GDB avec `runGdbBatch()`, puis extrait le signal, le backtrace et le cadre fautif. Ces données alimentent `CrashInfo`, qui est ensuite classifié et enrichi d'une correction.

```
#0 CrashSimulator::Simulator::triggerAccessViolation (this=0x7ffcd9efac0) at Simulator.cpp:193
193     mutable ptr[X] = 'X';
    rodata = 0x80af4962d09 "Read-only string"
    mutable ptr = 0x8baf4962d21 "Read-only string"

Backtrace
#1 CrashSimulator::Simulator::runOnce (this=0x7ffcd9efac0) at /home/dghriss@actiauluator/Bureau/pfe/.../local/Bureau/pfe/.S8maf4962d02/Simulator.cpp:122
No Locals.
#2 CrashSbsf9652809 in CrashSimulator::Crash (this=dghriss@actia.local_Legpanlata/Simulator.cpp:22
#3 0x008bsf9620941 rmg (argv=0x7fcd9efac0) at /home/dghriss@actia.local_project/simulator.cpp:222
```

FIGURE 2.5 – Localisation de la faute et backtrace extraits d'un fichier core par GDB

Le backtrace ordonne les appels actifs de la fonction fautive à `main()`. `bt full`, `frame 0`, `info registers`, `info threads` et `$_siginfo` apportent pile, contexte, registres, threads et signal.

La figure 2.6 expose les registres et le thread actif extraits lors d'un crash de type SIGSEGV. Le registre d'instruction RIP localise l'opération fautive, tandis que RSP et RBP décrivent l'état de la pile d'exécution.


```

seed = 3803755972
cat = CrashSimulator::CrashCategory::SEGMENTATION_FAULT
sev = 4
i = 0
cfg = {crash count = 10, intensity = 7, random_mode = true, logging = true, output_dir = "simulator_output",
csvPath = "simulator_output/crash_report.csv"
success = 0
rax 0x5baf4962cd21 100808408616225
rbx 0x5baf49634080 100808408645760
rcx 0x6 6
rdx 0x3 0
rsi 0x3 3
rdi 0x7ffcd9efa7e0 140723964048096
rbp 0x7ffcd9efa800 0x7ffcd9efa800
rsp 0x7ffcd9efa800 0x7ffcd9efa800
r8 0x5baf73c56550 100009119720784
r9 0x7ffcd9efa350 140723964846928
r10 0x1 1
r11 0xf80583fcb90f3c78 -538079450941125512
r12 0x1 1
r13 0x5baf4961fe6f 100808408563311
r14 0x5baf496339a0 100808408644000
r15 0x72431777e040 125662422107456
rip 0x5baf496238d9 0x5baf496238d9 <CrashSimulator::Simulator::triggerAccessViolation()+249>
eflags 0x10246 [ PF ZF IF RF ]
cs 0x33 51
ss 0x2b 43
ds 0x0 0
es 0x0 0
fs 0x0 0
info threads 0x0 0
* 1 Thread 0x7243177113c0 (LWP 474179) 0x00005baf496238d9 in CrashSimulator::Simulator::triggerAccessViolation(
at /src/Simulator.cpp:193

```

FIGURE 2.6 – Registres, instruction fautive et thread actif extraits par GDB lors d'un SIGSEGV

Ces informations complètent le backtrace en fournissant l'état précis du processeur au moment de la défaillance. L'analyse croisée du signal, du registre d'instruction, des pointeurs de pile et du thread actif permet de confirmer la nature du crash et de distinguer une adresse fautive d'une corruption plus générale du contexte d'exécution. L'interprétation détaillée des principaux registres observés dans cet exemple est reportée à l'annexe A.6 dans le but d'alléger le corps du chapitre.

Dans la solution proposée, CoreFileParser extrait automatiquement ces sorties de GDB, les normalise et les relie au fichier core analysé. Ces données structurées renforcent la classification de PatternMatcher et permettent à SolutionProposer de produire une explication cohérente avec le contexte réel du crash.

2.3 Analyse des besoins

Cette section rassemble les besoins fonctionnels, qui décrivent les services attendus, et les besoins non fonctionnels, qui précisent les contraintes de qualité et de déploiement.

2.3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les différentes fonctionnalités que le système doit assurer pour répondre aux objectifs du projet.

Ces besoins fonctionnels sont regroupés par module dans le tableau 2.1. Cette présentation relie chaque fonctionnalité attendue au résultat qu'elle doit produire.

TABLE 2.1 – Besoins fonctionnels des trois modules de la solution

Module Crash_Simulator		
Code	Besoin fonctionnel	Résultat attendu
BFS-01	Charger la configuration de la campagne	Lire dans <code>scenario.cfg</code> la catégorie, le nombre d'itérations, l'intensité et le mode d'exécution.
BFS-02	Préparer la production des fichiers core	Configurer et vérifier automatiquement <code>ulimit</code> et <code>core_pattern</code> .
BFS-03	Générer des crashes contrôlés	Déclencher les dix-huit catégories prises en charge avec un signal attendu connu.
BFS-04	Isoler chaque scénario	Exécuter chaque crash dans un processus fils afin que la campagne se poursuive après une terminaison fatale.
BFS-05	Reproduire une campagne aléatoire	Enregistrer la graine et les métadonnées nécessaires pour rejouer la même séquence.

Module Crash_Analyzer

Code	Besoin fonctionnel	Résultat attendu
BFA-01	Détecter les fichiers core valides	Identifier les fichiers ELF exploitables et leur exécutable associé.
BFA-02	Extraire le contexte avec GDB	Récupérer automatiquement le signal, le backtrace, le cadre fautif, les registres et les threads.
BFA-03	Classer la cause probable	Corréler le signal et les motifs de pile, puis attribuer une catégorie et un niveau de confiance.
BFA-04	Produire une aide au diagnostic	Fournir une explication en langage clair et une proposition de correction.
BFA-05	Traiter les fichiers core par lot	Poursuivre l'analyse lorsqu'un fichier est invalide ou incomplet.
BFA-06	Exporter et historiser les résultats	Générer les diagnostics aux formats CSV, JSON et SQLite.

Module Dashboard_Creator

Code	Besoin fonctionnel	Résultat attendu
BFD-01	Charger les rapports	Lire une source CSV, JSON ou SQLite sans appeler directement les composants C++.
BFD-02	Présenter une synthèse globale	Afficher les indicateurs, les statistiques et les graphiques de répartition des incidents.
BFD-03	Explorer et filtrer les incidents	Proposer une liste filtrable par signal, catégorie, date ou autre champ disponible.
BFD-04	Consulter le détail d'un crash	Afficher la cause probable, le correctif, le code source et le backtrace correspondant.
BFD-05	Assister l'investigation	Restituer le transcript GDB et les explications produites par l'analyseur.

2.3.2 Besoins non-fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles la solution doit répondre. Ces exigences tiennent compte des ressources limitées de la cible embarquée, de la coexistence des architectures x86_64 et ARM64 ainsi que de la nécessité de conserver un fonctionnement local et robuste. Le tableau 2.2 relie chaque besoin à un critère observable et à la méthode utilisée pour le valider.

TABLE 2.2 – Besoins non fonctionnels et méthodes de validation

Catégorie	Exigence	Méthode de validation
Efficacité	Analyser automatiquement un lot de fichiers core sans interrompre le traitement.	Traiter 51 fichiers et vérifier que chaque fichier core valide produit un diagnostic.
Orientation système	Exécuter la solution sur une cible Linux embarquée ARM64.	Compiler pour aarch64, puis produire et analyser un fichier core sur le Raspberry Pi 4.
Confidentialité	Conserver localement les fichiers core et les rapports, sans service cloud.	Vérifier l'absence d'API externe et limiter les transferts à SSH/SCP.
Maintenabilité	Séparer les responsabilités pour faire évoluer les modules indépendamment.	Tester chaque module séparément et vérifier l'absence de dépendances inutiles entre eux.

Ces exigences sont validées par trois contrôles complémentaires : une campagne de test sur un grand nombre de fichiers core pour évaluer l'efficacité du traitement, une exécution réelle sur le Raspberry Pi 4 pour confirmer l'orientation système et une vérification des échanges réseau pour garantir la confidentialité. Le temps d'exécution est mesuré à titre d'évaluation, sans représenter un seuil contractuel. Les résultats obtenus sont présentés au chapitre 3, tableau 3.15.

2.4 Conception de la solution

Cette partie détaille la conception globale de la solution *Crash Analyzer*. Elle décrit l'architecture générale du système ainsi que les différents diagrammes UML utilisés pour modéliser les interactions entre les modules *Crash_Simulator*, *Crash_Analyzer* et *Dashboard_Creator*, ainsi que les fonctionnalités implémentées par chaque module.

2.4.1 Architecture globale du système

La figure 2.7 synthétise la chaîne complète d'analyse post-mortem, depuis l'opération invalide jusqu'à la consultation du diagnostic. Elle distingue trois espaces d'exécution – matériel, noyau et espace utilisateur – et quatre phases fonctionnelles : production du crash, analyse, export et visualisation. Cette représentation permet surtout d'identifier les frontières de responsabilité entre le système Linux et les modules développés dans le cadre du projet.

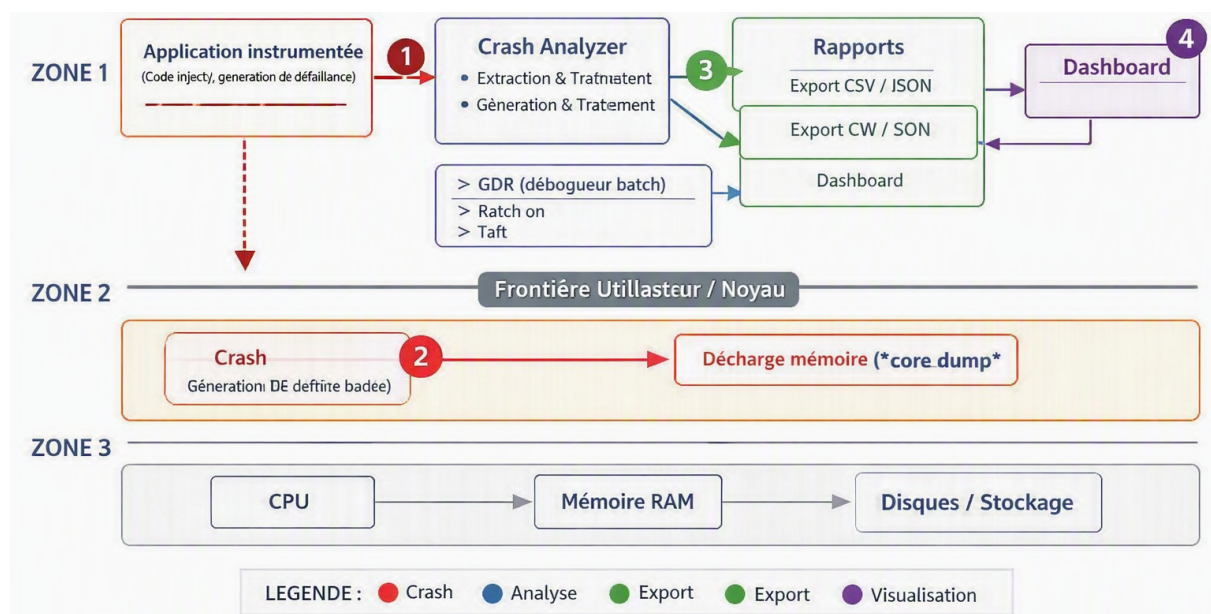


FIGURE 2.7 – Architecture globale de la solution *Crash Analyzer* et flux entre les espaces utilisateur, noyau et matériel

La séparation la plus importante se situe entre le noyau et l'espace utilisateur. Le matériel détecte l'exception, puis le noyau la traduit en signal fatal et produit le fichier core. *Crash_Analyzer* n'intervient qu'après cette production : il exploite le fichier core avec GDB, sans modifier l'application ni intercepter directement son exécution. Ce choix rend l'analyse non intrusive et donne la possibilité d'employer la solution avec une application tierce, à condition de disposer de l'exécutable correspondant et de ses symboles de débogage.

Ce tableau 2.3 résume le rôle et la technologie de chaque composant, sans reprendre le détail des échanges numérotés.

TABLE 2.3 – Synthèse des composants de l’architecture globale

Élément ou composant	Rôle principal	Technologie ou format
Application analysée	Exécuter l’opération invalide à l’origine du crash	C/C++ avec symboles de débogage
CPU, RAM et stockage	Détecter l’exception, conserver l’état du processus et stocker le fichier core	Matériel de la cible
Noyau Linux	Traduire l’exception en signal fatal et produire le fichier core	Linux, signaux POSIX
Fichier core	Conserver l’état du processus au moment de la dé-faillance	ELF
GDB	Charger le fichier core et l’exécutable, puis extraire le signal et le backtrace	GDB en mode batch
Crash_Analyzer	Structurer, classifier et exporter le diagnostic	C++17
Rapports générés	Persister les informations extraites et le diagnostic	CSV, JSON et SQLite
Dashboard_Creator	Présenter les diagnostics et l’historique à l’ingénieur	Python et Streamlit

L’architecture adopte un pipeline fondé sur deux interfaces persistantes. Le fichier core découple l’application en échec de l’analyseur, tandis que les rapports CSV, JSON et SQLite découplent l’analyseur du tableau de bord. Ainsi, une indisponibilité de l’interface Web n’empêche pas la production du diagnostic, et les résultats peuvent être consultés ultérieurement sans relancer GDB. Cette organisation facilite également l’évolution indépendante des règles de classification, des formats d’export et de l’interface de consultation.

Ce choix présente toutefois deux contraintes : l’analyse symbolique exige la version exacte de l’exécutable avec ses symboles, et la persistance des artefacts impose de gérer leur stockage et leur transfert. Ces contraintes sont acceptables au regard des bénéfices obtenus : fonctionnement local, traçabilité, reprise après interruption et limitation des pannes en cascade.

La contribution du projet réside dans l’intégration de mécanismes existants au sein d’une chaîne automatisée et cohérente : exploitation du fichier core Linux, pilotage de GDB, classification de la cause, proposition d’une correction, export multiformat et consultation de l’historique. Dans la figure 2.7, la fonction d’assistance du Dashboard correspond au *Crash Assistant*, dont les réponses sont déterministes et fondées sur les résultats de l’analyseur ; aucun service externe d’intelligence artificielle n’est utilisé.

2.5 Architecture logicielle des modules

Cette section expose les rôles des trois modules et justifie leur séparation. L’objectif n’est pas de répéter les besoins fonctionnels, mais de montrer comment les choix logiciels assurent la robustesse, la testabilité et la portabilité de la solution.

2.5.1 Module Crash_Simulator (C++)

Comme l’illustre la figure 2.8, le module *Crash_Simulator* produit les données de validation destinées à *Crash_Analyzer*. Il déclenche des défauts contrôlés dont la cause et le signal sont connus, servant ainsi de référence pour évaluer la qualité du diagnostic.

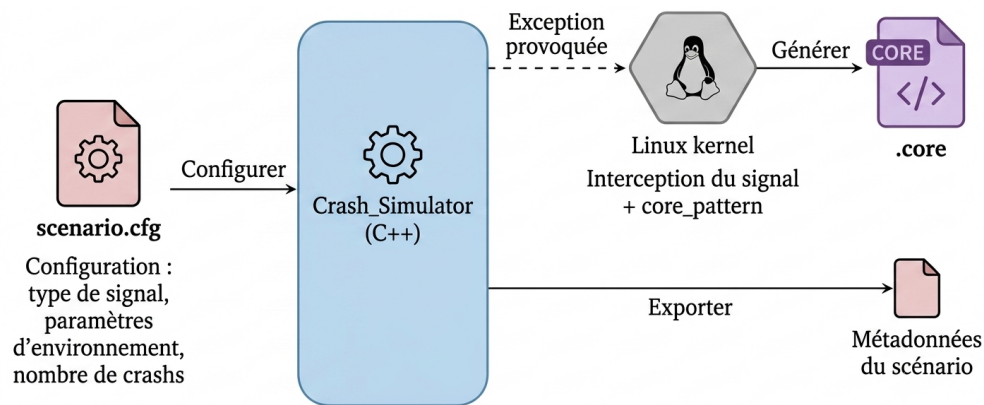


FIGURE 2.8 – Fonctionnement général et flux de données du module Crash_Simulator

CrashTypes définit les types de crash et les signaux POSIX associés. Randomizer produit les paramètres aléatoires et conserve la graine nécessaire à la reproduction d'une campagne. Simulator exécute le scénario sélectionné et déclenche la défaillance, tandis que le fichier `scenario.cfg` regroupe le type de crash, le nombre d'itérations et les autres paramètres de test. Les déclarations sont séparées dans le répertoire `include`, alors que les implémentations et le point d'entrée sont placés dans `src`.

Chaque scénario est exécuté dans un processus fils afin qu'un signal critique n'arrête ni le programme principal ni les essais suivants. Le module provoque l'opération invalide, mais le fichier core est produit par le noyau Linux conformément à `core_pattern` et à la limite définie par `ulimit`. Les sorties comprennent le fichier core et les métadonnées du scénario, qui sont ensuite comparés au rapport de Crash_Analyzer pour vérifier le signal détecté et la catégorie retournée.

Le choix du C++ assure le contrôle sur la mémoire, les signaux et les primitives POSIX. La séparation de ce module isole le code volontairement défaillant de la chaîne d'analyse et donne la possibilité d'enrichir les scénarios de test sans modifier Crash_Analyzer.

2.5.2 Module Crash_Analyzer (C++)

Comme l'illustre la figure 2.9, le module Crash_Analyzer représente le cœur du diagnostic. Il reçoit un fichier core et l'exécutable associé, puis produit un rapport structuré à partir des informations extraites et des symboles de débogage. Ces symboles permettent de relier les adresses mémoire aux fonctions et aux lignes du code source.

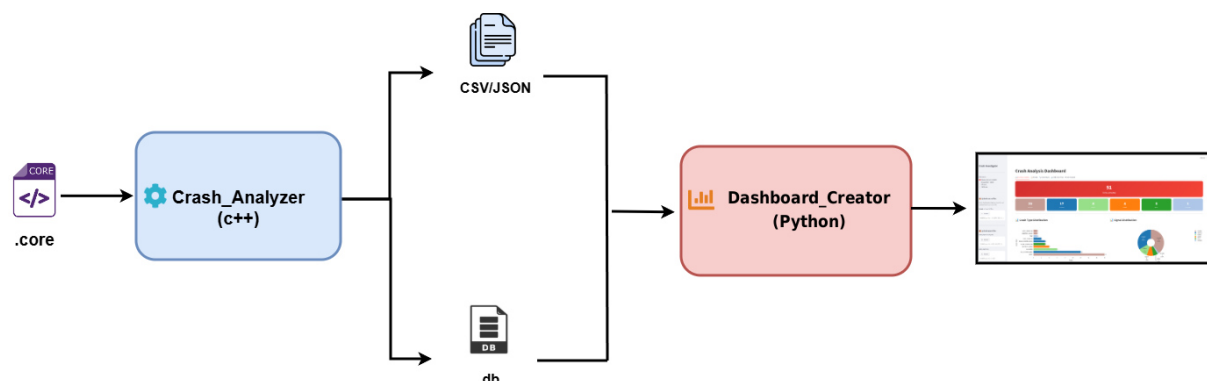


FIGURE 2.9 – Acheminement des données à travers le module Crash_Analyzer

Sur le plan interne, les déclarations des composants sont regroupées dans `include`, tandis que leurs implémentations et le point d'entrée `main.cpp` sont placés dans `src`. `CoreFileParser` pilote GDB et extrait le signal, le backtrace, les registres et le cadre fautif. `PatternMatcher` exploite ensuite ces données pour identifier la cause du crash, notamment dans le cas d'un SIGSEGV, puis `SolutionProposer` complète le diagnostic par une explication et une proposition de correction.

Les résultats sont transmis aux exporteurs CSV, JSON et SQLite dans le but d'assurer leur traçabilité et leur historisation. Le choix du C++ assure la maîtrise des processus et des appels système, tout en limitant la surcharge d'exécution sur la cible. Enfin, la séparation avec `Crash_Simulator` donne la possibilité d'analyser des applications tierces sans dépendre du code du générateur ni d'un scénario connu à l'avance.

2.5.3 Module Dashboard_Creator (Python)

Comme l'illustre la figure 2.10, le module `Dashboard_Creator` assure la restitution des diagnostics produits par `Crash_Analyzer`. Il s'exécute sur le PC hôte afin de préserver les ressources du Raspberry Pi et de maintenir la cible en mode *headless*, c'est-à-dire sans écran, clavier ni souris, avec une administration distante via SSH. La figure présente son fonctionnement général, de la lecture des rapports à leur visualisation.

FIGURE 2.10 – Acheminement des données à travers le module `Dashboard_Creator`

Le fichier `app_dash.py` représente le cœur du module : il initialise l'interface Streamlit, charge les données et orchestre les tableaux, les filtres, les graphiques et les vues détaillées. Le répertoire `coredumps/` conserve les fichiers core utiles à la consultation, `uploaded_sources/` reçoit les fichiers sources importés et `crash_test/` regroupe les scripts employés pour les essais du tableau de bord. Les fichiers `crash_report.csv` et `crash_report.json`, produits en amont par `Crash_Analyzer`, représentent des entrées structurées pour l'interface.

Le `Dashboard_Creator` exploite les trois formats de diagnostic selon leur usage. Le format CSV est retenu pour les tableaux et les échanges simples, tandis que JSON préserve le détail structuré de chaque incident. SQLite complète ces deux formats en facilitant les recherches et le calcul de statistiques sur l'historique.

Cette complémentarité assure une restitution complète sans imposer un format unique à tous les usages. Python et Streamlit ont été retenus pour créer rapidement les filtres, les graphiques et les vues détaillées sans développer séparément une API et une interface front-end. Le framework apporte un modèle d'exécution léger et des composants adaptés aux applications de données [9].

La séparation du `Dashboard_Creator` permet enfin de faire évoluer l'interface sans recompiler les binaires C++ ni modifier leur déploiement. Le principal compromis reste la dépendance à un environnement Python sur le poste hôte, qui ne partage toutefois pas les contraintes de ressources de la cible embarquée.

2.5.4 Justification architecturale d'ensemble

Ce tableau 2.4 synthétise les technologies retenues, les raisons de leur choix et les compromis associés. Les choix répondent directement aux contraintes du projet : accès aux mécanismes

bas niveau de Linux, portabilité vers ARM64, fonctionnement local, historisation légère et restitution accessible depuis le poste hôte.

TABLE 2.4 – Justification des principaux choix de l’architecture logicielle

Technologie ou décision	Justification	Compromis accepté
C++17 pour <code>Crash_Simulator</code> et <code>Crash_Analyzer</code>	Accès aux processus, aux signaux POSIX et à la mémoire, exécution native et compilation native ou croisée	Développement plus exigeant et gestion rigoureuse des ressources
GDB en mode batch	Extraction automatisée du signal, du backtrace, des registres et des threads sans session interactive	Sortie textuelle variable selon le signal, la version et l’architecture
Python, Streamlit et Plotly pour <code>Dashboard_Creator</code>	Construction rapide d’une interface Web interactive, de filtres et de graphiques orientés données	Environnement Python requis sur le poste hôte
SQLite sans serveur [10]	Historisation locale, fichier unique, faible consommation de ressources et déploiement simple	Accès concurrents massifs moins adaptés
CSV et JSON	Interopérabilité avec les tableurs, les scripts et d’autres outils, tout en conservant les données détaillées	Redondance partielle entre les formats exportés
CMake et chaîne de compilation ARM64	Réutilisation de la même logique de construction pour <code>x86_64</code> et <code>ARM64</code>	Dépendances de la cible à maintenir dans le sysroot

Ainsi, cette architecture privilégie donc la simplicité de déploiement et la reproductibilité. Les interfaces par fichiers introduisent un léger coût de synchronisation, mais elles permettent à chaque module d’évoluer indépendamment et assurent un fonctionnement local compatible avec les contraintes de confidentialité du projet.

Notre contribution dans cette phase de conception revient à avoir traduit les contraintes industrielles en une architecture modulaire vérifiable. Elle comprend la définition des responsabilités des trois modules, le choix des interfaces par fichiers, la structuration de `CrashInfo`, la séparation entre extraction, classification, recommandation et export, ainsi que la répartition des traitements entre le Raspberry Pi et le poste hôte. Les technologies employées sont existantes ; l’apport du projet tient à leur assemblage dans une chaîne post-mortem adaptée au contexte ARM64, locale et non intrusive.

2.5.5 Modularité et séparation des responsabilités

Les responsabilités sont réparties entre les différents composants de la chaîne d’analyse pour garantir une architecture modulaire et faiblement couplée. Le module `Crash_Simulator` assure produire les scénarios de défaillance, tandis que le noyau Linux assure la création et la sauvegarde des fichiers core. Le module `Crash_Analyzer` prend ensuite en charge l’analyse de ces fichiers et l’extraction des informations nécessaires au diagnostic. Enfin, `Dashboard_Creator` se consacre exclusivement à la visualisation des rapports d’analyse préalablement générés et persistés.

Le répertoire `coredumps_input/` représente l’interface d’échange entre le simulateur et l’analyseur, tandis que les formats CSV, JSON et SQLite assurent la transmission et la persistance des résultats entre l’analyseur et le tableau de bord. Ce découpage des responsabilités permet de limiter les dépendances entre les composants : chaque module peut évoluer indépendamment des autres, sous réserve du respect des interfaces d’échange définies. Cette organisation favorise la testabilité, la maintenance et l’évolutivité du système.

2.6 Diagramme d'activité global

Ce diagramme d'activité illustré par la figure 2.11 formalise le déroulement complet d'une campagne de test, depuis la compilation croisée jusqu'à la consultation du diagnostic. Il répartit les responsabilités entre l'ingénieur de test, le Raspberry Pi 4 ARM64, le noyau Linux embarqué et le PC hôte sous Ubuntu x86_64.

Après le transfert des binaires par SCP et leur lancement à distance par SSH, Crash_Simulator déclenche sur la cible un scénario configuré ou sélectionné aléatoirement. Le noyau intercepte le signal critique et produit le fichier core, que Crash_Analyzer exploite avec GDB pour produire les rapports CSV, JSON et SQLite. Ces résultats sont ensuite rapatriés vers le PC hôte et consultés dans le tableau de bord Streamlit. Ce flux assure ainsi la traçabilité de chaque incident, de sa production jusqu'à son diagnostic.

Le diagramme d'activité est justifié par la présence de responsabilités réparties sur plusieurs nœuds. Le premier choix distingue une application tierce du Crash_Simulator; le second distingue une catégorie configurée d'une sélection aléatoire. Les deux branches convergent avant l'intervention du noyau, ce qui confirme que la production du fichier core reste identique quelle que soit l'origine du crash. La barre de synchronisation précédant les exports montre que CSV, JSON et SQLite sont produits à partir du même diagnostic, avant le rapatriement des résultats et leur consultation sur le poste hôte. Cette vue valide ainsi l'enchaînement global sans détailler les classes internes, traitées dans les diagrammes suivants.

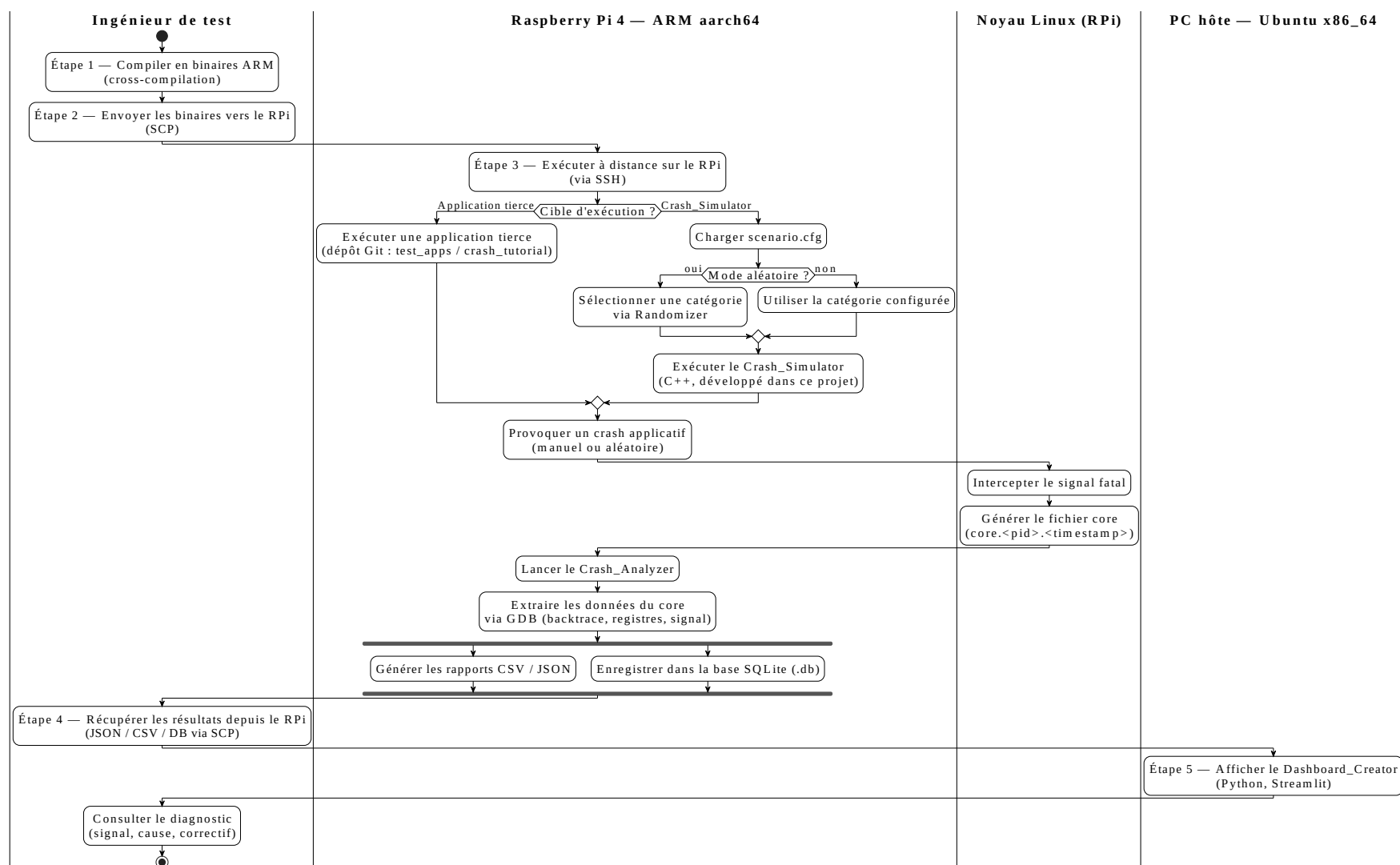


FIGURE 2.11 – Diagramme d'activité global de la solution *Crash Analyzer*

2.7 Conception des modules

Cette section détaille la conception détaillée des modules Crash_Analyzer et Crash_Simulator à travers leurs diagrammes de classes et de séquence.

2.7.1 Conception du module Crash_Analyzer

Ce module Crash_Analyzer applique à chaque fichier core une chaîne de traitements allant de l'invocation automatisée de GDB à l'extraction du backtrace et des registres, puis à l'identification de la cause probable, à la proposition d'une correction et à l'exportation du rapport.

a) Diagramme de classes

La figure 2.12 détaille la structure statique du module Crash_Analyzer. Elle conserve les attributs, les méthodes et les relations autour de la structure centrale CrashInfo, progressivement enrichie par les composants d'extraction, de classification, de recommandation et d'export.

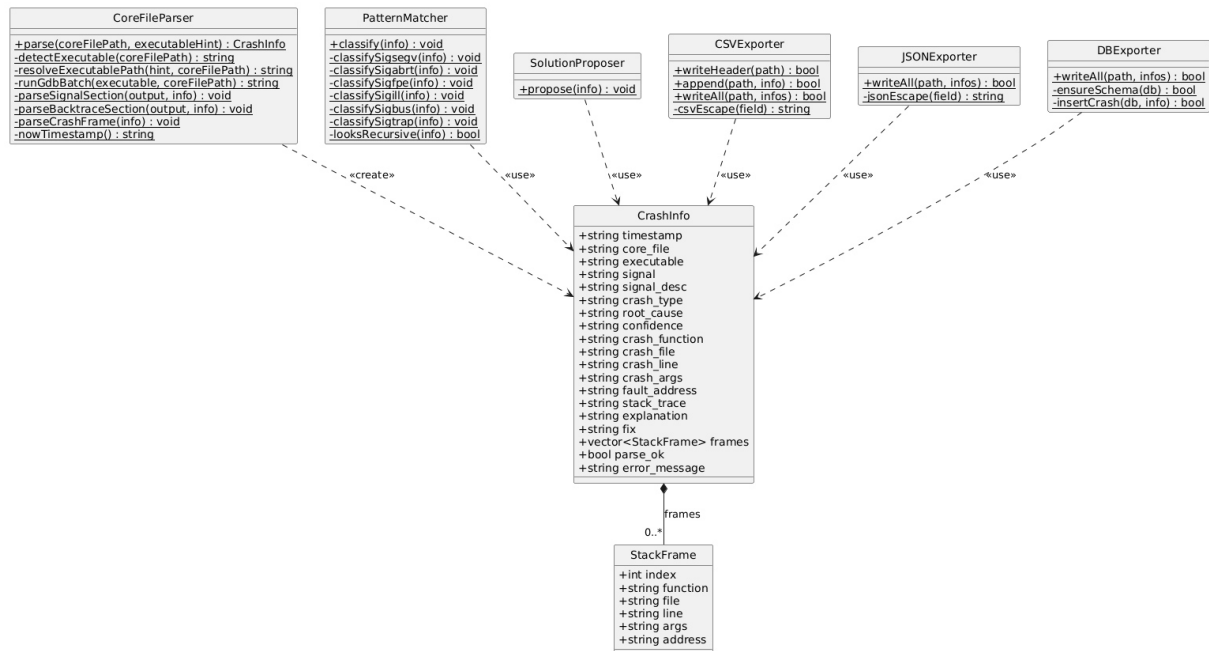


FIGURE 2.12 – Diagramme de classes du module Crash_Analyzer

Le tableau 2.5 complète le diagramme en résumant le rôle et le mécanisme majeur des composants du module.

CoreFileParser identifie l'exécutable associé au fichier core, pilote GDB et extrait le signal, la pile d'appels (*backtrace*), le cadre fautif et les registres.

PatternMatcher exploite ensuite ces informations pour déterminer la cause du crash et évaluer un niveau de confiance, sans recours à l'intelligence artificielle : la classification s'appuie sur une analyse de motifs (*pattern matching*), mise en œuvre au moyen d'expressions régulières et fondée sur des règles et des signatures connues.

Enfin, SolutionProposer complète le diagnostic par une explication et une proposition de correction. Les classes CSVExporter, JSONExporter et DBExporter assurent la persistance des objets CrashInfo, individuellement ou par lot, garantissant une séparation claire des responsabilités et une évolution indépendante des formats de sortie.

TABLE 2.5 – Synthèse des classes du module Crash_Analyzer

Élément ou composant	Rôle principal	Technologie ou format
CoreFileParser	Détecter l'exécutable, piloter GDB et extraire les informations du fichier core	C++17 et GDB batch
PatternMatcher	Classifier le crash à partir du signal et des motifs extraits	C++17, règles de classification
SolutionProposer	Ajouter l'explication et la proposition de correction	C++17
CrashInfo	Centraliser les données techniques et le diagnostic d'un incident	Structure C++17
StackFrame	Représenter une entrée de la pile d'appels	Structure C++17
CSVExporter, JSONExporter, DBExporter	Sérialiser et historiser les objets CrashInfo	CSV, JSON et SQLite

Le tableau montre que CrashInfo représente l'objet central du système, partagé entre les modules d'extraction, de classification, de recommandation et les trois mécanismes d'export.

b) Diagrammes de séquence

Les diagrammes de séquence permettent de représenter les interactions dynamiques entre les différents composants du module au cours de l'exécution d'une fonctionnalité donnée. Ils mettent en évidence l'enchaînement chronologique des messages échangés entre les classes du module Crash_Analyzer.

Ainsi, ce type de diagramme complète la vue statique précédente en montrant l'ordre effectif dans lequel CoreFileParser, PatternMatcher, SolutionProposer et les exporteurs transforment un fichier core en rapport structuré.

Diagramme de séquence de l'analyse des fichiers core.

La figure 2.13 illustre le traitement d'un dossier contenant un ou plusieurs fichiers core par le module Crash_Analyzer, depuis le lancement de la commande jusqu'à l'export des rapports.

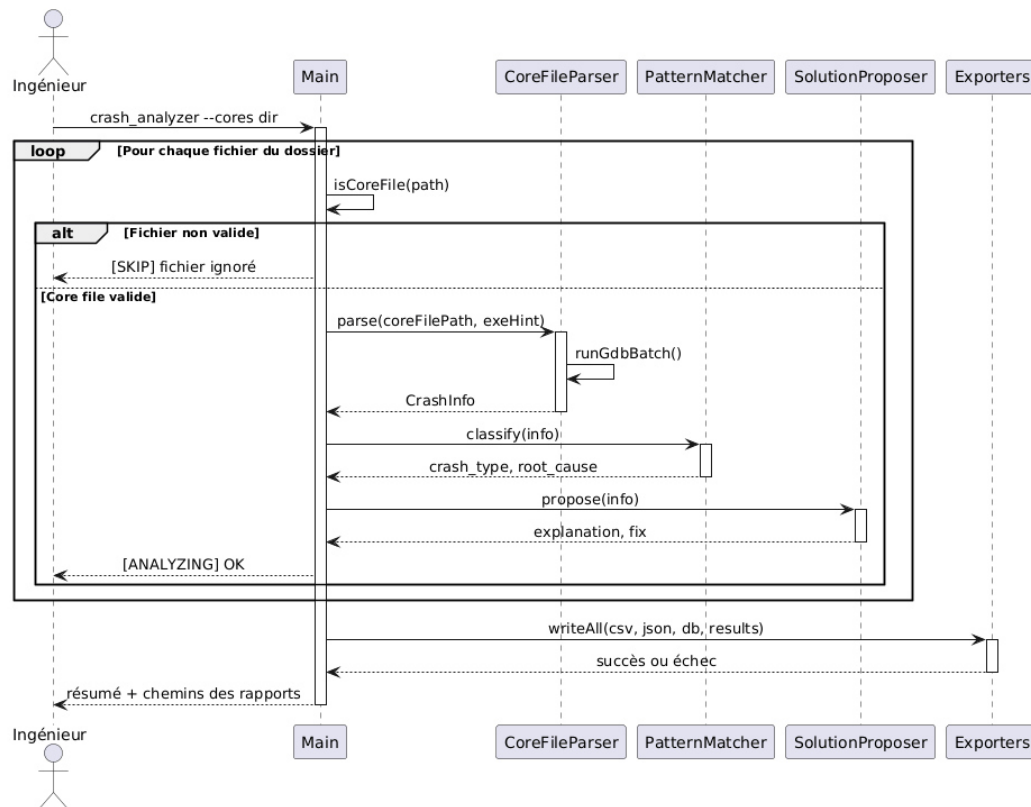


FIGURE 2.13 – Diagramme de séquence – analyse d’un dossier de fichiers core

Le processus débute lorsque l’ingénieur lance la commande `crash_analyzer -cores dir`, prise en charge par `Main`. Le programme parcourt alors le dossier et vérifie chaque chemin avec `isCoreFile(path)`. Lorsqu’un fichier n’est pas valide, il est ignoré sans interrompre le lot. Pour chaque fichier core exploitable, `Main` appelle ensuite `parse(coreFilePath, exeHint)`; `CoreFileParser` exécute GDB en mode batch avec `runGdbBatch()` et retourne un objet `CrashInfo`.

Une fois les informations extraites, `Main` les transmet à `PatternMatcher` par l’appel `classify(info)`, qui détermine le type de crash et sa cause racine. `SolutionProposer` exploite ensuite le même objet au moyen de `propose(info)` pour produire une explication et un correctif suggéré. Lorsque tous les fichiers ont été traités, `writeAll(csv, json, db, results)` produit les rapports CSV, JSON et SQLite. Le processus s’achève par l’affichage d’un résumé de l’analyse et des chemins des rapports destinés à l’ingénieur.

Ce diagramme met en évidence le rôle de `Main` dans l’orchestration du traitement par lot. `CoreFileParser`, `PatternMatcher` et `SolutionProposer` interviennent successivement pour construire le diagnostic de chaque fichier valide, tandis que les exporteurs rassemblent les résultats une fois la boucle terminée.

L’ordre des opérations est contraint par les données produites à chaque étape. La classification doit précéder la production de la recommandation, puisque le correctif dépend du type de crash identifié. L’export global intervient seulement après l’analyse de tous les fichiers valides, pour produire des rapports complets et cohérents.

Dans le diagramme de classes, `PatternMatcher::classify()` enrichit directement l’objet `CrashInfo`. La flèche de retour `crash_type, root_cause` du diagramme de séquence repré-

sente donc les champs renseignés après l'appel, et non un second objet indépendant. Cette lecture maintient la cohérence entre la signature statique de la méthode et le résultat fonctionnel montré dans la séquence.

2.7.2 Conception du module Crash_Simulator

L'architecture du module Crash_Simulator s'appuie sur une configuration `scenario.cfg` chargée par la classe Simulator. Selon cette configuration, Simulator emploie directement la catégorie demandée ou sollicite Randomizer, puis appelle la méthode `trigger*()` correspondante. L'opération invalide provoque un signal que le noyau Linux intercepte avant de produire le fichier core.

a) Diagramme de classes

La figure 2.14 détaille la structure statique du module Crash_Simulator. Son architecture s'organise autour de la classe Simulator, des structures de configuration et de rapport, de la classe Randomizer et de l'énumération commune CrashCategory.

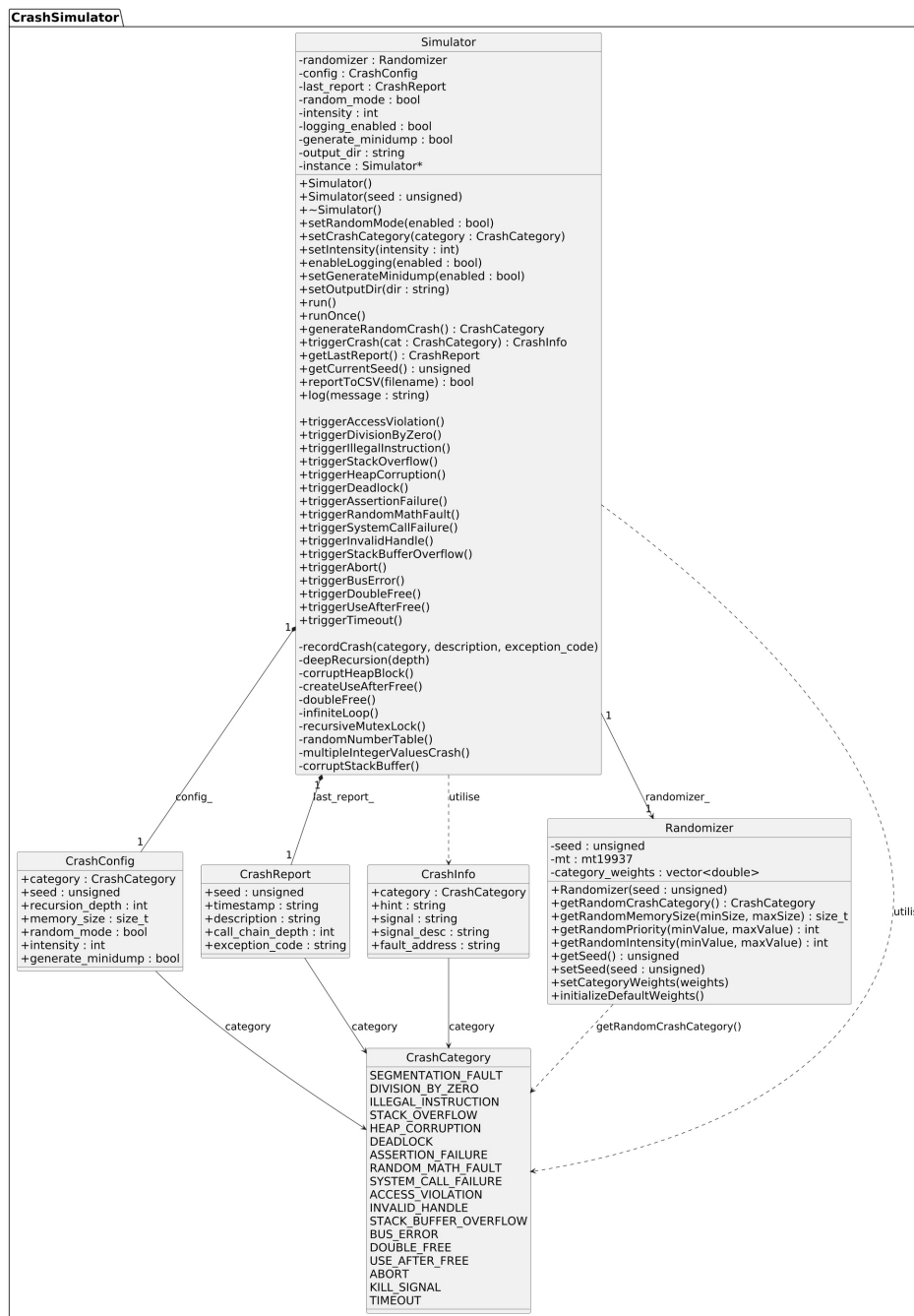


FIGURE 2.14 – Diagramme de classes du module Crash_Simulator

TABLE 2.6 – Synthèse des classes du module Crash_Simulator

Élément ou composant	Rôle principal	Technologie ou mécanisme
Simulator	Orchestrer la configuration, la sélection et le déclenchement des scénarios	C++17, patron Singleton
CrashConfig	Conserver les paramètres d'une campagne	Structure C++17
CrashReport	Conserver le résultat et les métadonnées du dernier crash	Structure C++17
CrashInfo	Décrire la catégorie, le signal et l'horodatage d'un scénario	Structure C++17
Randomizer	Produire des scénarios aléatoires reproductibles à partir d'une graine	std::mt19937
CrashCategory	Centraliser les catégories de crash prises en charge	Énumération C++17

Le tableau 2.6 synthétise l'organisation interne de Crash_Simulator. La répartition faiblement couplée des responsabilités favorise la reproductibilité des campagnes, la maintenance et l'ajout de scénarios.

b) Diagrammes de séquence

Les diagrammes de séquence représentent l'ordre des échanges entre les classes de Crash_Simulator et les systèmes externes sollicités pendant l'exécution.

Diagramme de séquence de la production d'un crash.

La figure 2.15 décrit le scénario nominal permettant de déclencher une défaillance configurée et de confirmer la création du fichier core.

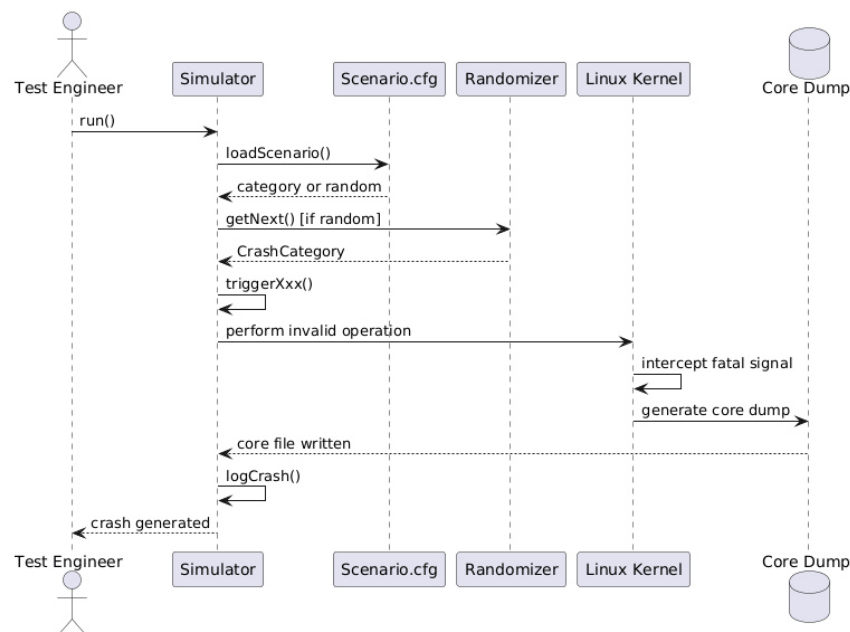


FIGURE 2.15 – Diagramme de séquence – scénario nominal de production d'un crash

La séquence commence lorsque l'opérateur de test appelle `run()`. Simulator charge alors `Scenario.cfg` avec `loadScenario()`. Si le mode aléatoire est actif, Randomizer choisit une valeur de `CrashCategory`; sinon, la catégorie configurée est utilisée directement. Simulator invoque ensuite la méthode `triggerXxx()` correspondante.

Le noyau Linux intercepte le signal fatal et crée le fichier core. Après la confirmation *core file written*, `logCrash()` consigne l'incident et le retour *crash generated* informe l'opérateur. Le diagramme distingue ainsi clairement les responsabilités : Simulator provoque l'opération invalide, tandis que le noyau produit le vidage mémoire.

La vue dynamique reste cohérente avec le diagramme de classes. Le fragment conditionnel correspond à `randomizer_`, `logCrash()` alimente `last_report_` et le libellé `getNext()` désigne l'opération `getRandomCrashCategory()`.

2.8 Interactions entre modules

Après leur étude individuelle, les modules sont reliés par un flux unidirectionnel allant de la production du crash à sa visualisation.

Crash_Simulator lit `scenario.cfg`, prépare les fichiers core et lance le scénario dans un processus enfant. Après l'écriture du core par le noyau, Crash_Analyzer retrouve l'exécutable, invoque GDB, classe l'incident et exporte le diagnostic. Sur le poste hôte, Dashboard_Creator exploite ensuite SQLite, CSV ou JSON pour construire l'interface Web.

Le découplage repose sur `coredumps_input/`, interface entre le simulateur et l'analyseur, puis sur `crash_report.csv`, `crash_report.json` et `crash_report.db`, qui transmettent les diagnostics au tableau de bord. SQLite conserve l'historique après l'arrêt de Crash_Analyzer.

La figure 2.16 précise la répartition des modules et des artefacts entre le poste hôte et la cible, ainsi que les protocoles d'échange.

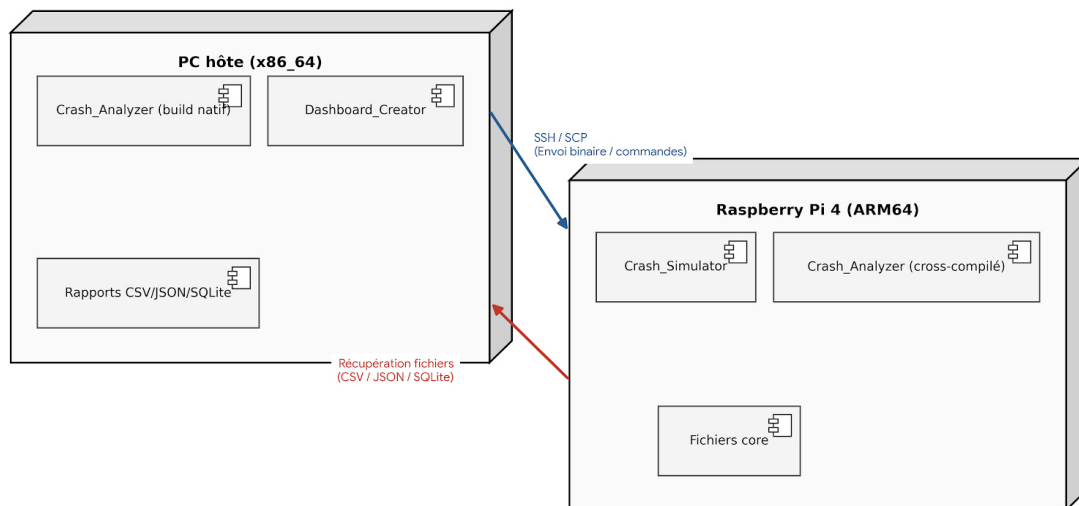


FIGURE 2.16 – Diagramme de déploiement UML de la solution entre le PC hôte et le Raspberry Pi 4

Le PC x86_64 construit les binaires, archive les rapports et exécute le tableau de bord. Le Raspberry Pi 4 ARM64 lance le simulateur et l'analyseur près des fichiers core. SSH/SCP assure l'envoi des binaires vers la cible et le rapatriement des rapports ; le tableau de bord consulte ainsi des artefacts persistants sans connexion directe à l'analyseur.

Ce découplage limite les pannes en cascade : l'absence ou la corruption d'un fichier core n'arrête pas le lot, et les exports restent consultables si Streamlit est indisponible.

Conclusion

Ce chapitre a établi les fondements de la solution. L'étude de Linux embarqué, d'ARM64, de la compilation croisée et des fichiers core a précisé les contraintes techniques, tandis que l'analyse des besoins a défini les fonctions et les exigences de qualité.

La conception sépare Crash_Simulator, Crash_Analyzer et Dashboard_Creator. Les diagrammes ont précisé leurs responsabilités, leurs échanges et leur implantation entre le Raspberry Pi 4 et le PC hôte. Le chapitre suivant présente leur mise en œuvre et leur validation sur ARM64.

Réalisation, intégration et validation

Sommaire

3.1	Introduction	39
3.2	Environnement de travail	40
3.2.1	Environnement matériel	40
3.2.2	Environnement logiciel	40
3.2.3	Choix technologiques justifiés	41
3.3	Développement du module Crash_Simulator	42
3.3.1	Objectifs et backlog du Sprint 1	42
3.4	Développement du module Crash_Analyzer	46
3.4.1	Objectifs et backlog du Sprint 2	46
3.5	Développement du Dashboard_Creator avec Streamlit	52
3.5.1	Objectifs et backlog du Sprint 3	52
3.6	Pipeline de déploiement sur la cible embarquée	58
3.6.1	Objectifs et backlog du Sprint 4	58
3.7	Analyse critique des limites constatées	65

3.1 Introduction

Ce chapitre expose la réalisation, l'intégration et la validation du système *Crash Analyzer*. Le travail est organisé en quatre sprints consacrés au *Crash_Simulator*, à *Crash_Analyzer*, au *Dashboard_Creator* et au déploiement sur Raspberry Pi 4. Cette organisation incrémentale s'inspire du cadre Scrum [12]. Chaque sprint expose successivement son objectif, sa démarche de réalisation, les difficultés rencontrées et les résultats obtenus.

La validation combine des tests fonctionnels, des tests d'intégration entre les modules et des essais de bout en bout sur ARM64. Une campagne de 51 fichiers core permet de vérifier la portabilité, la robustesse et la cohérence des diagnostics. Les tableaux et les captures d'écran présentés représentent les principales preuves des résultats obtenus.

Démarche de réalisation et flux de travail global

Cette figure 3.1 synthétise le flux de travail mis en œuvre, tandis que l'architecture générale de la solution est présentée par la figure 2.7 du chapitre 2. Le fonctionnement global s'organise en cinq étapes :

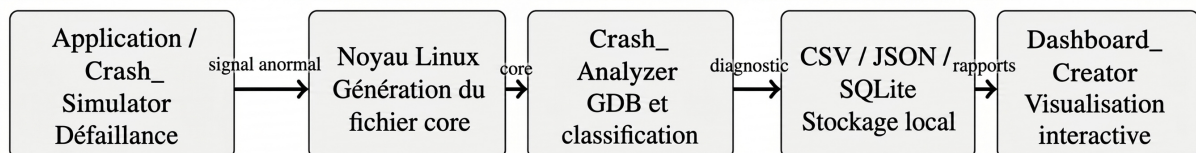


FIGURE 3.1 – Flux global, du crash applicatif à la visualisation du diagnostic

1. Crash de l'application : une défaillance survient sur le système Linux embarqué, soit dans une application tierce, soit dans un scénario contrôlé produit par le *Crash_Simulator* sur le Raspberry Pi 4 ;
2. Génération du fichier core : le noyau Linux intercepte le signal fatal, termine le processus fautif et produit un fichier *core dump* contenant son état au moment de l'incident ;
3. Analyse automatique : *Crash_Analyzer* traite le fichier core et pilote GDB dans le but d'extraire le signal, le backtrace, les registres et les informations relatives aux threads. Il classe ensuite la cause probable et produit un diagnostic structuré ;
4. Stockage et export : les diagnostics sont conservés localement dans une base SQLite et exportés aux formats CSV et JSON pour permettre leur historisation et leur exploitation par d'autres outils ;
5. Visualisation : *Dashboard_Creator*, développé avec Streamlit, lit les rapports et permet à l'ingénieur de visualiser, filtrer et explorer les incidents au moyen d'une interface Web interactive.

Les fichiers core puis les rapports structurés représentent ainsi les interfaces entre les modules. Ce découplage permet de remplacer ou de faire évoluer un composant sans modifier l'ensemble de la chaîne.

3.2 Environnement de travail

3.2.1 Environnement matériel

Le projet s'appuie sur deux machines complémentaires : un PC hôte utilisé pour le développement et une cible embarquée Raspberry Pi 4 destinée à l'exécution et à la validation de la solution. Cette carte intègre un processeur Cortex-A72 64 bits adapté à l'évaluation d'une chaîne ARM64 [11]. Le tableau 3.1 expose les caractéristiques utiles dans le cadre de nos essais.

TABLE 3.1 – Caractéristiques des machines utilisées dans le projet

Caractéristique	PC hôte (développement)	Raspberry Pi 4 (cible embarquée)
Architecture	x86_64	ARM64 (aarch64, Cortex-A72)
Système d'exploitation	Ubuntu 22.04 LTS	Debian
Rôle principal	Développement, compilation croisée, tableau de bord	Exécution du <code>Crash_Simulator</code> et de <code>Crash_Analyzer</code> , fichiers core
Connexion	Client SSH/SCP vers la cible	Serveur SSH activé

3.2.2 Environnement logiciel

L'environnement logiciel rassemble les outils, langages et bibliothèques que nous avons utilisés pour développer, compiler, transférer et valider la solution.

a) Outils de développement

L'environnement de développement réunit les outils nécessaires à la compilation native et croisée, au débogage et à l'exécution du tableau de bord. Le tableau 3.2 expose les versions vérifiées sur le poste hôte dans le but d'assurer la reproductibilité de la chaîne de construction.

Outil vérifié	Version utilisée
Compilateur natif g++	11.4.0
Système de construction CMake	3.22.1
Débogueur GDB	12.1
Compilateur croisé aarch64-linux-gnu-g++	11.4.0
Interpréteur Python	3.10.12

TABLE 3.2 – Versions des outils de développement utilisés

Ainsi, cette vérification confirme la disponibilité du compilateur C++, de CMake, de GDB, de la chaîne de compilation croisée ARM64 et de Python. Ces outils couvrent ainsi la construction des modules, l'analyse des fichiers core et la visualisation des diagnostics.

Le projet a été versionné avec Git et hébergé sur GitLab ACTIA. La stratégie retenue reposait sur quatre branches de travail, une pour chaque sprint, complétées par des commits réguliers associés aux évolutions de `Crash_Simulator`, `Crash_Analyzer`, `Dashboard_Creator` et de la chaîne de déploiement. Cette organisation a assuré la traçabilité des modifications et facilité l'intégration progressive des modules.

La figure 3.2 expose l'historique Git et l'organisation des branches utilisées pendant le développement.

Cette figure met en évidence les branches dédiées aux Sprints 2, 3 et 4 ainsi que leurs fusions successives dans la branche principale. Le premier sprint désigne l'initialisation et à la


```

user@host:~/project$ git log --oneline --graph --all --decorate
* a3f21c9 (HEAD -> main) merge: intégration sprint 4 - assistant
|\
| * 7c8e410 (sprint4-assistant) feat: règles de correctifs par catégorie
| * 4d1a09b feat: SolutionProposer squelette
|/
* 9b6f7e2 merge: intégration sprint 3 - dashboard
|\
| * 5e2c881 (sprint3-dashboard) feat: export CSV/JSON depuis le dashboard
| * 1f4b330 fix: rafraîchissement liste CrashInfo
| * 0a7c2e4 feat: page Streamlit initiale
|/
* 6a90cd4 merge: intégration sprint 2 - analyzer
|\
| * 2d7e819 (sprint2-analyzer) feat: classification PatternMatcher
| * 88a1c05 feat: pilotage GDB en mode batch
|/
* 1a0e774 init: structure CMake du projet

user@host:~/project$ git log --oneline | wc -l
27

```

FIGURE 3.2 – Représentation de l’historique des commits sur GitHub et organisation des branches par sprint

structuration du projet. L’historique comporte au total 27 commits, ce qui assure la traçabilité des principales étapes de développement.

b) Langages et bibliothèques

La solution combine C++17 et Python 3 selon les responsabilités de chaque module. C++17 sert à Crash_Simulator et Crash_Analyzer, car il donne la possibilité d’accéder aux mécanismes système Linux, de manipuler les processus et les signaux, et de produire des binaires compatibles avec ARM64. Python est réservé au Dashboard_Creator, développé avec Streamlit et Plotly pour charger les rapports, calculer les indicateurs et produire les visualisations interactives. SQLite assure enfin la persistance locale des diagnostics sans requiertr de serveur. La section A.3 des annexes récapitule ces technologies et leurs usages.

c) Outil de débogage

L’outil central de débogage est GDB 12.1, piloté en mode batch par Crash_Analyzer. Cette exécution non interactive automatise l’extraction du signal, du backtrace, du cadre fautif, des registres et des informations relatives aux threads. Elle assure un traitement reproductible de plusieurs fichiers core et apporte les données techniques ensuite exploitées par le mécanisme de classification.

3.2.3 Choix technologiques justifiés

Le tableau 3.3 synthétise les majeures technologies retenues et les raisons de leur adoption dans le cadre du projet.

C++17 a été retenu pour Crash_Simulator et Crash_Analyzer en raison de son exécution native, de son accès aux primitives Linux et de sa compatibilité ARM64. CMake emploie une même configuration pour les constructions native et croisée, ce qui rend la production des binaires reproductible [8].

GDB fournit l’analyse automatisée des fichiers core sur les deux architectures. SQLite préserve localement les diagnostics sans serveur [10], tandis que Streamlit et Plotly assurent leur consultation au moyen d’une interface Web interactive [9].

TABLE 3.3 – Synthèse des choix technologiques et justifications

Composant	Technologie retenue	Justification
Crash_Simulator et Crash_Analyzer	C++17 / g++ / chaîne de compilation ARM64	Exécution native, bas niveau, portabilité
Base de données	SQLite3	Léger, sans serveur, embarqué
Dashboard_Creator	Python 3 / Streamlit / Plotly	Interface Web rapide et interactive
Système de construction	CMake	Construction native et croisée ARM64
Débogueur	GDB batch	Standard Linux, scriptable, multiarchitecture
Versionnement	Git / GitLab ACTIA	Versionnement et traçabilité

Le Raspberry Pi 4 représente une cible ARM64 accessible pour valider la chaîne complète, sans toutefois remplacer la carte cliente finale d'ACTIA. Ces choix permettent au premier sprint de produire des crashes maîtrisés servant de référence au développement de Crash_Analyzer.

3.3 Développement du module Crash_Simulator

3.3.1 Objectifs et backlog du Sprint 1

L'objectif du Sprint 1 est de disposer d'un générateur de crashes contrôlés, reproductibles et isolés, capable d'alimenter les étapes suivantes avec des fichiers core représentatifs. Le sprint doit couvrir les majeures familles de signaux, permettre une sélection déterministe ou aléatoire des scénarios et poursuivre une campagne après la terminaison fatale d'un processus fils.

a) Backlog et périmètre fonctionnel

Le tableau 3.4 détaille le backlog du module Crash_Simulator. Il regroupe les user stories retenues, leur priorité et l'estimation de leur charge en points.

TABLE 3.4 – Backlog du Sprint 1 — module Crash_Simulator

ID	User Story	Priorité	Points
US-01	En tant qu'ingénieur, je veux produire un crash SIGSEGV contrôlé pour tester Crash_Analyzer	Haute	3
US-02	En tant qu'ingénieur, je veux choisir le type de crash via un fichier de configuration	Haute	2
US-03	En tant qu'ingénieur, je veux que le Crash_Simulator configure automatiquement core_pattern et ulimit	Haute	2
US-04	En tant qu'utilisateur, je veux exécuter une campagne de tests en mode aléatoire avec sauvegarde de la graine pour reproduire exactement la même séquence de crashes	Haute	3
US-05	En tant qu'ingénieur, je veux couvrir les 18 types de crash de CrashCategory	Haute	5

b) Diagramme de cas d'utilisation

La figure 3.3 expose les fonctions retenues pour ce premier sprint et les interactions de l'ingénieur de test avec le Crash_Simulator.

Le diagramme résume la préparation de l'environnement, le choix du mode, le déclenchement du crash et la vérification du fichier core dans coredumps_input/. Les relations *include* imposent la configuration préalable de core_pattern et de ulimit. L'ingénieur pilote la campagne, tandis que le noyau produit le core; le mode déterministe valide un cas précis et la graine du mode aléatoire assure la reproductibilité des scénarios.

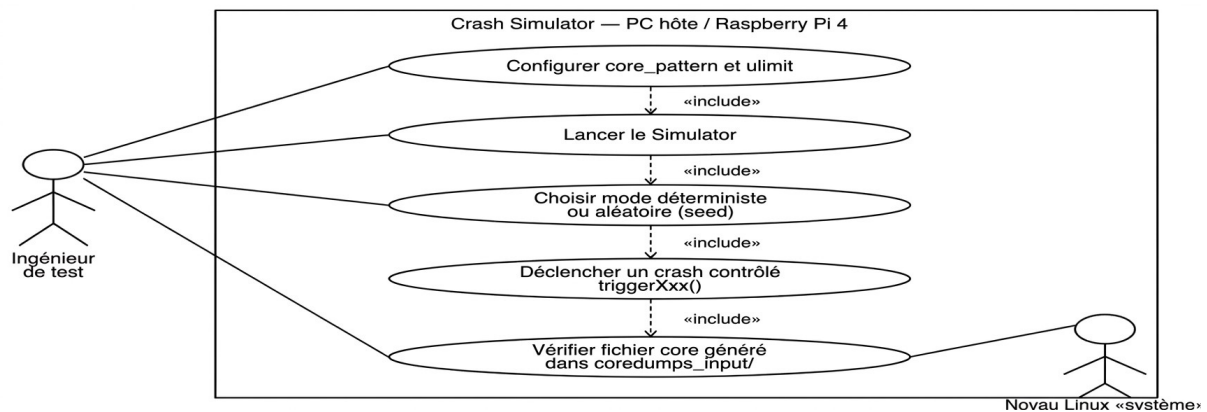


FIGURE 3.3 – Diagramme de cas d'utilisation du Sprint 1 — module Crash_Simulator

3.3.2 Démarche de réalisation

Cette réalisation suit quatre étapes : définir les catégories dans `CrashTypes.hpp`, implémenter chaque défaut, configurer la sélection via `scenario.cfg`, puis isoler chaque exécution dans un processus fils après préparation du noyau.

Ce découpage a facilité le diagnostic : lorsqu'un résultat différait du signal attendu, nous pouvions vérifier séparément le scénario, la configuration système et la collecte du statut du processus fils. Il évite également de confondre une erreur du `Crash_Simulator` avec une absence de production du fichier core par le noyau.

a) Implémentation — classes `Simulator` et `Randomizer`

Le module s'organise en plusieurs fichiers C++17. Le fichier `simulator_main.cpp` représente le point d'entrée, lit `scenario.cfg` et isole chaque scénario au moyen de `fork()`. `Simulator.cpp` regroupe un ensemble de méthodes privées `triggerXxx()` dédiées à la simulation des différents types de crashes, tandis que `Randomizer.cpp` implémente le générateur pseudo-aléatoire pondéré.

Le fichier `CrashTypes.hpp` définit l'énumération `CrashCategory` ainsi que les structures `CrashConfig` et `CrashReport`. Enfin, `scenario.cfg` rassemble les paramètres de campagne, ce qui permet de faire évoluer indépendamment le déclenchement des crashes, la sélection aléatoire et les structures partagées.

Les méthodes de déclenchement couvrent divers signaux et erreurs courantes : `SIGSEGV`, `SIGABRT`, `SIGFPE`, `SIGBUS`, `SIGILL`, `SIGTRAP`, ainsi que des fautes logicielles telles que la double libération, le *use-after-free*, le débordement de pile, la corruption du tas ou l'interblocage. Chaque méthode regénère une cause spécifique.

`Randomizer` emploie `std::mt19937`, une graine enregistrée et des pondérations par catégorie. Le fichier `scenario.cfg` externalise le nombre, l'intensité et le répertoire des scénarios, sans requiertr de recompilation.

Des exemples de `scenario.cfg`, de scripts de configuration et de rapports générés sont décrits dans la section A.2 des annexes.

b) Configuration des fichiers core avec `setup_coredump.sh`

Avant chaque campagne, `setup_coredump.sh` active la production avec `ulimit -c unlimited`, prépare `coredumps_input/` et configure un nom contenant le PID et l'horodatage. Le script relit ensuite les paramètres actifs pour vérifier leur application effective.

Ce contrôle évite un échec observé sous Ubuntu : un `core_pattern` commençant par `|` transmet le crash à un gestionnaire externe au lieu de créer le fichier attendu. La figure 3.4 met en évidence l'exécution avec `sudo`, le répertoire choisi, la limite autorisée et les valeurs relues dans le noyau.

```
dghriss@actia.local@H09-DSK-PFE01: ~/Bureau/pfe/2_juillet/pfe_2026-main/crash_simulator_project/$ sudo bash scripts/setup_coredump.sh
[sudo] Mot pass de passe de dghriss@actia.local :
Making binaries executable...
Binaries are now executable
```

Linux Kernel Coredump Setup

```
Dump directory: ../crash_analyzer_project/coredumps_input
ulimit -c unlimited (current session)
core_pattern -> ../crash_analyzer_project/coredumps_input/core.%p.%t
core_uses_pid = 1

— Current Status —
core_pattern : ../crash_analyzer_project/coredumps_input/core.%p.%t
core_uses_pid : 1
ulimit -c : unlimited
dump dir : ../crash_analyzer_project/coredumps_input

Coredump setup complete.
Core files will be named: core.<exe>.<pid>.<timestamp>
```

FIGURE 3.4 – Configuration et vérification des paramètres de production des fichiers core

Ces valeurs affichées confirment que l'écriture directe des fichiers core est correctement préparée pour la campagne.

c) Compilation du module avec CMake

La figure 3.5 détaille la configuration CMake, la compilation des sources et la création de `build/bin/crash_simulator`. Les options utilisées pour produire un exécutable adapté à l'analyse des crashes sont synthétisées dans le tableau 3.5.

TABLE 3.5 – Options de compilation CMake du module Crash_Simulator

Option	Rôle
<code>-g</code>	Ajoute les symboles de débogage nécessaires pour relier les adresses aux fonctions et aux lignes du code source.
<code>-O0</code>	Désactive les optimisations susceptibles de modifier le comportement des scénarios.
<code>-rdynamic</code>	Exporte les symboles de l'exécutable dans le but d'améliorer leur résolution dans les backtraces.
<code>-pthread</code>	Active la compilation et l'édition de liens avec la bibliothèque POSIX Threads, utilisée par les scénarios concurrents.

```
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main$ cd crash_simulator_project
mkdir -p build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Debug
cmake --build . --parallel $(nproc)
cd ..
-- The CXX compiler identification is GNU 11.4.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/dghriss@actia.local/Bureau/pfe/2_juillet/pfe_2026-main/crash_simulator_project/build
[ 25%] Building CXX object CMakeFiles/crash_simulator.dir/simulator/src/simulator_main.cpp.o
[ 50%] Building CXX object CMakeFiles/crash_simulator.dir/simulator/src/simulator.cpp.o
[ 75%] Building CXX object CMakeFiles/crash_simulator.dir/simulator/src/Randomizer.cpp.o
[100%] Linking CXX executable bin/crash_simulator
[100%] Built target crash_simulator
```

FIGURE 3.5 – Compilation native en mode Debug du module Crash_Simulator avec CMake

Le message final confirme une construction sans erreur. Pour le scénario de débordement de pile, l'association de `-O0` et d'une variable volatile empêche l'optimisation de la récursion

terminale et préserve le crash attendu. La même configuration CMake est ensuite réemployée avec la chaîne de compilation ARM64.

d) Exécution et production des crashes

Le simulateur est exécuté à l'aide de la commande `sudo ./build/bin/crash_simulator -config scenario.cfg`. La campagne comporte dix itérations, avec une intensité définie dans le fichier de configuration `scenario.cfg`. La figure 3.6 expose, pour les derniers scénarios, la catégorie, la graine, le signal ainsi que la recommandation associée, notamment pour les signaux SIGABRT (cas bloquant) et SIGBUS.

```
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/crash_simulator_project$ sudo ./build/bin/crash_simulator --config scenario.cfg
Seed      : 1555700036
Info      : Runtime assertion failed
Hint      : Fix the assertion condition. Check preconditions.
Coredump generated (signal 6)

=====

Crash 9 / 10
Category  : Deadlock / Infinite Loop
Seed      : 1858628387
Info      : Process stuck in infinite loop or deadlock
Hint      : Review mutex usage. Add timeout to blocking operations.
Coredump generated (signal 11)

=====

Crash 10 / 10
Category  : Bus Error
Seed      : 2259890351
Info      : Bus error - misaligned memory access or hardware fault
Hint      : Check pointer alignment (especially on ARM). Validate mmap usage.
Coredump generated (signal 7)

=====

Simulator done. 10/10 crash(es) simulated.
Coredumps (if generated): ../crash_analyzer_project/coredumps_input
```

FIGURE 3.6 – Exécution des derniers scénarios et bilan de la campagne du Crash_Simulator

Le bilan 10/10 valide l'isolation par processus fils : chaque crash reste local au scénario courant et le processus principal poursuit le lot. Les graines enregistrées permettent de reproduire la même campagne.

La figure 3.7 complète cette validation par un test d'intégration : Crash_Analyzer relie chaque fichier core au signal reçu et à la catégorie détectée.

```
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/crash_analyzer_project/build$ ./crash_analyzer --cores ../coredumps_input --out ./analyzer_output
=== Crash Analyzer (C++) ===
Scanning: ../coredumps_input
[ANALYZING] core.unique_crash_ap.284372.1783499749 ... OK [SIGABRT / ABORT]
[ANALYZING] core.advanced_crash_.289124.1783501297 ... OK [SIGABRT / ABORT]
[ANALYZING] core.crash_simulator.290286.1783501551 ... OK [SIGABRT / ASSERTION_FAILURE]
[ANALYZING] core.crash_simulator.290285.1783501551 ... OK [SIGABRT / HEAP_CORRUPTION]
[ANALYZING] core.crash_simulator.290287.1783501552 ... OK [SIGABRT / HEAP_CORRUPTION]
[ANALYZING] core.advanced_crash_.286955.1783500633 ... OK [SIGABRT / ABORT]
[ANALYZING] core.advanced_crash_.286948.1783500633 ... OK [SIGTRAP / TRAP]
[ANALYZING] core.unique_crash_ap.284371.1783499749 ... OK [SIGABRT / ABORT]
[ANALYZING] core.advanced_crash_.286950.1783500633 ... OK [SIGABRT / HEAP_CORRUPTION]
[ANALYZING] core.unique_crash_ap.288871.1783501264 ... OK [SIGABRT / ABORT]
[ANALYZING] core.crash_simulator.290284.1783501551 ... OK [SIGABRT / ABORT]
[ANALYZING] core.crash_simulator.290290.1783501552 ... OK [SIGSEGV / NULL_POINTER_DEREF]
[ANALYZING] core.unique_crash_ap.284374.1783499749 ... OK [SIGABRT / ABORT]
[ANALYZING] core.unique_crash_ap.284373.1783499749 ... OK [SIGABRT / ABORT]
[ANALYZING] core.advanced_crash_.286951.1783500633 ... OK [SIGABRT / HEAP_CORRUPTION]
[ANALYZING] core.unique_crash_ap.288870.1783501264 ... OK [SIGABRT / HEAP_CORRUPTION]
[ANALYZING] core.unique_crash_ap.288850.1783501264 ... OK [SIGABRT / ABORT]
[ANALYZING] core.crash_simulator.290246.1783501548 ... OK [SIGBUS / BUS_ERROR]
[ANALYZING] core.advanced_crash_.286949.1783500633 ... OK [SIGSYS / OTHER_SIGSYS]
[ANALYZING] core.unique_crash_ap.288862.1783501264 ... OK [SIGABRT / ABORT]
```

FIGURE 3.7 – Exécution de Crash_Analyzer sur un lot de fichiers core générés par le Crash_Simulator — détection des signaux et types de crash

Cette correspondance confirme que les fichiers générés par le simulateur sont directement exploitables par l'analyseur.

3.3.3 Résultats et validation

La validation comporte deux niveaux : six scénarios représentatifs exécutés séparément pour comparer le signal obtenu au signal attendu, puis une campagne aléatoire de dix itérations pour vérifier la continuité du traitement. Le tableau 3.6 met en évidence que chaque scénario produit le signal attendu et un fichier core exploitable.

TABLE 3.6 – Résultats des tests fonctionnels du module `Crash_Simulator`

Scénario déclenché	Signal attendu	Core dump généré	Résultat
Déréférencement pointeur nul	SIGSEGV	Oui	OK
Récursion infinie (stack overflow)	SIGSEGV	Oui	OK
Division entière par zéro	SIGFPE	Oui	OK
Appel explicite <code>abort()</code>	SIGABRT	Oui	OK
Accès mémoire mal aligné	SIGBUS	Oui	OK
Instruction illégale	SIGILL	Oui	OK

Cette campagne automatique atteint dix exécutions sur dix. Ce résultat confirme que l'isolation des scénarios empêche une défaillance d'interrompre le lot, tandis que les graines enregistrées assurent sa reproductibilité. Les fichiers core sont ensuite reconnus par `Crash_Analyzer`, comme l'illustre la figure 3.7, ce qui valide l'intégration entre les deux modules.

Le Sprint 1 a permis de livrer un ensemble cohérent de scénarios, le module `Randomizer`, la configuration des fichiers core ainsi que le protocole de validation. Cette contribution représente la base des entrées contrôlées nécessaires au développement et aux tests de `Crash_Analyzer`, avant son application sur des fichiers core issus d'applications tierces.

3.4 Développement du module `Crash_Analyzer`

3.4.1 Objectifs et backlog du Sprint 2

L'objectif du Sprint 2 est de transformer automatiquement un fichier core brut en un diagnostic structuré. Le résultat attendu englobe l'identification du signal, l'extraction du contexte GDB, la classification de la cause probable, l'estimation d'un niveau de confiance, la proposition d'une piste de correction et l'export dans trois formats complémentaires.

a) Backlog et périmètre fonctionnel

Ce backlog du Sprint 2 (tableau 3.7) priorise le pilotage reproductible de GDB et la classification, dont dépend la fiabilité des exports CSV, JSON et SQLite.

TABLE 3.7 – Backlog du Sprint 2 — module Crash_Analyzer

ID	User Story	Priorité	Points
US-06	En tant qu'ingénieur, je veux que Crash_Analyzer détecte automatiquement les fichiers core ELF	Haute	2
US-07	En tant qu'ingénieur, je veux que GDB soit piloté en mode batch de façon reproductible	Haute	5
US-08	En tant qu'ingénieur, je veux obtenir le signal, le backtrace et la frame #0 pour chaque crash	Haute	5
US-09	En tant qu'ingénieur, je veux une classification automatique avec niveau de confiance	Haute	8
US-10	En tant qu'ingénieur, je veux un correctif suggéré en langage naturel pour chaque crash	Moyenne	3
US-11	En tant qu'ingénieur, je veux les rapports exportés en CSV, JSON et SQLite	Haute	5

b) Diagramme de cas d'utilisation du Sprint 2

La figure 3.8 expose les fonctions prises en charge par Crash_Analyzer et les échanges avec GDB.

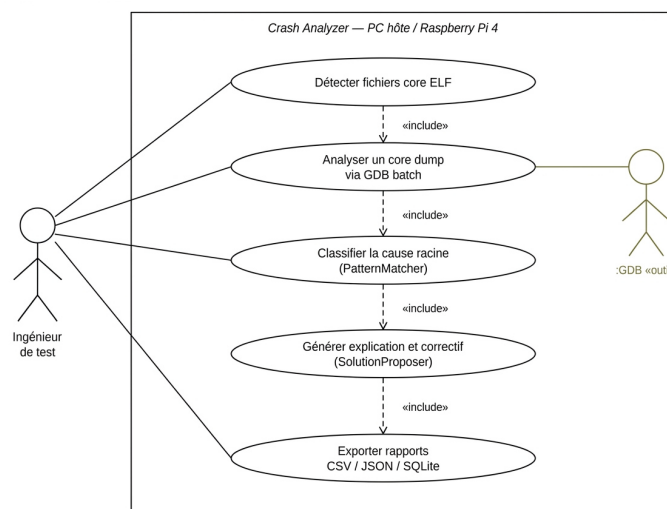


FIGURE 3.8 – Diagramme de cas d'utilisation du Sprint 2 — analyse, classification et export des fichiers core

Le diagramme montre que l'ingénieur déclenche une analyse sans intervenir dans la session du débogueur. Crash_Analyzer détecte les fichiers core ELF, pilote GDB, classe la cause, produit une proposition de correction puis alimente les trois formats de sortie. Ce découpage a guidé l'organisation des classes CoreFileParser, PatternMatcher, SolutionProposer et des exporteurs.

3.4.2 Démarche de réalisation

La démarche suit le flux réel de traitement d'un incident : compilation et contrôle de l'architecture, extraction du contexte avec GDB, classification par règles, production d'une

explication, puis persistance des résultats. Ce découpage permet de tester séparément chaque transformation avant de valider la chaîne complète.

a) Compilation avec CMake

Avant l'intégration fonctionnelle, nous avons d'abord compilé Crash_Analyzer nativement en mode Debug, puis vérifié la production du binaire ARM64 avec CMake et la chaîne `aarch64-linux-gnu-g++`. Des répertoires de construction distincts empêchent le mélange des objets x86_64 et ARM64. Après l'édition de liens, la commande `file` contrôle systématiquement l'architecture du binaire avant son exécution ou son transfert.

b) CoreFileParser : pilotage GDB en mode batch

```
user@raspberrypi:~/crash_analyzer$ gdb -batch -x commands.gdb ./crash_app
coredumps_input/core.1234
===SIGINFO===
Program terminated with signal SIGSEGV, Segmentation fault.
#0 0x0000556f3a2b1234 in process_buffer (buf=0x0) at buffer.c:42
===BACKTRACE===
#0 0x0000556f3a2b1234 in process_buffer (buf=0x0) at buffer.c:42
#1 0x0000556f3a2b1189 in handle_request (req=0x7ffe4c2a1d20) at server.c:87
#2 0x0000556f3a2b1050 in main (argc=1, argv=0x7ffe4c2a1e58) at main.c:15
===FRAME0===
#0 0x0000556f3a2b1234 in process_buffer (buf=0x0) at buffer.c:42
42      size_t len = strlen(buf);
Arguments: buf = 0x0
Locals: len = <optimized out>
===THREADS===
  Id  Target Id                Frame
* 1   Thread 0x7f... "crash_app" process_buffer (buf=0x0) at buffer.c:42
  2   Thread 0x7f... "worker-1" 0x00007f8a... in pthread_cond_wait

[GDB batch mode execution completed successfully]
user@raspberrypi:~/crash_analyzer$
```

Exécution de GDB en mode batch et extraction structurée du contexte d'un crash SIGSEGV

c) Évolution de la stratégie de détection des signaux

La méthode `parseSignalSection()` analyse la sortie de GDB au moyen de trois expressions régulières complémentaires. Leur ordre permet de privilégier une extraction précise, puis d'employer une règle plus tolérante si le format varie selon la version de GDB ou la plateforme.

1. Extraction du signal et de sa description. La première expression régulière recherche la ligne standard produite par GDB :

```
std::regex sigLineRe(
    R"(Program terminated with signal (\w+),\s*([^\.\n]+))");
```

Ainsi, Pour une sortie telle que `Program terminated with signal SIGSEGV, Segmentation fault.`, le premier groupe renseigne `info.signal` avec `SIGSEGV`, tandis que le second renseigne `info.signal_desc` avec `Segmentation fault`.

2. Extraction de l'adresse fautive. Une deuxième expression recherche séparément le champ `si_addr` dans le résultat de `print $_siginfo` :

```
std::regex addrRe(R"(si_addr\s*=\s*(0x[0-9a-fA-F]+))");
```

Ainsi, une valeur telle que `si_addr = 0x0` est enregistrée dans `info.fault_address`. Cette recherche est systématique et indépendante de la détection précédente : elle reste donc exécutée même si la ligne standard du signal n'a pas été reconnue.

3. Repli en cas d'échec de la détection principale. Si `info.signal` est encore vide, une expression plus permissive recherche une occurrence de la forme `SIGxxx` dans l'ensemble de la sortie :

```
if (info.signal.empty()) {  
    std::regex altRe(R"((SIG[A-Z]+))");  
    if (std::regex_search(gdbOutput, m, altRe)) {  
        info.signal = m[1].str();  
    }  
}
```

Cette règle de repli reste capable de reconnaître des signaux tels que `SIGSEGV` ou `SIGABRT` lorsque la phrase complète ne respecte pas le format prévu. Elle garantit toutefois uniquement l'identification du signal : la description peut demeurer vide, car le texte associé n'est pas capturé par ce mécanisme.

Ainsi, cette stratégie sépare ainsi trois tâches : identifier précisément le signal, récupérer indépendamment l'adresse fautive et assurer une tolérance aux variations de la sortie GDB.

d) **PatternMatcher : classification automatique des causes**

Ainsi, nous avons choisi une approche fondée sur des règles explicites plutôt qu'un modèle statistique, car elle est déterministe, interprétable et adaptée à une analyse hors ligne.

`PatternMatcher` analyse le signal, le backtrace et l'adresse fautive dans le but d'identifier la cause probable du crash. Le tableau 3.8 expose les principales règles appliquées.

TABLE 3.8 – Règles de classification appliquées par `PatternMatcher`

Signal	Condition observée	crash_type	Confiance
SIGSEGV	Plus de 20 frames et une fonction répétée	STACK_OVERFLOW	HIGH
SIGSEGV	Adresse fautive nulle	NULL_POINTER_DEREF	HIGH
SIGSEGV	Adresse invalide non nulle	INVALID_MEMORY_ACCESS	MEDIUM
SIGABRT	Frame contenant assert	ASSERTION_FAILURE	HIGH
SIGABRT	Frame malloc/free/alloc	HEAP_CORRUPTION	MEDIUM
SIGABRT	Aucun motif spécifique	ABORT	LOW
SIGFPE	Toujours	ARITHMETIC_ERROR	HIGH
SIGILL	Toujours	ILLEGAL_INSTRUCTION	MEDIUM
SIGBUS	Toujours	BUS_ERROR	MEDIUM
SIGTRAP	Toujours	TRAP	MEDIUM

e) **SolutionProposer : explications et correctifs**

Après la classification, `SolutionProposer` relie chaque type de crash à une explication et à une action corrective. Le tableau 3.9 donne quatre exemples représentatifs. Nous avons volontairement conservé des recommandations courtes afin qu'elles servent de point de départ au diagnostic sans prétendre remplacer la revue du code par l'ingénieur.

f) **Exporteurs CSV / JSON / SQLite**

Comme mentionné précédemment, trois représentations complémentaires ont été produites : le CSV pour l'exploitation tabulaire, le JSON pour conserver une structure détaillée et SQLite

TABLE 3.9 – Exemples de règles SolutionProposer — explications et correctifs générés

crash_type	Explication générée	Correctif proposé
NULL_POINTER_DEREF	Pointeur nul ou non initialisé	Tester le pointeur et l'allocation
STACK_OVERFLOW	Récursion non bornée	Ajouter un arrêt ou une version itérative
ARITHMETIC_ERROR	Division par zéro ou dépassement	Valider le diviseur avant calcul
HEAP_CORRUPTION	Double libération ou débordement	Recompiler avec les sanitizers

pour l'historisation. SQLite a été préféré à MySQL ou PostgreSQL, car il ne requiert aucun service et convient aux écritures séquentielles de Crash_Analyzer. Ce choix simplifie le déploiement, au prix d'une gestion moins adaptée aux accès concurrents massifs, qui ne font pas partie du besoin actuel.

Le tableau 3.10 met en regard les caractéristiques de ces trois formats et précise leur rôle dans la solution.

TABLE 3.10 – Comparaison des formats d'export CSV, JSON et SQLite

Critère	CSV	JSON	SQLite
Organisation	Lignes et colonnes	Objets clés-valeurs	Tables relationnelles
Données imbriquées	Limitées	Oui, dont backtrace	Oui, tables/champs
Consultation	Tableur ou Pandas	Éditeur ou analyseur JSON	SQL ou outil SQLite
Recherche/filtrage	Chargement séquentiel	Parcours programmatique	Filtres, tris, agrégats
Usage projet	Export synthétique	Rapport détaillé	Historique/tableau de bord
Limite	Structures complexes	Historique volumineux	Outil/requête requis

g) Sortie CSV

Le CSVExporter écrit une ligne par crash analysé selon un schéma à quinze colonnes dérivé de la structure CrashInfo, décrite dans le diagramme de classes de la figure 2.12 du chapitre 2. Le fichier crash_report.csv regroupe notamment le signal, le type de crash, la cause probable, le niveau de confiance, la fonction fautive, l'explication générée et le correctif proposé. Ce format simplifie l'exploitation des résultats dans un tableur ou dans un script de traitement complémentaire.

h) Sortie JSON

Le JSONExporter génère un objet par crash, enrichi d'un tableau détaillant les frames du backtrace. La figure 3.9 illustre cette structure pour un crash SIGABRT de type ABORT.

La capture met en évidence la structure hiérarchique du rapport : les informations générales du crash sont regroupées avec la cause racine, la confiance, la localisation et le tableau des frames. Elle vérifie également que l'explication et le correctif générés par SolutionProposer sont exportés avec le diagnostic.

3.4.3 Difficultés rencontrées

La difficulté majeure a été la variabilité de la sortie GDB selon le signal, l'architecture et la version du débogueur. L'utilisation de marqueurs explicites, d'une expression régulière principale et d'une règle de repli a rendu l'extraction plus robuste.

```

timestamp: 2026-07-08 11:28:41
core_file: heecile: ./coredumps/input/core.crash_simulator.290286.1783501551
executable: /home/dghriss@actia.local/Burelnu/pfe/2_jueltre/pfe_2026-main/crash_analyzer_project/build/
bin/crash_simulator
signal: SIGABRT
signal_desc: Aborted
crash_type: ASSERTION_FAILURE
root_cause: Assertion failed, aborting program
confidence: HIGH
crash_function: __phrend_kill_implementation
crash_file: ./nptl/phrend_kill.c
crash_line: 44
crash_args: no_tid=0, signo=6, threadid=132733E94239680
fault_address: 0x46dee
stack_trace: #0 __phrend_kill_implementation at ./nptl/phrend_kill.c:44 | #1 __phrend_kill_internal
explanation: An assert() condition evaluated to false, so the program intentionally aborted to avoid continuing
invalidation state.
fix: Inspect the failed condition and the program state at that point; fix the logic that violates the
asserted invariant.
backtrace: [ __phrend_kill_implementation'
__phrend_kill_internal
__GI__phrend_kill
__GI_raise
CrashSimulator::Simulator::triggerAssertionFailure'
CrashSimulator::Simulator::runOnce'
CrashSimulator::Simulator::triggerCrash
[ main

```

FIGURE 3.9 – Structure JSON d’un rapport unitaire — crash SIGABRT avec backtrace détaillé et correctif généré par SolutionProposer

Ainsi, une seconde difficulté concernait plusieurs causes partageant le même signal, notamment le pointeur nul et le débordement de pile sous SIGSEGV. L’adresse fautive, la profondeur du backtrace et la répétition des fonctions ont été combinées pour les distinguer.

Cette démarche reste toutefois fondée sur des règles et peut produire une confiance moyenne lorsque le contexte est incomplet.

3.4.4 Résultats et validation

Le tableau 3.11 expose les résultats fonctionnels du Sprint 2. Les huit scénarios ont produit le signal et la catégorie attendus. Les niveaux MEDIUM observés pour certaines causes traduisent volontairement une preuve moins spécifique dans le backtrace, et non un échec de détection.

TABLE 3.11 – Résultats de la phase de test du Sprint 2 — détection des signaux par PatternMatcher

Signal provoqué	Déecté	crash_type retourné	Confiance	Conforme
SIGSEGV (ptr nul)	SIGSEGV	NULL_POINTER_DEREF	HIGH	Oui
SIGSEGV (récur-sion)	SIGSEGV	STACK_OVERFLOW	HIGH	Oui
SIGABRT (assert)	SIGABRT	ASSERTION_FAILURE	HIGH	Oui
SIGABRT (heap)	SIGABRT	HEAP_CORRUPTION	MEDIUM	Oui
SIGFPE	SIGFPE	ARITHMETIC_ERROR	HIGH	Oui
SIGBUS	SIGBUS	BUS_ERROR	MEDIUM	Oui
SIGILL	SIGILL	ILLEGAL_INSTRUCTION	MEDIUM	Oui
SIGTRAP	SIGTRAP	TRAP	MEDIUM	Oui

La figure 3.10 détaille le traitement d’un lot de fichiers core pendant la campagne de validation.

Le journal met en évidence l’enchaînement des analyses sans arrêt global lorsqu’un fichier est invalide ou qu’une classification possède une confiance moyenne. Cette exécution a permis

crash_analyzer_project > build > analyzer_output > crash_report.csv									
timestamp,	core_file,	signal,	signal_desc,	crash_type	root_cause	confidence,	crash_function	crash_file,	crash_line
#666666	core.crash_	SIGSEGV	Segmenta	STACK_OVE	Stack exhausted	HIGH	CrashSimulator: Simulator.	245	
_simulat	.tor.355369	(#CC0000)	on fault	RFLOW	deep/unbounded	(#00AA00)	Simulator::trig cpp		
				(#0066CC)	recursion in 'Cra		gerStackOverflo		
#666666	core.unique	SIGABRT	Aborted	ABORT	Program expli	LOW	#008800	pthread_k	44
_crash_ap	.cor.327166	(#CC0000)		(#0066CC)	icitly called	(#888888)	__pthread_kill_ ill.c		
					abort()		implementation		
#666666	core.crash_	SIGABRT	Aborted	ASSERTION_	Assertion fai	HIGH	#008800	#008800	44
simul.ator.	(#CC0000)			FAILURE	led, aborting	(#00AA00)	__pthread_kill_ pthread_k		
326945				(#0066CC)	program		implementation ill.c		
#666666	core.crash_	SIGABRT	Aborted	HEAP_CORR	Heap corruption	HIGH	#008800	#008800	44
simul.ator.	(#CC0000)			UPTION	detected by	(#00AA00)	__pthread_kill_ pthread_k		
359847				(#0066CC)	glibc allocator		implementation ill.c		
#666666	core.crash_	SIGBUS	Bus	BUS_ERROR	Misaligned memory	MEDIUM	#008800	#008800	278
simul.ator.	(#CC0000)		error	(#0066CC)	access or access	(#FF8800)	CrashSimulator: Simulator.		
326922					beyond mapped		:Simulator::tricpp		
					file/region		ggerBusError		

FIGURE 3.10 – Campagne d’analyse d’un lot de fichiers core — progression, classification et production des rapports

de vérifier la continuité du traitement, l’écriture progressive des rapports et la cohérence des catégories retournées sur plusieurs familles de signaux.

a) Contribution et analyse critique

Le travail réalisé au Sprint 2 comprend le pilotage automatisé de GDB, la construction de CrashInfo, la production des propositions de correction ainsi que les exporteurs CSV, JSON et SQLite. La classification des crashes s’appuie sur une analyse déterministe de motifs (*pattern matching*) fondée sur des règles et des signatures connues, sans recours à l’intelligence artificielle. Cette démarche assure une reproductibilité totale des diagnostics et une transparence de la logique de catégorisation. L’apport spécifique réside dans l’intégration de ces mécanismes au sein d’une chaîne locale adaptée aux fichiers core ARM64, complétée par un protocole de test comparant systématiquement le signal provoqué, la catégorie attendue et le diagnostic produit.

b) Limite de la collecte contextuelle

L’analyse post-mortem fondée sur les signaux se limite aux données disponibles après l’arrêt du processus : horodatage, signal, fonction fautive et backtrace. Les métriques dynamiques, telles que la charge du processeur, l’utilisation de la mémoire et les identifiants des processus ou des threads accessibles via /proc, sont exclues, car elles requièraient une surveillance active incompatible avec le cadre strictement post-mortem retenu. Ce compromis privilégie la fiabilité des données vérifiables après l’incident plutôt que leur exhaustivité et prépare le Sprint 3, consacré à la restitution des résultats.

3.5 Développement du Dashboard_Creator avec Streamlit

3.5.1 Objectifs et backlog du Sprint 3

L’objectif du Sprint 3 est de rendre les diagnostics consultables par un ingénieur sans exiger la maîtrise détaillée de GDB. Le Dashboard_Creator doit charger les trois formats de rapport, exposer une synthèse, permettre le filtrage des incidents, rapprocher le diagnostic du code source et proposer une assistance déterministe à la correction.

a) Backlog et périmètre fonctionnel

Le tableau 3.12 détaille le périmètre du Sprint 3. Nous avons séparé la vue synthétique, l'exploration détaillée, l'accès au code et l'assistance afin que chaque onglet réponde à une tâche précise de l'ingénieur.

TABLE 3.12 – Backlog du Sprint 3 — Dashboard_Creator

ID	User Story	Priorité	Points
US-12	En tant qu'ingénieur, je veux voir les statistiques globales des crashes sur un tableau de bord	Haute	5
US-13	En tant qu'ingénieur, je veux filtrer et rechercher les crashes par signal, type ou fonction	Haute	3
US-14	En tant qu'ingénieur, je veux visualiser le code source autour de la ligne de crash	Moyenne	5
US-15	En tant qu'ingénieur, je veux consulter le backtrace GDB complet depuis le Dashboard_Creator	Moyenne	3
US-16	En tant qu'ingénieur, je veux un assistant qui me guide vers la correction d'un crash	Basse	8

b) Diagramme de cas d'utilisation du Sprint 3

La figure 3.11 résume les interactions offertes par le Dashboard_Creator.

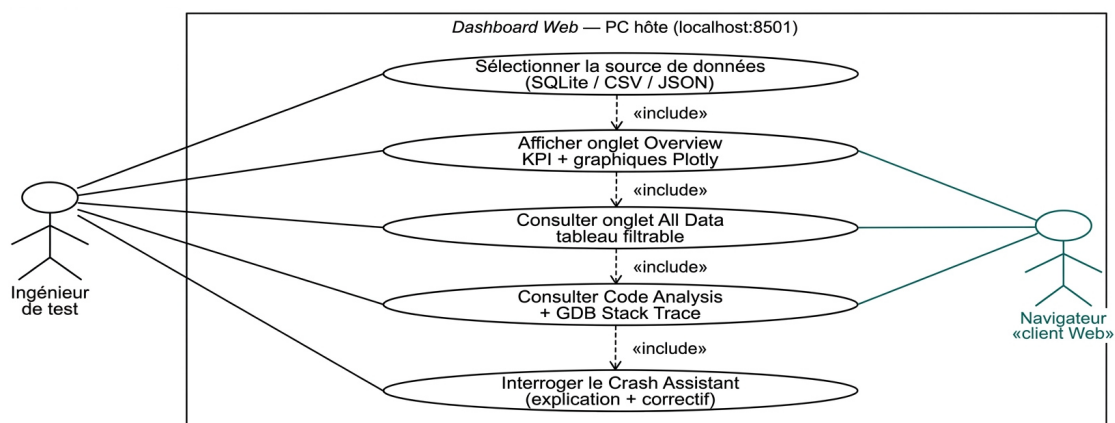


FIGURE 3.11 – Diagramme de cas d'utilisation du Sprint 3 — Dashboard_Creator

Les fonctions mises à la disposition de l'ingénieur de test sont synthétisées dans ce diagramme : chargement des rapports, consultation des statistiques, filtrage des incidents, analyse du code, lecture du backtrace et utilisation de l'assistant. La représentation précise ainsi le périmètre fonctionnel couvert pendant le Sprint 3.

3.5.2 Démarche de réalisation

La réalisation a été organisée autour d'un modèle de données commun. Les sources CSV, JSON et SQLite sont d'abord normalisées dans un DataFrame, puis les cinq onglets exploitent cette représentation unique. Cette démarche limite les divergences entre les vues et permet de vérifier progressivement le chargement, les filtres, la localisation dans le code et l'assistance.

a) Architecture et choix technologiques

Ainsi, l'application Streamlit `app_dash.py` lit CSV, JSON ou SQLite sans appeler le C++. `@st.cache_data` évite de retraiter les rapports à chaque interaction et l'interface évolue sans recompilation embarquée.

Le flux de travail interne suit quatre opérations : chargement et normalisation de la source, conversion en DataFrame, application des filtres, puis construction de la vue sélectionnée. Les onglets emploient donc le même modèle de données, ce qui évite de dupliquer la logique de lecture entre Overview, All Data, Code Analysis, GDB et Crash Assistant.

Ainsi, un développement équivalent avec Flask ou Django aurait nécessité de réaliser séparément les routes, les API, les pages HTML, les styles CSS et le rafraîchissement des données. Streamlit prend directement en charge ces fonctions pour une application orientée données, ce qui a réduit la complexité de réalisation du Dashboard_Creator.

b) Réalisation — les cinq onglets du Dashboard_Creator

Comme l'illustre la figure 3.12, l'interface est organisée en cinq onglets complémentaires : *Overview / Analytics*, *All Data*, *Code Analysis*, *GDB / Stack Trace* et *Crash Assistant*. Les paragraphes suivants exposent leur réalisation et leur rôle.

Onglet Overview / Analytics — synthèse des incidents

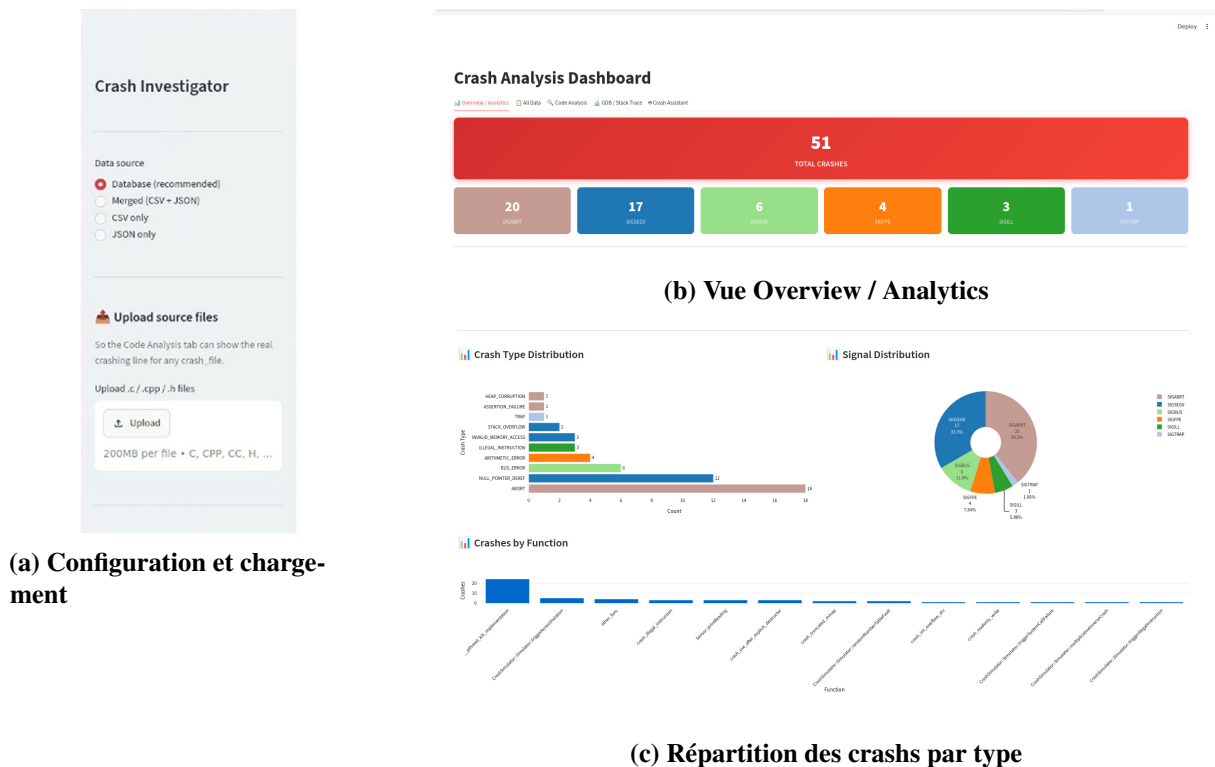


FIGURE 3.12 – Vue synthétique du Dashboard_Creator : configuration, statistiques et répartition des crashes

La figure 3.12 présente trois vues complémentaires du tableau de bord. La sous-figure (a) montre la sélection et le chargement d'une source CSV, JSON ou SQLite. La sous-figure (b) synthétise ensuite les 51 incidents analysés. Chaque incident désigne un fichier core produit après la réception d'un signal fatal, dont la signification est rappelée ci-dessous :

- SIGABRT (20 incidents) : arrêt volontaire et anormal demandé par le programme, souvent après l'échec d'une assertion ou l'appel à abort() ;
- SIGSEGV (17 incidents) : accès à une adresse mémoire interdite, par exemple lors du déréférencement d'un pointeur nul ou invalide ;

- SIGBUS (6 incidents) : accès mémoire impossible en raison d'une adresse mal alignée ou d'une zone mémoire non valide ;
- SIGFPE (4 incidents) : erreur arithmétique, telle qu'une division entière par zéro ;
- SIGILL (3 incidents) : tentative d'exécution d'une instruction inconnue ou non prise en charge par le processeur ;
- SIGTRAP (1 incident) : interruption provoquée par un piège de débogage ou une instruction de type *breakpoint*.

La sous-figure (c) complète cette lecture par les catégories déterminées par PatternMatcher. Contrairement au signal, qui détaille la manière immédiate dont le processus s'est arrêté, la catégorie indique la cause logicielle probable. Ainsi, ABORT désigne un arrêt explicitement déclenché par l'application, tandis que NULL_POINTER_DEREF correspond au déréférencement d'un pointeur nul. Ces deux catégories sont les plus fréquentes, avec respectivement 18 et 12 occurrences.

Onglet All Data — tableau filtrable

L'onglet *All Data* expose les 51 incidents sous forme d'un tableau filtrable. La figure 3.13 montre cette vue ainsi que les commandes d'export disponibles.

timestamp	core_file	signal	signal_desc	crash_type	root_cause	crash_function	crash_file
2026-07-08 16:02:40	./coredumps_input/core.unique_crash_ap.359393...	SIGABRT	Aborted	ABORT	Program explicitly called abort()	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:41	./coredumps_input/core.crash_simulator.359824...	SIGBUS	Bus error	BUS_ERROR	Misaligned memory access or access beyond mapped file/region in '__pthread_ki...	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:42	./coredumps_input/core.no_abort_crash_313122...	SIGSEGV	Segmentation fault	NULL_POINTER_DEREF	Dereference of a NULL pointer in 'crash_use_after_explicit_destructor'	crash_use_after_explicit_destructor	
2026-07-08 16:02:42	./coredumps_input/core.unique_crash_ap.327162...	SIGABRT	Aborted	ABORT	Program explicitly called abort()	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:43	./coredumps_input/core.no_abort_crash_359531...	SIGSEGV	Segmentation fault	NULL_POINTER_DEREF	Dereference of a NULL pointer in 'crash_use_after_explicit_destructor'	crash_use_after_explicit_destructor	
2026-07-08 16:02:43	./coredumps_input/core.unique_crash_ap.354954...	SIGABRT	Aborted	ABORT	Program explicitly called abort()	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:44	./coredumps_input/core.advanced_crash_315547...	SIGTRAP	Trace/breakpoint trap	TRAP	Trap instruction hit (breakpoint, debugger trap, or compiler-inserted guard) in '__pth...	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:44	./coredumps_input/core.unique_crash_ap.354956...	SIGABRT	Aborted	ABORT	Program explicitly called abort()	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:45	./coredumps_input/core.unique_crash_ap.327165...	SIGABRT	Aborted	ABORT	Program explicitly called abort()	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:45	./coredumps_input/core.no_abort_crash_359529...	SIGILL	Illegal instruction	ILLEGAL_INSTRUCTION	CPU attempted to execute an invalid/corrupt instruction (often a corrupted function)	crash_illegal_instruction	
2026-07-08 16:02:45	./coredumps_input/core.crash_simulator.326945...	SIGABRT	Aborted	ASSERTION_FAILURE	Assertion failed, aborting program	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:46	./coredumps_input/core.crash_simulator.355384...	SIGABRT	Aborted	ABORT	Program explicitly called abort()	__pthread_kill_implementation	./nptl/pthread_kill.
2026-07-08 16:02:46	./coredumps_input/core.no_abort_crash_359511...	SIGSEGV	Segmentation fault	INVALID_MEMORY_ACCE	Access to invalid/unmapped memory address 0x7ecb911f2000 in 'crash_readonly_w...	crash_readonly_write	

FIGURE 3.13 – Tableau filtrable des incidents dans le Dashboard_Creator

La figure 3.13 expose les principales colonnes du diagnostic, notamment le signal, le type de crash, la cause probable, la fonction fautive et le fichier source. Les commandes situées sous le tableau permettent d'exporter les données aux formats CSV et JSON.

Analyse détaillée — code source et pile d'appels

Les onglets *Code Analysis* et *GDB / Stack Trace* permettent de passer du diagnostic structuré aux éléments techniques qui l'expliquent. La difficulté principale a été d'associer les chemins enregistrés dans le rapport aux fichiers disponibles sur le poste hôte. Une correspondance fondée sur le nom du fichier et le numéro de ligne fourni par GDB a été utilisée. La figure 3.14 regroupe les trois vues mobilisées dans cette analyse.

Code source et ligne fautive (figure 3.14(a)) : l'onglet *Code Analysis* affiche le fichier dans son contexte et met en évidence l'instruction associée au crash. Les informations issues du

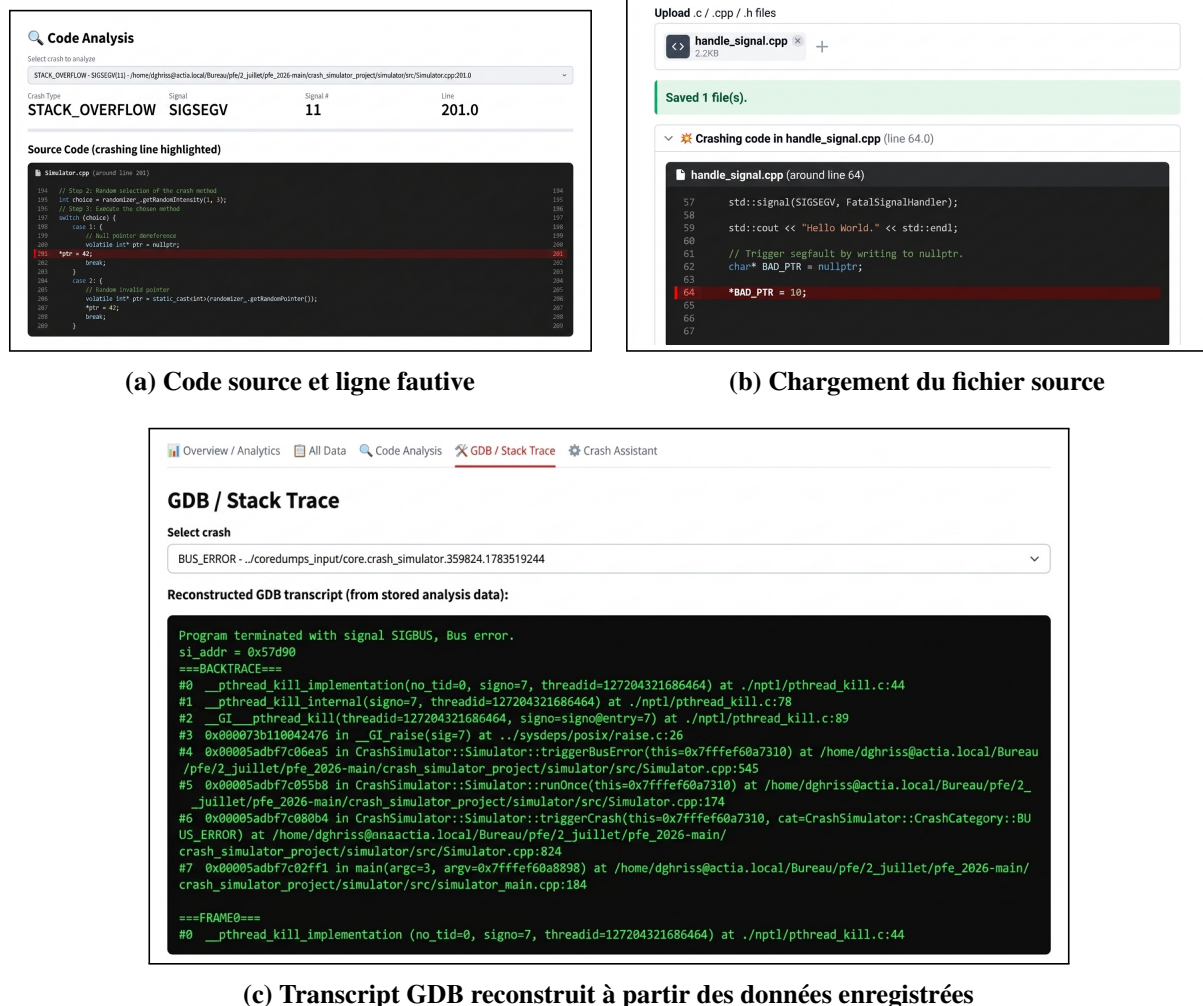


FIGURE 3.14 – Analyse détaillée d'un crash : code source, localisation et stack trace

diagnostic, notamment le signal, la catégorie et la ligne détectée, permettent à l'ingénieur de passer directement du rapport à la zone de code concernée.

Chargement du fichier source (figure 3.14(b)) : le fichier `handle_signal.cpp` est importé manuellement lorsque le chemin enregistré sur la cible ne correspond pas à celui du poste hôte. L'extrait affiché autour de la ligne 64 et la mise en évidence de l'instruction fautive confirment la correspondance entre la localisation fournie par GDB et le code chargé.

Transcript GDB reconstruit et pile d'appels (figure 3.14(c)) : il ne s'agit pas d'une exécution manuelle de GDB depuis le tableau de bord. Pour l'incident de type `BUS_ERROR`, la vue *GDB / Stack Trace* présente une reconstruction du transcript GDB à partir des données d'analyse générées automatiquement par `Crash_Analyzer` et enregistrées dans le rapport. Cette démarche permet de restituer les informations pertinentes de l'analyse, notamment la pile d'appels, le signal ayant provoqué l'arrêt de l'application et les différentes frames associées au crash.

Ainsi, ces trois vues sont complémentaires : la première localise l'instruction, la deuxième établit le lien avec le fichier source disponible et la troisième restitue le contexte d'exécution ayant conduit à la défaillance.

Onglet Crash Assistant

L'onglet *Crash Assistant* guide l'ingénieur au moyen de quatre questions prédéfinies. Cette démarche déterministe assure des réponses reproductibles et locales. La figure 3.15 expose les deux réponses principales, tandis que les vues complémentaires sont conservées en annexe.

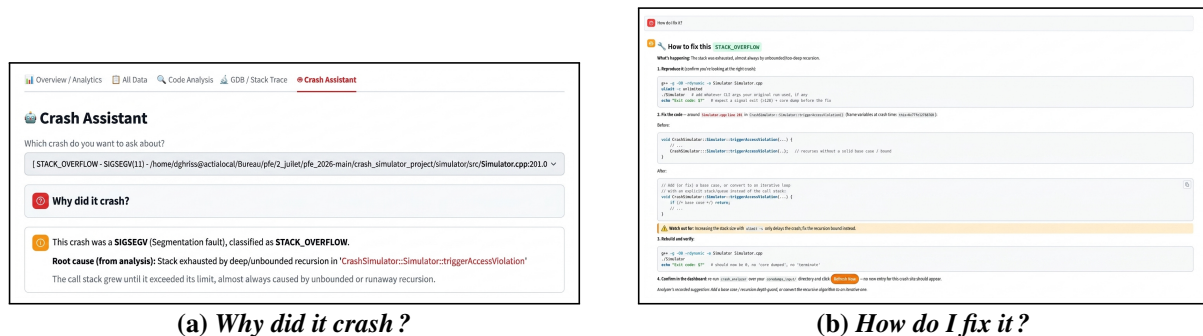


FIGURE 3.15 – Fonctions principales du *Crash Assistant* : explication et proposition de correction

La figure 3.15 expose les deux réponses essentielles : l'explication de la cause et la proposition de correction. Les vues de localisation et de pile d'appels, fondées sur les mêmes données, sont reportées aux figures A.3a et A.3b des annexes.

3.5.3 Difficultés rencontrées

Deux principales difficultés ont été rencontrées :

- la normalisation commune des sources CSV, JSON et SQLite, dans le but d'assurer la cohérence des formats et des champs lors de l'importation des données ;
- la correspondance des chemins entre le Raspberry Pi et le poste hôte, fondée sur le nom du fichier et le numéro de ligne, pour garantir la traçabilité des crashes dans un environnement distribué.

Ces contraintes ont nécessité la mise en place d'un mécanisme d'adaptation automatique des chemins et d'une vérification de cohérence entre les métadonnées des fichiers core et les rapports générés. Les solutions adoptées ont également permis d'améliorer la robustesse du pipeline d'analyse et la fiabilité des diagnostics générés.

3.5.4 Résultats et validation

Ainsi, lors du test final, la base SQLite contenant 51 crashes a été chargée et parcourue à travers les cinq onglets du tableau de bord. Le tableau 3.13 synthétise les contrôles réalisés sur ce jeu de données, notamment :

- la vérification de la continuité du traitement et de la classification des signaux ;
- la validation des correspondances entre les fichiers core et les diagnostics produits ;
- l'évaluation de la fiabilité des exporteurs CSV, JSON et SQLite.

Ces résultats confirment la stabilité du pipeline et la reproductibilité des analyses sur l'architecture ARM64. Ils démontrent également la pertinence du protocole de test et la cohérence des catégories retournées.

a) Contribution et analyse critique

Notre contribution comprend la conception de la navigation, le développement de l'application et de ses cinq onglets, l'intégration des graphiques Plotly, des filtres et des fonctions

TABLE 3.13 – Résultats de validation fonctionnelle du Dashboard_Creator

Fonction vérifiée	Donnée ou action de test	Résultat observé	Conforme
Chargement et continuité du traitement	Base SQLite contenant 51 incidents	Les 51 lignes sont chargées sans perte ni interruption	Oui
Classification des signaux	Agrégation par signal et catégorie	Indicateurs et catégories cohérents avec la base	Oui
Fiabilité des exporteurs	Comparaison des sources CSV, JSON et SQLite	Les champs essentiels et le nombre d'incidents concordent	Oui
Correspondance core-diagnostic	Fichier source et numéro de ligne issus du rapport	La ligne fautive est localisée et mise en évidence	Oui
Parcours des cinq onglets	Sélection de plusieurs familles de crashes	Données, backtrace, explication et correction sont affichés	Oui

d'export, ainsi que la mise en évidence de la ligne fautive et l'implémentation des quatre réponses du *Crash Assistant*. Nous avons également construit le jeu de validation de 51 incidents et vérifié la cohérence des informations affichées à partir des trois formats de rapport.

Crash_Analyzer et le Dashboard_Creator étant opérationnels, la dernière étape revient à valider la chaîne dans son environnement cible. Le Sprint 4 automatise donc la compilation croisée, le transfert des binaires vers le Raspberry Pi et le rapatriement des rapports vers le poste hôte.

3.6 Pipeline de déploiement sur la cible embarquée

3.6.1 Objectifs et backlog du Sprint 4

L'objectif du Sprint 4 est de démet en évidence que la chaîne complète peut être construite, déployée et exécutée de manière reproductible sur une cible ARM64. Le processus doit couvrir la compilation croisée, le contrôle des binaires, leur transfert, la préparation des fichiers core, l'exécution locale de l'analyse et le rapatriement des rapports vers le poste hôte.

Le tableau 3.14 rassemble les opérations nécessaires pour rendre le déploiement reproductible. Nous avons choisi d'automatiser ces étapes, car les premiers transferts manuels entraînaient des oublis de fichiers de configuration et des incohérences de répertoires.

TABLE 3.14 – Backlog du Sprint 4 — déploiement embarqué

ID	User Story	Priorité	Points
US-17	En tant qu'ingénieur, je veux compiler le Crash_Simulator et Crash_Analyzer pour ARM64	Haute	3
US-18	En tant qu'ingénieur, je veux transférer les binaires ARM64 vers le Raspberry Pi via SCP	Haute	2
US-19	En tant qu'ingénieur, je veux exécuter la chaîne complète sur le Raspberry Pi via SSH	Haute	3
US-20	En tant qu'ingénieur, je veux rapatrier les rapports CSV/JSON/SQLite vers le PC hôte	Haute	2
US-21	En tant qu'ingénieur, je veux automatiser tout le déploiement via un script <code>deploy_all.sh</code>	Moyenne	5

La démarche comporte huit opérations successives regroupées en cinq phases : construction des binaires ARM64, déploiement sur la cible, exécution et analyse, rapatriement des rapports, puis visualisation. Elle comprend la compilation croisée, la vérification de l'architecture, le transfert des exécutables, la configuration de la production des fichiers core, l'exécution de

Crash_Simulator, l'analyse par Crash_Analyzer, la récupération des rapports et leur consultation dans Dashboard_Creator. Chaque phase produit un résultat vérifiable avant le passage à la suivante, ce qui permet de distinguer plus facilement une erreur de compilation, de transfert, d'exécution ou d'analyse.

La figure 3.16 synthétise les échanges entre le PC hôte x86_64 et le Raspberry Pi 4 ARM64 : compilation et contrôle des binaires, transfert, production et analyse des cores, puis rapatriement des rapports vers le Dashboard_Creator.

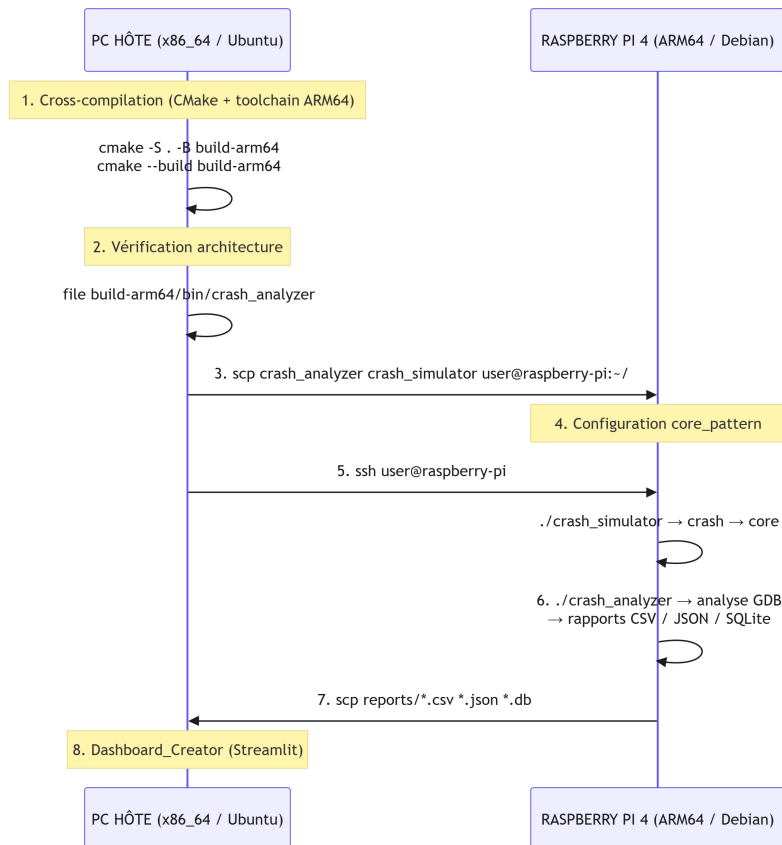


FIGURE 3.16 – Séquence de déploiement et d'exécution sur la cible Raspberry Pi 4

3.6.2 Étapes de la cross-compilation

La compilation croisée s'appuie sur `toolchain-rpi4.cmake`, qui sélectionne les compilateurs `aarch64-linux-gnu-gcc/g++` et les bibliothèques ARM64 du `sysroot`. CMake réemploie ainsi les mêmes sources et la même logique de construction que pour la version native, sans mélanger les dépendances x86_64. Après chaque construction, la commande `file` vérifie que le binaire obtenu cible bien l'architecture `aarch64` avant son transfert.

La figure 3.17 détaille la détection de la toolchain et de SQLite, puis la compilation de `Crash_Analyzer`.

```
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/crash_analyzer_project$
cmake .. -DCMAKE_TOOLCHAIN_FILE=../toolchain-rpi4.cmake -DCMAKE_BUILD_TYPE=Release
cmake --build . --parallel "$(nproc)"
cd ../..
-- The CXX compiler identification is GNU 11.4.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/aarch64-linux-gnu-g++ - skipped
-- Detecting CXX compile features - done
-- Found PkgConfig: /usr/bin/pkg-config (found version "0.29.2")
-- Checking for module 'sqlite3'
-- Found sqlite3, version 3.37.2
-- Configuring done
-- Generating done
-- Build files have been written to: /home/dghriss@actia.local/Bureau/pfe/2_juillet/pfe_2026
[ 25%] Building CXX object CMakeFiles/crash_analyzer.dir/src/PatternMatcher.cpp.o
[ 25%] Building CXX object CMakeFiles/crash_analyzer.dir/src/main.cpp.o
[ 37%] Building CXX object CMakeFiles/crash_analyzer.dir/src/SolutionProposer.cpp.o
[ 50%] Building CXX object CMakeFiles/crash_analyzer.dir/src/JSONExporter.cpp.o
[ 62%] Building CXX object CMakeFiles/crash_analyzer.dir/src/CoreFileParser.cpp.o
[ 75%] Building CXX object CMakeFiles/crash_analyzer.dir/src/DBExporter.cpp.o
[ 87%] Building CXX object CMakeFiles/crash_analyzer.dir/src/CSVExporter.cpp.o
[100%] Linking CXX executable crash_analyzer
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld : /usr/lib/x86_64
[100%] Built target crash_analyzer
```

FIGURE 3.17 – Cross-compilation ARM64 du module Crash_Analyzer

La progression jusqu'à 100 % et le message *Built target crash_analyzer* confirment la production de l'exécutable ARM64. Les bibliothèques hôtes incompatibles signalées pendant la configuration sont écartées au profit de celles du sysroot.

Cette même toolchain est ensuite appliquée à Crash_Simulator. La capture de contrôle correspondante est reportée à la figure A.1 des annexes. Après vérification avec file, les deux exécutables ARM64 sont prêts à être transférés vers le Raspberry Pi.

3.6.3 Transfert et exécution sur la cible

Nous avons regroupé la création des répertoires, la vérification des binaires et leur transfert SCP dans `deploy_all.sh`. La figure 3.18 met en évidence l'exécution de ce flux de travail automatisé.

```
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main$ ./deploy_all.sh
>>> Checking ARM64 binaries before transfer...
OK: crash_analyzer_project/build-arm64/crash_analyzer (ARM aarch64)
OK: crash_simulator_project/build-arm64/bin/crash_simulator (ARM aarch64)
OK: crash_tutorial-master/build-arm64/example1 (ARM aarch64)
OK: crash_tutorial-master/build-arm64/example2 (ARM aarch64)
OK: crash_tutorial-master/build-arm64/example3 (ARM aarch64)
OK: crash_tutorial-master/build-arm64/example4 (ARM aarch64)
OK: buffer-overflow-attack/stack_arm64 (ARM aarch64)
OK: buffer-overflow-attack/stack_nopro_arm64 (ARM aarch64)
>>> Creating directory structure on the Raspberry Pi...
>>> Sending crash_analyzer binary...
>>> Sending crash_simulator binary...
>>> Sending scenario configuration file scenario.cfg...
>>> Sending dashboard (full_dashboard/ folder)...
>>> Sending test_apps (sources + ARM64 binaries, from
crash_analyzer_project/test_apps/ which contains ALL .cpp files:
app2, unique_crash_app, my_app, advanced_crash_app)...
>>> Sending crash_tutorial-master (sources + ARM64 binaries example1-4)...
-> deployed AS A SIBLING of crash_analyzer_project (not inside it):
~/crash_project/crash_tutorial-master/
This is required so crash_analyzer can automatically locate
these binaries when analyzing a core dump from example1/2/3/4.
>>> Sending buffer-overflow-attack (third-party validation project)...
-> deployed AS A SIBLING of crash_analyzer_project, same reason as
crash_tutorial-master: crash_analyzer must find stack.c to
resolve the full source path when analyzing its core dumps.
>>> Updating execution permissions...
Done. Next step: see README_RPI.MD, Part E (coredumps config) then F (execution).
```

FIGURE 3.18 – Exécution du script `deploy_all.sh` — vérification des binaires ARM64, création des répertoires et transfert vers le Raspberry Pi 4

La capture met en évidence les contrôles exécutés avant le transfert, la création de l'arborescence distante et la copie des fichiers par SCP. Ces vérifications empêchent qu'un binaire natif x86_64 ou qu'un fichier de configuration manquant soit envoyé sur la cible.

a) Exécution sur le Raspberry Pi

Avant le lancement de la campagne, la limite définie par `ulimit` est levée dans le but d'autoriser la production des fichiers core. Le paramètre `core_pattern` définit ensuite leur nom et leur emplacement. Enfin, le mode d'accès 1777 appliqué au répertoire de stockage autorise chaque processus à y créer ses fichiers, tandis que le bit de protection empêche un utilisateur de supprimer ceux des autres. La figure 3.19 détaille la vérification de cette configuration directement sur le Raspberry Pi.

```
pi@raspberrypi:~$ ulimit -c unlimited
mkdir -p ~/crash_project/crash_analyzer_project/coredumps_input
sudo sh -c "echo '$HOME/crash_project/crash_analyzer_project/coredumps_input/core.%e.%p.%t' > /proc/sys/kernel/core_pattern"
sudo chmod 1777 ~/crash_project/crash_analyzer_project/coredumps_input
[sudo] password for pi:
```

FIGURE 3.19 – Configuration des fichiers core sur Raspberry Pi — définition de `core_pattern` et préparation du répertoire de stockage

Après cette préparation, les binaires ont été exécutés par SSH. La validation de la robustesse et du caractère générique de `Crash_Analyzer` s'appuie sur trois scénarios complémentaires.

Scénario 1 : Application de test dédiée

L'application `unique_crash_app` est exécutée sur le Raspberry Pi avec l'option `-all`. Elle lance successivement quinze scénarios de signaux dans des processus fils distincts. Cette isolation permet de poursuivre la campagne lorsqu'un processus fils se termine à la suite d'un signal fatal.


```

pi@raspberrypi:~/crash_project/test_apps $ ./unique_crash_app --all
Running ALL 15 unique crashes in subprocesses...

Testing --sighup
Crash: SIGHUP (Signal 1)
Hangup signal - terminal disconnected
Sending SIGHUP to self...
Child killed by signal 1

Testing --sigint
Crash: SIGINT (Signal 2)
Interrupt signal - Ctrl+C
Sending SIGINT to self...
Child killed by signal 2

Testing --sigstkflt
Crash: SIGSTKFLT (Signal 16)
Stack fault on coprocessor (x86)
SIGSTKFLT not available on this architecture
Child exited with code 1

Testing --sigrtm
Crash: SIGRTMIN+0 (Signal 34)
Real-time signal 0
Sending SIGRTMIN to self...
Child killed by signal 34

Testing --terminate

```

FIGURE 3.20 – Exécution de l’application de test unique_crash_app sur Raspberry Pi — validation de signaux complémentaires au Crash_Simulator

Comme l’illustre la figure 3.20, les signaux SIGHUP, SIGINT et SIGRTMIN n’interrompent pas l’exécution du lot. Le signal SIGSTKFLT, indisponible sur l’architecture ARM64, est détecté puis ignoré proprement. Ce scénario vérifie ainsi que Crash_Analyzer traite des signaux variés générés par une application indépendante de Crash_Simulator.

Scénario 2 : Génération contrôlée

Dans ce scénario, Crash_Simulator provoque des défaillances contrôlées et produit les fichiers core directement sur le Raspberry Pi. L’exécution s’appuie sur le fichier scenario.cfg, qui définit notamment le nombre de crashes, leur intensité et le répertoire de stockage des fichiers core.

```

sudo ./build/bin/crash_simulator --config scenario.cfg

Random Crash Simulator v3.0 (Linux)
Embedded Fault Generator - Automated Pipeline

Configuration:
crash_count : 10
intensity   : 7
coredump_dir : ../crash_analyzer_project/coredumps_input

Configuring OS kernel coredumps...
✓ ulimit -c unlimited set.
✓ core_pattern set: ../crash_analyzer_project/coredumps_input/core.%e.%p.%t

Crash 1 / 10
Category : Stack Overflow
Seed     : 921351850
Severity : CRITICAL
Info     : Stack memory exhausted due to deep recursion
Hint     : Add recursion depth limit. Increase stack size or use iterative approach.
CoreDump generated (signal 11)

```

FIGURE 3.21 – Génération contrôlée d’un fichier core par Crash_Simulator sur Raspberry Pi

La figure 3.21 montre la configuration automatique de ulimit et de core_pattern, puis l’exécution du premier cas de la campagne. Le scénario présenté provoque un dépassement de pile par récursion profonde. Le simulateur indique la cause attendue et confirme la production du fichier core à la suite du signal 11.

Les fichiers produits sont ensuite analysés localement sur la cible par Crash_Analyzer. Cette organisation évite le transfert préalable de vidages mémoire potentiellement volumineux. Elle permet surtout de comparer la cause volontairement provoquée, connue avant l'essai, avec le diagnostic produit par l'analyseur. La concordance entre ces deux informations valide la précision du diagnostic de Crash_Analyzer.

Scénario 3 : Applications tierces issues de GitHub

Pour valider la robustesse et le caractère générique de Crash_Analyzer, des applications externes volontairement vulnérables issues de dépôts GitHub ont été sélectionnées.

Contrairement au module Crash_Simulator, utilisé pour produire des scénarios contrôlés, ces programmes représentent des exécutables indépendants du projet. Ils ne partagent ni le fichier de configuration, ni les catégories de l'analyseur, ni le code de production des défaillances. Ce scénario illustre ainsi la capacité de Crash_Analyzer à traiter un crash à partir d'une interface générique, composée d'un fichier core valide et de l'exécutable correspondant.

```
pi@raspberrypi:~/crash_project $ cd ~/crash_project/buffer-
overflow-attac.k
ulimit -c unlimited

./stack_arm64          # -> SIGABRT (stack smashing detected)
./stack_nopro_arm64
Segmentation fault (core dumped)
Segmentation fault (core dumped)
```

FIGURE 3.22 – Exécution d'applications de test issues de GitHub — production de fichiers core à la suite de dépassements de tampon sur ARM64

La figure 3.22 expose les deux variantes ARM64 documentées dans cette campagne. Les programmes `stack_arm64` et `stack_nopro_arm64` provoquent des dépassements de tampon, se terminent par une faute de segmentation et produisent chacun un fichier core. Ces fichiers sont ensuite traités par la même chaîne d'analyse GDB et soumis aux règles de classification appliquées aux crashes simulés, sans adaptation spécifique.

Ces essais confirment que Crash_Analyzer peut analyser toute application Linux ARM64 dès lors qu'un fichier core valide et l'exécutable correspondant sont disponibles. Ils démontrent que l'outil apporte une analyse reproductible pour l'identification et la classification des anomalies mémoire, indépendamment de Crash_Simulator. Cette validation sur des exécutables externes renforce la fiabilité et la généricité de la chaîne d'analyse dans un environnement embarqué ARM64, attestant de la maturité du prototype pour une utilisation sur des cas réels.

3.6.4 Récupération et visualisation des résultats

Une fois l'analyse terminée sur le Raspberry Pi, les rapports CSV et JSON ainsi que les fichiers core sont rapatriés sur le poste hôte. Cette récupération sépare le rôle de la cible embarquée, consacrée à l'exécution et à l'analyse des crashes, de celui du PC, utilisé pour l'archivage et la visualisation. Elle évite également d'installer et d'lancer l'interface Streamlit sur le Raspberry Pi, ce qui limite l'utilisation de ses ressources.

Ces rapports structurés alimentent directement le Dashboard_Creator, tandis que les fichiers core conservés assurent la traçabilité et permettent une analyse complémentaire. Les

commandes SCP de récupération des rapports CSV et JSON ainsi que des cores sont présentées à la figure A.2a des annexes.

a) Affichage du Dashboard_Creator

Après le rapatriement, le Dashboard_Creator est lancé sur le poste hôte. Le serveur Streamlit emploie ici le port configurable 8503 et reste accessible par localhost ; la capture de démarrage est reportée à la figure A.2b des annexes.

b) Difficultés rencontrées au Sprint 4

La difficulté majeure a été de maintenir la cohérence entre deux architectures et deux ensembles de bibliothèques. La séparation des répertoires de construction, la chaîne de compilation CMake et le contrôle systématique avec file ont empêché le transfert accidentel d'un binaire x86_64.

Ces premiers déploiements manuels ont également révélé des oublis de fichiers de configuration et des divergences de chemins. Leur automatisation par SSH/SCP a rendu le processus reproductible.

La validation reste toutefois effectuée sur Raspberry Pi 4, qui représente une cible ARM64 représentative mais ne remplace pas la carte cliente finale.

c) Résultats et validation du Sprint 4

Le tableau 3.15 réunit les résultats de validation de la chaîne déployée. Il vérifie notamment la portabilité, l'architecture modulaire, le traitement par lot, le rejet des fichiers non conformes et l'accès local au tableau de bord.

TABLE 3.15 – Vérification des exigences non fonctionnelles — résultats mesurés

Code	Exigence	Résultat mesuré	Conforme
BNF-001	Compatible x86_64 et ARM64	Exécutée sur les deux architectures	Oui
BNF-002	Architecture modulaire	Modules indépendants	Oui
BNF-003	Lot de plusieurs cores	51 cores traités au cours d'une même campagne	Oui
BNF-004	Rejet des non-ELF	isCoreFile() les ignore	Oui
BNF-005	Dashboard_Creator local	Accès via localhost, port configurable	Oui

Le paramètre 0.0.0.0 correspond à la configuration par défaut de Streamlit utilisée en environnement de développement et de démonstration ; le poste hôte et la cible communiquent sur un réseau isolé sans accès Internet. Une configuration de production restreindrait l'écoute à l'interface locale (127.0.0.1) pour garantir strictement l'absence d'exposition réseau.

d) Contribution et analyse critique

Notre contribution dans ce sprint englobe la création de la chaîne de compilation ARM64, l'adaptation des fichiers CMake, le développement de deploy_all.sh, la configuration des fichiers core sur la cible et la définition du protocole de rapatriement des résultats. Nous avons ainsi transformé une suite de commandes manuelles en une chaîne reproductible allant de la construction jusqu'à l'affichage dans le tableau de bord.

3.7 Analyse critique des limites constatées

Ces essais confirment la faisabilité de la solution, mais ils font également apparaître plusieurs limites à prendre en compte avant une industrialisation. Le tableau 3.16 synthétise leur impact et les évolutions envisageables.

TABLE 3.16 – Analyse critique des principales limites observées pendant la validation

Limite constatée	Impact sur la solution	Évolution proposée
Classification fondée sur des règles	Une cause nouvelle ou un backtrace incomplet peut conduire à une catégorie générique ou à une confiance moyenne	Enrichir les règles avec des campagnes supplémentaires et étudier une analyse sémantique assistée
Contexte strictement post-mortem	La charge CPU, la mémoire disponible et certains états du processus ne sont plus accessibles après sa terminaison	Ajouter un collecteur optionnel et léger fondé sur <code>/proc</code> , activé uniquement pendant les campagnes de validation
Dépendance aux symboles de débogage	Sans l'exécutable exact et ses symboles, la localisation se limite à des adresses ou à des fonctions système	Associer automatiquement chaque version déployée à ses binaires et symboles dans un dépôt d'artefacts
Absence de tests unitaires automatisés	La validation repose principalement sur des campagnes fonctionnelles et des tests d'intégration	Ajouter des tests unitaires ciblant notamment <code>PatternMatcher</code> et <code>CoreFileParser</code> pour renforcer la non-régression lors des futures évolutions
Validation sur Raspberry Pi 4	La portabilité ARM64 est démontrée, mais le comportement et les dépendances peuvent différer sur la carte cliente finale	Rejouer le protocole complet sur la cible industrielle et intégrer ces tests dans la chaîne CI/CD

Ces limites ne remettent pas en cause les résultats obtenus, mais elles délimitent leur portée. La solution est validée comme outil local d'aide au diagnostic sur les scénarios et architectures testés ; une généralisation industrielle exige davantage de données, une gestion systématique des symboles et une validation sur le matériel cible précisif.

Conclusion

Ce chapitre a permis de démontrer la faisabilité d'une chaîne locale et automatisée d'analyse post-mortem sur une cible Linux ARM64. L'intégration des trois modules assure une continuité cohérente entre la production d'une défaillance, l'exploitation du fichier core, la production du diagnostic et sa restitution au sein du tableau de bord. Les différents essais réalisés ont ainsi confirmé la cohérence de l'architecture modulaire, ainsi que la reproductibilité du processus d'analyse et de validation.

La mise en œuvre de cette solution a également permis d'identifier les principales conditions nécessaires à une éventuelle industrialisation, notamment la gestion systématique des symboles de débogage, le renforcement de la couverture des tests automatisés et la validation sur la carte cible définitive. Ces éléments permettent de délimiter clairement la portée actuelle du prototype, tout en constituant une base solide pour les améliorations et les perspectives qui seront présentées dans la conclusion générale.



CONCLUSION GÉNÉRALE

Ce projet a abouti à une chaîne fonctionnelle de diagnostic des fichiers core sur Linux embarqué. L'objectif de production reproductible des défaillances a été atteint grâce à `Crash_Simulator`, qui couvre dix-huit catégories de crashes et isole chaque scénario dans un processus fils. L'analyse des incidents a été automatisée par `Crash_Analyzer`, qui pilote GDB, extrait les informations pertinentes, identifie une cause probable et produit un diagnostic structuré. Enfin, `Dashboard_Creator` assure la consultation des résultats, leur filtrage et leur comparaison dans une interface organisée en cinq onglets.

La portabilité attendue a également été vérifiée. Les composants C++ ont été construits pour x86_64, compilés de manière croisée pour ARM64, puis exécutés sur un Raspberry Pi 4. La campagne finale rassemble 51 incidents associés aux signaux SIGABRT, SIGSEGV, SIGBUS, SIGFPE, SIGILL et SIGTRAP. Les essais menés avec une application de test dédiée et avec des exécutables vulnérables issus de GitHub montrent en outre que l'analyseur ne dépend pas exclusivement des fichiers produits par le simulateur.

Les objectifs d'export et de traçabilité ont été concrétisés par la production de rapports CSV, JSON et SQLite. Ces formats rendent les résultats à la fois consultables dans le tableau de bord, archivables avec les artefacts du logiciel et réutilisables par d'autres outils.

Au-delà de l'assemblage des technologies retenues, le travail réalisé a ajouté plusieurs éléments concrets au processus de validation :

- un générateur configurable de crashes capable de poursuivre une campagne malgré la terminaison fatale de ses processus fils ;
- un moteur d'analyse C++ associant l'extraction GDB à une classification par règles, à un niveau de confiance et à des recommandations de correction ;
- une couche d'export commune vers trois formats complémentaires, adaptée à l'historisation locale des incidents ;
- un tableau de bord Streamlit permettant d'explorer les signaux, les catégories, les détails d'un incident et la reconstruction du transcript GDB ;
- une chaîne de compilation croisée et un script `deploy_all.sh` automatisant la vérification de l'architecture, la préparation des répertoires et le transfert des binaires vers la cible ;
- un protocole de validation ARM64 comprenant la configuration de `core_pattern`, l'exécution des campagnes, l'analyse locale et le rapatriement sécurisé des rapports.

La solution développée réduit les opérations manuelles nécessaires après un crash et fournit des résultats homogènes d'une campagne à l'autre. Elle représente la principale valeur ajoutée du projet pour les activités de test et de diagnostic embarqué.

Les résultats obtenus doivent toutefois être interprétés dans le périmètre des essais réalisés. La classification repose principalement sur des règles et des motifs connus. Ainsi, une pile

d'appels (*backtrace*) incomplète ou l'apparition d'un scénario de défaillance non prévu peut conduire à une catégorie générique ou à un niveau de confiance limité. La qualité du diagnostic dépend également de la disponibilité de l'exécutable correspondant au fichier core ainsi que de ses symboles de débogage. En l'absence de ces éléments, GDB peut principalement retourner des adresses mémoire, ce qui limite la précision de la localisation du défaut.

En complément, le choix d'une analyse strictement post-mortem ne permet pas de récupérer certaines informations dynamiques qui ne sont plus accessibles après l'arrêt du processus, telles que la consommation CPU, l'occupation mémoire ou certains états accessibles via `/proc`. De plus, la campagne de validation réalisée s'est principalement appuyée sur des tests fonctionnels et d'intégration. L'absence actuelle de tests unitaires automatisés pour les composants de classification et d'extraction représente une limite pour la détection précoce des régressions.

Concernant la portabilité, le fonctionnement de la solution sur l'architecture ARM64 a été démontré sur Raspberry Pi 4. Une validation complémentaire reste toutefois nécessaire sur la carte industrielle cible dans le but de vérifier la compatibilité avec son environnement matériel et logiciel, notamment ses bibliothèques, ses pilotes et ses contraintes spécifiques.

Les performances de traitement n'ont également pas fait l'objet d'une caractérisation approfondie sur des campagnes de très grande taille. Enfin, la version actuelle de la solution est majoritairement destinée à l'analyse des fichiers core au format ELF sous Linux et ne prend pas en charge, à ce stade, les formats de fichiers de vidage générés par Windows.

Perspectives d'évolution

Sur la base des limites identifiées, plusieurs perspectives d'évolution peuvent être envisagées dans le but d'améliorer la fiabilité, la couverture fonctionnelle, la portabilité et les performances de la solution. Ces évolutions peuvent être organisées selon plusieurs axes prioritaires :

- renforcement de la qualité : ajouter des tests unitaires pour `PatternMatcher`, `CoreFileParser` et les exporteurs, puis intégrer des tests de non-régression fondés sur un corpus versionné de fichiers core ;
- extension de la couverture : prendre en charge de nouveaux signaux, des crashes multithreads, des corruptions du tas et des backtraces partiellement symbolisés, tout en enrichissant progressivement les règles de classification ;
- gestion des symboles : associer automatiquement chaque version déployée à son exécutable et à ses symboles dans un dépôt d'artefacts dans le but de garantir la reproductibilité d'une analyse ancienne ;
- amélioration des performances : mesurer séparément le temps d'ouverture du core, l'exécution de GDB, la classification et l'export, puis paralléliser de manière contrôlée le traitement par lot et éviter les extractions redondantes ;
- industrialisation : exécuter la campagne complète sur la cible cliente, intégrer l'analyse à la chaîne CI/CD et archiver automatiquement le diagnostic avec la version du logiciel testée ;
- extension multiplateforme : conserver le traitement ELF pour Linux et ajouter une couche d'abstraction capable d'exploiter les *minidumps* Windows avec un débogueur adapté.

Cette évolution requiera des extracteurs spécifiques, tout en maintenant un modèle de rapport commun ;

- assistance par intelligence artificielle : employer un modèle local pour rapprocher des incidents similaires, résumer des piles d'appels complexes et suggérer des pistes d'investigation. Cette assistance devra compléter les preuves fournies par GDB, rester explicable et soumettre toute recommandation à la validation de l'ingénieur.

Le projet fournit donc un socle utilisable pour analyser et historiser des crashes Linux ARM64. Il atteint les objectifs fixés pour le prototype tout en faisant apparaître clairement les travaux encore requis avant une utilisation industrielle à grande échelle : automatisation des tests, maîtrise des symboles, mesure des performances, validation sur la cible finale et élargissement progressif des systèmes pris en charge.



BIBLIOGRAPHIE

- [1] ACTIA Group. *Histoire, valeurs et activités du Groupe ACTIA* [en ligne]. Disponible sur : <https://www.actia.com/actia-histoire-histoire-et-valeurs/> Consulté le 19 juillet 2026.
- [2] Sentry. *Application Crashes : Native SDK Documentation* [en ligne]. Disponible sur : <https://docs.sentry.io/platforms/native/usage/crashes/> Consulté le 19 juillet 2026.
- [3] Google. *Breakpad : A Crash-Reporting System* [logiciel et documentation en ligne]. Disponible sur : <https://chromium.googlesource.com/breakpad/breakpad> Consulté le 19 juillet 2026.
- [4] Chromium Project. *Crashpad Overview Design* [en ligne]. Disponible sur : https://github.com/chromium/crashpad/blob/main/doc/overview_design.md Consulté le 19 juillet 2026.
- [5] Kerrisk, Michael. *signal(7) – Overview of Signals*. Linux *man-pages* project [en ligne]. Disponible sur : <https://man7.org/linux/man-pages/man7/signal.7.html> Consulté le 19 juillet 2026.
- [6] Kerrisk, Michael. *core(5) – Core Dump File*. Linux *man-pages* project [en ligne]. Disponible sur : <https://man7.org/linux/man-pages/man5/core.5.html> Consulté le 19 juillet 2026.
- [7] Free Software Foundation. *Debugging with GDB : The GNU Source-Level Debugger* [documentation en ligne]. Disponible sur : <https://sourceware.org/gdb/current/onlinedocs/gdb.html/> Consulté le 19 juillet 2026.
- [8] Kitware. *cmake-toolchains(7) – CMake 4.4 Documentation* [documentation en ligne]. Disponible sur : <https://cmake.org/cmake/help/latest/manual/cmake-toolchains.7.html> Consulté le 24 juillet 2026.
- [9] Streamlit. *Development Concepts : Architecture, App Design and Configuration* [documentation en ligne]. Disponible sur : <https://docs.streamlit.io/develop/concepts> Consulté le 24 juillet 2026.
- [10] SQLite Consortium. *SQLite Documentation : Appropriate Uses and Architectural Overview* [documentation en ligne]. Disponible sur : <https://www.sqlite.org/docs.html> Consulté le 24 juillet 2026.
- [11] Raspberry Pi Ltd. *Processors : BCM2711 and Raspberry Pi 4 Model B* [documentation en ligne]. Disponible sur : <https://www.raspberrypi.com/documentation/computers/processors.html> Consulté le 24 juillet 2026.
- [12] Schwaber, Ken et Sutherland, Jeff. *Le Guide Scrum : le guide de référence de Scrum*. Novembre 2020 [en ligne]. Disponible sur :

- <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-French.pdf>
Consulté le 19 juillet 2026.
- [13] Billoo, Mohammed. *Embedded Linux Essentials Handbook*. Birmingham : Packt Publishing, 2026, 450 p. ISBN 978-1-83546-930-9.
- [14] James, Kedrian ; Valakuzhy, Kevin ; Snow, Kevin et Monroe, Fabian. “CrashTalk : Automated Generation of Precise, Human-Readable Descriptions of Software Security Bugs”. Dans : *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024. DOI :<https://doi.org/10.1145/3626232.3653256>. Consulté le 19 juillet 2026.
- [15] Zhang, Yiming ; Hu, Yuxin ; Li, Haonan ; Shi, Wenxuan ; Ning, Zhenyu ; Luo, Xiapu et Zhang, Fengwei. “Alligator in Vest : A Practical Failure-Diagnosis Framework via Arm Hardware Features”. Dans : *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, p. 917–928. DOI :<https://doi.org/10.1145/3597926.3598106>. Consulté le 19 juillet 2026.
- [16] Kallingal Joshy, Ashwin et Le, Wei. “FuzzerAid : Grouping Fuzzed Crashes Based on Fault Signatures”. Dans : *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, p. 1–12. DOI :<https://doi.org/10.1145/3551349.3556959>. Consulté le 19 juillet 2026.
- [17] van Tonder, Rijnard ; Kotheimer, John et Le Goues, Claire. “Semantic Crash Bucketing”. Dans : *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, p. 612–622. DOI :<https://doi.org/10.1145/3238147.3238200>. Consulté le 18 août 2026.
- [18] Jiang, Zhiyuan ; Jiang, Xiyue ; Hazimeh, Ahmad ; Tang, Chaojing ; Zhang, Chao et Payer, Mathias. “Igor : Crash Deduplication Through Root-Cause Clustering”. Dans : *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, p. 3318–3336. DOI :<https://doi.org/10.1145/3460120.3485364>. Consulté le 18 août 2026.
- [19] Zhang, Xing ; Chen, Jiongyi ; Feng, Chao ; Li, Ruilin ; Diao, Wenrui ; Zhang, Kehuan ; Lei, Jing et Tang, Chaojing. “DeFault : Mutual Information-Based Crash Triage for Massive Crashes”. Dans : *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering*, 2022, p. 635–646. DOI :<https://doi.org/10.1145/3510003.3512760>. Consulté le 18 août 2026.
- [20] Remil, Youcef ; Bendimerad, Anes ; Mathonat, Romain ; Raïssi, Chedy et Kaytoue, Mehdi. “DeepLSH : Deep Locality-Sensitive Hash Learning for Fast and Efficient Near-Duplicate Crash Report Detection”. Dans : *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, art. 198, p. 1–12. DOI :<https://doi.org/10.1145/3597503.3639146>. Consulté le 18 août 2026.
- [21] [Freedesktop.org / systemd. systemd-coredump – Acquire, Save and Process Core Dumps and Metadata](https://www.freedesktop.org/software/systemd/man/systemd-coredump.html) [documentation en ligne]. Disponible sur : <https://www.freedesktop.org/software/systemd/man/systemd-coredump.html> Consulté le 10 août 2026.

- [22] LLVM Project. *The LLDB Debugger : Documentation and Tutorial* [documentation en ligne]. Disponible sur : <https://lldb.llvm.org/> Consulté le 10 août 2026.
- [23] RTCA. *DO-178C : Software Considerations in Airborne Systems and Equipment Certification*. Washington, DC : RTCA, 2011. Présentation officielle disponible sur : <https://www.rtca.org/do-178/> Consulté le 10 août 2026.
- [24] Organisation internationale de normalisation (ISO). *ISO 26262-1 :2018 – Road Vehicles – Functional Safety – Part 1 : Vocabulary*. 2^e éd., décembre 2018 [norme internationale]. Disponible sur : <https://www.iso.org/standard/68383.html> Consulté le 10 août 2026.
- [25] Chang, Boyu ; Zhao, Binbin ; Zhang, Qiao ; Liu, Peiyu ; Tian, Yuan ; Beyah, Raheem et Ji, Shouling. “FirmRCA : Towards Post-Fuzzing Analysis on ARM Embedded Firmware with Efficient Event-Based Fault Localization”. Dans : *2025 IEEE Symposium on Security and Privacy*, 2025. DOI :<https://doi.org/10.1109/SP61157.2025.00002>. Consulté le 10 août 2026.
- [26] Tanksalkar, Sai Ritvik ; Muralee, Siddharth ; Danduri, Srihari ; Amusuo, Paschal ; Bianchi, Antonio ; Davis, James C. et Machiry, Aravind Kumar. “LEMIX : Enabling Testing of Embedded Applications as Linux Applications”. Dans : *34th USENIX Security Symposium (USENIX Security 25)*, Seattle, WA : USENIX Association, 2025, p. 6937–6956. ISBN 978-1-939133-52-6. Disponible sur : <https://www.usenix.org/conference/usenixsecurity25/presentation/tanksalkar> Consulté le 10 août 2026.



ANNEXES

A.1 Glossaire technique

TABLE A.1 – Glossaire des principaux termes techniques

Backtrace	Pile des appels actifs au moment du crash, extraite par GDB.
Core dump	Instantané ELF de la mémoire, des registres et des threads du processus.
Compilation croisée	Construction sur l'hôte x86_64 d'un binaire destiné à ARM64.
Symboles de débogage	Informations reliant les adresses aux fonctions et lignes du code source.
Sysroot	En-têtes et bibliothèques de la cible utilisés pendant la compilation croisée.

A.2 Interfaces textuelles du projet

Les principaux artefacts sont le fichier `scenario.cfg`, les scripts `setup_coredump.sh` et `deploy_all.sh`, ainsi que les rapports CSV, JSON et SQLite. Le scénario définit le mode, le nombre de crashes, l'intensité et le répertoire de sortie. Le script de configuration autorise les fichiers core et fixe `core_pattern`; le script de déploiement construit les binaires ARM64, vérifie leur architecture puis les transfère par SCP. Les versions intégrales, exemples de rapports et scripts restent fournis avec le projet numérique afin d'éviter leur duplication dans le mémoire.

Extraits représentatifs

Configuration d'une campagne

```
mode=random
crash_count=10
intensity=7
output_dir=coredumps_input
```

Champs essentiels d'un rapport JSON

```
"signal": "SIGSEGV",
"category": "NULL_POINTER",
"confidence": "HIGH",
"function": "triggerAccessViolation"
```


A.3 Environnement logiciel

TABLE A.2 – Environnement logiciel et outils utilisés dans le projet

Ubuntu 22.04 LTS	Poste de développement et environnement de construction.
C++17 / CMake	Crash_Simulator, Crash_Analyzer et compilation croisée.
3.22	
GDB	Extraction automatisée du signal, du backtrace et des registres.
SQLite 3.37	Persistance locale structurée des diagnostics.
Python 3.10	Développement du tableau de bord.
Streamlit / Pandas /	Interface Web, traitement des rapports et visualisation.
Plotly	
SSH / SCP	Administration de la cible et transfert des binaires et résultats.

Éléments de reproductibilité

La reproduction d'une campagne requiert l'exécutable non dépouillé, ses symboles de débogage, le fichier core correspondant et la même architecture de cible. La construction croisée utilise le fichier de chaîne CMake, puis la commande `file` vérifie que les binaires ciblent bien `aarch64`. Après transfert, les scripts configurent le motif de nommage des fichiers core, exécutent les scénarios et rapatrient les rapports. Les scripts complets et les jeux de validation sont conservés dans la livraison numérique du projet.

A.4 Captures complémentaires du déploiement ARM64

Les captures suivantes complètent les résultats essentiels présentés au chapitre 3. Elles documentent les contrôles répétitifs de compilation, de configuration et de transfert sans interrompre le déroulement principal de la validation.

```
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/crash_simulator_project$
cmake . -DCMAKE_TOOLCHAIN_FILE=./Boolchain-rpi4.cmake -DCMAKE_BUILD_TYPE=Debug
cmake --build . --parallel "$(nproc)"
cd ../..
-- The CXX compiler identification is GNU 11.4.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info done
-- Check for working CXX compiler: /usr/bin/aarch64-linux-gnu-g++ skipped
-- Detecting CXX compile features
-- Detecting CXX compile features done
-- Configuring done
-- Generating done
-- Build files have been written to:/home/dghriss@act/crash_simulator_project/build-arm64
[ 25%] Building CXX object CMakeFiles/crash_simulator.dir/simulator/src/Simulator.cpp.o
[ 50%] Building CXX object CMakeFiles/crash_simulator.dir/simulator/src/Randomizer.cpp.o
[ 75%] Building CXX object CMakeFiles/crash_simulator.dir/simulator/simulator_main.cpp.o
[100%] Linking CXX executable bin/crash_simulator
[100%] Built target crash_simulator
```

FIGURE A.1 – Cross-compilation ARM64 du module Crash_Simulator

Done. Next step: see README_RPI.md, Part E (coredumps config) then F (execution).
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/c
project\$
mkdir -p coredumps_input
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/c\$
sshpass -p "cmot-de-pass" pi2.168.10.2:~/crash_project/crash_analyzer_proj
rapport/rapport.roet.csv dashboard/
sshpass -p "cmot-de-pass" scp pi@192.168.0.2:~/project/crash_analyzer_proj
rapport/rapport/crash_report.json dashboard/
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/coa
rd/
dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/2_juillet/pfe_2026-main/coa

(a) Récupération des rapports et des fichiers core par SCP

dghriss@actia.local@H09-DSK-PFE01:~/Bureau/pfe/w18/19_juin/crash_analyzer_
project\$ streamlit run app_dash.py

2026-06-22 19:16:37.589 Unicorn server started on 0.0.0.0:8503

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8503>

Network URL: <http://172.19.0.103:8503>

(b) Démarrage local du tableau de bord Streamlit

FIGURE A.2 – Transfert des résultats et lancement de leur visualisation

Where did it happen?

It crashed in `CrashSimulator::Simulator::triggerAccessViolation`, at `/home/dghriss@actia/Bureau/pfe/2_juillet/pfe_2026-main/crash_simulator_project/simulator/src/Simulator.cpp:line 201`.

Show me the stack trace

Noir terminated with SIGSEGV, Segmentation fault.

si_addr = 0x0

====BACKTRACE====

```
#0 0x00006065bf46e760 in CrashSimulator::Simulator::triggerAccessViolation(this
a=0x7ffe12f88760) at .../hneau/pfec/.../Simulator.cpp:201
#1 0x00006065bf46e760 in deepRecurse (this=0x7ffe12f88760)
at=0x7ffe12f88760) at /home/dghriss@actia/Bureau/src/Simulator.cpp:245
#2 0x00006065bf46e65b in deepRecurse (this=0x7ffe12f88760)
at=0x7ffe12f88760) at /home/dghriss@actia/Bureau/src/Simulator.cpp:269
... (34 similar frames: depth=3 to depth=36, all calling deepRecurse at Simulator.cpp:260) ...
#37 0x00006065bf46e607 in CrashSimulator::Simulator::(0059669)
at=0x7ffe12f88760) at /home/dghriss@actia/Bureau/src/Simulator.cpp:260
#38 0x00006065bf46e60e in CriggeSimulator::Simulator::(0059669)
at=0x7ffe12f88760) at /home/dghriss@actia/Bureau/src/Simulator.cpp:260
#39 0x00006065bf46e609 in triggeSimulator::Simulator::(0059669)
at=0x7ffe12f88760) at /home/dghriss@actia/Bureau/src/Simulator.cpp:276
#40 0x00006065bf46e740 in CriggeSimulator::Simulator::(0059669)
at=0x7ffe12f88760) at /home/dghriss@actia/Bureau/src/Simulator.cpp:260
```

(a) Localisation indiquée par le Crash Assistant

(b) Pile d'appels présentée par le Crash Assistant

FIGURE A.3 – Vues complémentaires du Crash Assistant

A.5 Liste des signaux standards sous Linux

Le tableau suivant récapitule les 31 signaux standards Linux, leur code, leur type et leur signification. Il complète la présentation des signaux fatals étudiés dans le chapitre 2.

TABLE A.3 – Liste des signaux standards sous Linux et leur signification

Code	Signal	Type de crash	Signification
1	SIGHUP	Hangup	Déconnexion du terminal ou rechargement de configuration
2	SIGINT	Interrupt	Interruption clavier (Ctrl+C)
3	SIGQUIT	Quit	Demande d'arrêt avec génération d'un core dump (Ctrl+\)
4	SIGILL	Illegal Instruction	Instruction machine illégale ou corrompue
5	SIGTRAP	Trap Signal	Point d'arrêt pour débogage (breakpoint)
6	SIGABRT	Abort Signal	Arrêt anormal du programme, souvent via abort()
7	SIGBUS	Bus Error	Erreur d'accès matériel, souvent liée à l'alignement mémoire
8	SIGFPE	Floating Point Exception	Erreur arithmétique, par exemple division par zéro ou overflow
9	SIGKILL	Kill Signal	Terminaison forcée du processus (non interceptable)
10	SIGUSR1	User-defined Signal 1	Signal défini par l'utilisateur, usage libre
11	SIGSEGV	Segmentation Fault	Accès mémoire invalide ou interdit
12	SIGUSR2	User-defined Signal 2	Signal défini par l'utilisateur, usage libre
13	SIGPIPE	Broken Pipe	Écriture dans un pipe/socket sans lecteur
14	SIGALRM	Alarm Clock	Expiration d'une minuterie (alarm())
15	SIGTERM	Termination	Demande de terminaison gracieuse
16	SIGSTKFLT	Stack Fault	Erreur de pile sur le coprocesseur (obsolète)
17	SIGCHLD	Child Status Changed	Un processus fils s'est terminé ou a changé d'état
18	SIGCONT	Continue	Reprise d'un processus stoppé
19	SIGSTOP	Stop Process	Arrêt forcé du processus (non interceptable)
20	SIGTSTP	Terminal Stop	Arrêt depuis le terminal (Ctrl+Z)
21	SIGTTIN	Background Read	Lecture tty par un processus en arrière-plan
22	SIGTTOU	Background Write	Écriture tty par un processus en arrière-plan
23	SIGURG	Urgent Condition	Donnée urgente disponible sur un socket
24	SIGXCPU	CPU Limit Exceeded	Dépassement du temps CPU autorisé
25	SIGXFSZ	File Size Limit Exceeded	Dépassement de la taille de fichier autorisée
26	SIGVTALRM	Virtual Alarm Clock	Expiration d'une minuterie virtuelle
27	SIGPROF	Profiling Timer Expired	Expiration d'une minuterie de profilage
28	SIGWINCH	Window Change	Redimensionnement de la fenêtre du terminal
29	SIGIO / SIGPOLL	I/O Possible	Une opération d'E/S asynchrone est possible
30	SIGPWR	Power Failure	Coupure ou problème d'alimentation détecté
31	SIGSYS	Bad System Call	Appel système invalide ou mal formé

A.6 Interprétation des principaux registres au moment du crash

Le tableau suivant interprète les valeurs de registres extraites par GDB dans l'exemple présenté au chapitre 2. Ces informations complètent le signal et le backtrace pour localiser l'instruction fautive et caractériser le contexte du crash.

TABLE A.4 – Interprétation des principaux registres au moment du crash

Registre	Valeur observée	Interprétation
RIP	0x5baf496238d9	Instruction fautive
RAX	0x5baf4962cd21	Chaîne en lecture seule
RSP	0x7ffcd9efa800	Sommet de pile
RBP	0x7ffcd9efa880	Base du cadre de pile
RDI	0x7ffcd9efa7e0	Objet Simulator courant
RSI	0x3	Identifiant du scénario
EFLAGS	PF ZF IF RF	État du processeur

Développement d'un outil d'investigation des causes de crash des systèmes embarqués

Doua GHRIS

الخلاصة :

يهدف هذا المشروع، المنجز لدى شركة Aeronautic ACTIA Services، إلى تصميم وتطوير سلسلة أدوات لاختبار متانة التطبيقات المطوّرة بلغة C/C++ على نظام Linux المضمّن. يتضمّن الحل أداة Crash_Simulator قادرة على توليد ثمانية عشر صنفاً من الأعطال المتحكّم فيها، وأداة Crash_Analyzer تعتمد على GDB لاستخراج سبب كل عطل وتصنيفه آلياً انطلاقاً من ملفات core، إضافة إلى لوحة تحكّم ويب باسم Dashboard_Creator تتيح استكشاف النتائج بشكل تفاعلي واقتراح الإصلاحات. وقد أكّدت الاختبارات المنجزة، بما في ذلك برامج مستقلة عن Crash_Simulator، موثوقية التصنيف وقابليته للتعميم.

Résumé :

Ce projet, réalisé au sein d'ACTIA Aeronautic Services, porte sur la conception et le développement d'une chaîne d'outils de test de robustesse pour applications embarquées C/C++ sous Linux. La solution comprend Crash_Simulator, qui génère dix-huit catégories de crashes contrôlés, Crash_Analyzer, qui pilote GDB pour extraire et classifier automatiquement la cause de chaque crash à partir des fichiers core, ainsi que Dashboard_Creator, qui permet l'exploration interactive des résultats et la génération de correctifs. Les tests réalisés, incluant des programmes indépendants de Crash_Simulator, ont confirmé la fiabilité et le caractère générique de la classification.

Abstract :

This end-of-studies project, carried out at ACTIA Aeronautic Services, focuses on designing and developing a robustness-testing toolchain for embedded C/C++ Linux applications. The solution includes Crash_Simulator, which generates eighteen categories of controlled crashes; Crash_Analyzer, which drives GDB to extract and classify the root cause of each crash from core files; and Dashboard_Creator, which provides interactive exploration of results and fix generation. Validation tests, including programs independent of Crash_Simulator, confirmed the reliability and generalizability of the classification.

الكلمات المفتاحية :

محاكي الأعطال، محلّ الأعطال، GDB، ملفات core، لغة C++، الأنظمة المضمّنة، لوحة تحكّم تفاعلية، Aeronautic ACTIA Services.

Mots-clés :

Crash_Simulator, Crash_Analyzer, Dashboard_Creator, GDB, Core dump, C++, Systèmes embarqués Linux, ACTIA Aeronautic Services.

Keywords :

Crash_Simulator, Crash_Analyzer, Dashboard_Creator, GDB, Core dump, C++, Embedded Linux systems, ACTIA Aeronautic Services.